



COMSATS University Islamabad, Sahiwal Campus

Cura Hub

By

Abdullah Babar	CIIT/SP22-BCS-001/SWL
Bushra Athar Butt	CIIT/SP22-BCS-033/SWL
Haseeb Shakoor	CIIT/SP22-BCS-043/SWL

Supervisor

Dr. Javed Ferzund

Bachelor of Science in Computer Science (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University Islamabad, Sahiwal Campus

Cura Hub

**A project presented to
COMSATS Institute of Information Technology, Islamabad**

**In partial fulfillment
of the requirement for the degree of
*Bachelor of Science in Computer Science (2022-2026)***

By

Abdullah Babar	CIIT/SP22-BCS-001/SWL
Bushra Athar Butt	CIIT/SP22-BCS-033/SWL
Haseeb Shakoor	CIIT/SP22-BCS-043/SWL

DECLARATION

I hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that I have developed this software and the accompanying report entirely on the basis of my personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, I will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Abdullah Babar

Bushra Athar Butt

Haseeb Shakoor

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “CuraHub” was developed by **Abdullah Babar(CIIT/SP22-BCS-001/SWL)**, **Bushra Athar Butt(CIIT/SP22-BCS-033/SWL)** **Haseeb Shakoor(CIIT/SP22-BCS-043/SWL)** under the supervision of “**Javed Ferzund**” and that, in their opinion, it is fully adequate, in scope, and of quality for the degree of Bachelor of Science in Computer Science.

Supervisor

Dr. Javed Ferzund

Associate Professor,

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Prof. Dr. Imran Sarwar Bajwa

Associate Professor

Department of Computer Science

The Islamia University of Bahawalpur, Pakistan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

CuraHub is a comprehensive mobile healthcare platform that I have personally developed to create a unified, accessible, and intelligent space for patients seeking medical guidance. During my academic journey and research into modern healthcare challenges, I identified that many patients struggle with finding the right specialists, understanding their symptoms, managing appointments, and getting timely medical support. CuraHub is my solution to bring all these essential healthcare needs onto a single, centralized platform.

The platform is organized into multiple core modules: User Registration and Authentication, Doctor Directory and Profiles, Appointment Management, AI Chat Assistant, Machine Learning Disease Prediction, Health Education, and Social Posting. Each module is designed thoughtfully users can register securely, discover doctors by specialty, book chat or video consultations, receive AI-driven recommendations, explore educational medical content, and interact through the social feed. The Disease Prediction module uses machine-learning models such as Regression, ANN, and CNN to provide early screening insights and guide users toward the correct department. The HealthEducation section includes curated medical content that helps users learn about diseases, symptoms, and wellness in a simple and accessible way.

I developed the mobile application using Flutter with Dart, ensuring smooth UI/UX and cross-platform performance. Firebase is used for authentication and database management, providing real-time and secure data handling. For payments, I integrated Stripe through a Node.js backend, enabling users to make secure online transactions for consultations. The platform also includes an intelligent AI Chat Assistant that offers instant responses, symptom guidance, and support to users throughout their experience. This project reflects not only my technical expertise but also my dedication to solving real-world healthcare challenges. Every feature, integration, and line of code in CuraHub has been developed by me with focus, responsibility, and a strong commitment to improving the digital healthcare experience for users in a meaningful way.

Acknowledgement

All praise is to Almighty Allah, who bestowed upon us a minute portion of His boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor, “**Dr. Javed Ferzund**”. Without his supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to him for his encouragement and continual help during this work. We are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Abdullah Babar

Bushra Athar Butt

Haseeb Shakoor

Abbreviations

Abbreviation	Meaning
CH	CuraHub
UI	User Interface
UX	User Experience
UI/UX	User Interface/User Experience
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
LLM	Large Language Model
API	Application Programming Interface
SDK	Software Development Kit
JSON	JavaScript Object Notation
JWT	JSON Web Token
OTP	One-Time Password
DB	Database
NoSQL	Non-Relational Database
FCM	Firebase Cloud Messaging
FDB	Firebase Database
FS	Firestore
FR	Firebase Realtime Database
FAB	Firebase App Build
FAS	Firebase Authentication Service
FST	Firebase Storage

CRUD	Create, Read, Update, Delete
HCI	Human-Computer Interaction
SRS	Software Requirement Specification
SDD	Software Design Document
UML	Unified Modeling Language
UC	Use Case
AC	Actor
QA	Quality Assurance
UT	Unit Testing
IT	Integration Testing
FT	Functional Testing
UAT	User Acceptance Testing
IDE	Integrated Development Environment
OS	Operating System
APK	Android Package
RAM	Random Access Memory

Table of Contents

1	Introduction.....	1
1.1	Brief	1
1.1.1	User Registration and Patient Interaction.....	2
1.1.2	Medical Cases and Smart Health Consultations.....	2
1.1.3	Symptom Analysis and AI Recommendations.....	2
1.1.4	Health Learning Hub	3
1.1.5	Q&A Health Forum.....	3
1.1.6	Real-Time Chat & Consultation.....	3
1.2	Relevance to Course Modules	4
1.2.1	Mobile & Web App Development	4
1.2.2	Software Engineering Principles.....	4
1.2.3	Database Management Systems.....	4
1.2.4	Artificial Intelligence (AI)	4
1.2.5	Human-Computer Interaction (HCI).....	4
1.3	Project Background	5
1.4	Literature Review	6
1.5	Analysis from Literature Review.....	7
1.5.1	User Registration and Patient Interaction.....	7
1.5.2	Medical Cases & Consultations	7
1.5.3	Symptom Analysis AI.....	8
1.5.4	Health Learning Hub	8
1.5.5	Q&A Health Forum.....	8

1.5.6	Real-Time Chat & Consultation.....	8
1.6	Methodology and Software Lifecycle.....	9
1.6.1	Modular Development Approach.....	9
1.6.2	Object-Oriented Development (OOD)	9
1.6.3	Agile Approach	9
1.6.4	Rationale Behind the Selected Methodology	10
1.6.5	Software Development Life Cycle (SDLC).....	10
2	Problem Definition	13
2.1	Problem Statement.....	13
2.2	Deliverables Requirements.....	14
2.2.1	CuraHub Platform	14
2.2.2	Documentation	14
2.2.3	Testing Reports	14
2.2.4	Deployment Setup	15
2.3	Development Requirements	15
2.3.1	Technologies and Tools	15
2.3.2	Server and Hosting Requirements.....	15
2.3.3	Performance and Scalability.....	16
2.3.4	Security and Privacy	16
2.3.5	Development Milestones.....	16
2.3.6	Collaboration and Version Control	16
2.4	Current System	17
3	Requirement Analysis	19
3.1	Use Case 1	19
3.1.1	Use Case 1 Description	20

3.2 Use Case 2	22
3.2.1 Use Case 2 Description	22
3.3 Use Case 3	24
3.3.1 Use Case 3 Description	25
3.4 Use Case 4.....	26
3.4.1 Use Case 4 Description	26
3.5 Use Case 5.....	28
3.5.1 Use Case 5 Description	28
3.6 Use Case 6.....	29
3.6.1 Use Case 6 Description	30
3.7 Functional Requirements.....	31
3.7.1 User Registration and Authentication	32
3.7.2 Medical Cases & Consultations Module.....	32
3.7.3 Real-Time Chat System	32
3.7.4 Q&A Health Forum.....	32
3.7.5 Bug Detection and Reporting.....	33
3.7.6 Learning Section	33
3.7.7 Medical Community.....	33
3.7.8 Admin Panel and Content Moderation.....	33
3.8 Non-Functional Requirements.....	33
3.8.1 Scalability.....	34
3.8.2 Usability	34
3.8.3 Performance	34
3.8.4 Security and Privacy	34
3.8.5 Maintainability	35

3.8.6	Compatibility.....	35
4	Design and Architecture	37
4.1	System Architecture	37
4.2	Data Representation.....	38
4.3	Process Flow.....	39
4.3.1	Activity Diagrams	39
4.4	Design Models.....	46
4.4.1	Sequence Diagram.....	46
4.4.2	Class Diagram	51
5	Implementation	56
5.1	Algorithm	56
5.1.1	User authentication.....	56
5.1.2	Social Posting.....	57
5.1.3	Interactive Health Consultations	57
5.1.4	Live Consultation Simulation.....	58
5.1.5	Real-Time Chat & Messaging Algorithm	58
5.2	External APIs	59
5.3	User Interface	61
5.3.1	Splash Screen	61
5.3.2	Entry Screen	62
5.3.3	User Sign up.....	63
5.3.4	Doctor Sign up	64
5.3.5	Patient Home Screen	65
5.3.6	Booking Appointment	66
5.3.7	Select Appointment Time Screen.....	67

5.3.8	Favorite Doctor Screen	68
5.3.9	Appointment Management Screen	69
5.3.10	Setting Screen.....	70
5.3.11	Notification Settings Screen.....	71
5.3.12	Change Password Screen	72
5.3.13	Password Reset Screen.....	73
5.3.14	Account Deleting Warning	74
5.3.15	AI Specialist Screen	75
5.3.16	AI Medical Assistant Screen	76
5.3.17	Specialized AI Doctor	77
5.3.18	AI Specialist Screen	78
5.3.19	Account Management Screen	79
5.3.20	Patient Profile Screen	80
5.3.21	Privacy Terms and Conditions Screen	81
5.3.22	Help Center Screen	82
5.3.23	Contact Us Screen	83
5.3.24	Logout Confirmation Screen.....	84
5.3.25	Suggested Specialist Screen.....	85
5.3.26	Cancel Confirmation Appointment Screen	86
5.3.27	Doctor Patient Chat Screen	87
5.3.28	Patient Conversation List Screen	88
5.3.29	Patient Profile Screen.....	89
6	Testing and Evaluation	91
6.1	Manual Testing.....	91
6.1.1	System Testing.....	91

6.1.2	Unit Testing.....	92
6.1.3	Functional Testing.....	93
6.1.4	Integration Testing	94
6.2	Tools and Technologies.....	96
7	Conclusion	99
7.1	Future Work	99
7.1.1	Advanced AI Capabilities for Code Assistance	99
7.1.2	Real-Time Chat Enhancement	100
7.1.3	Mobile Application	100
7.1.4	Enhanced Learning Resources and Medical Case Simulations	100
7.1.5	Integration with More Code Repositories.....	100
8	References.....	102

List of Figures

Figure 3.1: User Management	19
Figure 3.2: Doctor & Department Management.....	22
Figure 3.3: Appointments and Scheduling	24
Figure 3.4: Chat Interaction.....	26
Figure 3.5: Call Functionality	28
Figure 3.6: Payment Management.....	30
Figure 4.1: CCH architecture.....	37
Figure 4.2: User Registration and patient interaction activity diagram.....	40
Figure 4.3: Medical Cases and Consultation Activity Diagram.....	42
Figure 4.4: System Analysis and AI Recommendation Activity Diagram.....	43
Figure 4.5: Q&A Health Forum Activity Diagram	44
Figure 4.6: Real-Time Chat Activity Diagram.....	45
Figure 4.7: Sequence Diagram for AI Symptom Checker.....	46
Figure 4.8: Sequence Diagram for appointment booking and scheduling.....	47
Figure 4.9: Sequence diagram for Medical Consultation	48
Figure 4.10: Sequence diagram for Medical Records Management.....	49
Figure 4.11: Sequence diagram for real-time chat.....	50
Figure 4.12: Class Diagram.....	52
Figure 5.1: Splash Screen	61
Figure 5.2: Entry Screen.....	62
Figure 5.3: Login Screen	63
Figure 5.4: Doctor Sign Up	64
Figure 5.5: Patient Home Screen.....	65
Figure 5.6: Booking Appointment.....	66
Figure 5.7: Select Appointment Time screen	67
Figure 5.8: Favorite Doctor screen	68
Figure 5.9: Appointment Management screen	69
Figure 5.10: Setting screen	70
Figure 5.11: Notification Setting Screen	71
Figure 5.12: Change Password Screen	72

Figure 5.13: Password Reset Screen.....	73
Figure 5.14: Account Deleting Screen	74
Figure 5.15: AI Specialist screen.....	75
Figure 5.16: AI Medical Assistant Screen.....	76
Figure 5.17: Specialized AI Doctor.....	77
Figure 5.18: AI Specialist Screen	78
Figure 5.19: Account Management screen	79
Figure 5.20: Patient Profile Screen.....	80
Figure 5.21: Privacy Terms and Conditions Screen	81
Figure 5.22: Help Center Screen	82
Figure 5.23: Contact Us Screen.....	83
Figure 5.24: Logout Confirmation Screen.....	84
Figure 5.25: Suggested Specialist Screen.....	85
Figure 5.26: Cancel Confirmation Appointment.....	86
Figure 5.27: Doctor Patient Chat Screen.....	87
Figure 5.28: Doctor Patient Chat List Screen.....	88
Figure 5.29: Patient Profile Screen.....	89

List of Tables

Table 1.1: Literature Review	6
Table 3.1: Authentication, Registration, and Posting	20
Table 3.2: Doctor & Department Management	22
Table 3.3: Appointments and Scheduling	25
Table 3.4: Chat Interaction	27
Table 3.5: Call Functionality	28
Table 3.6: Payment Management	30
Table 5.1: External APIs.....	60
Table 6.1: Admin Login Test Cases	92
Table 6.2: Login as User.....	93
Table 6.3: Logout as an Admin	93
Table 6.4: Logout as a User	93
Table 6.5: Log in as different roles.....	94
Table 6.6: Integration Testing.....	95
Table 6.7: Tools and Technologies for Proposed Project.....	97

Chapter 1

INTRODUCTION

1 Introduction

CuraHub is a comprehensive mobile- and web-based healthcare platform that I developed to address the challenges patients and doctors face in communication, early diagnosis, and smart medical assistance. As digital healthcare becomes increasingly essential in today's world, there is a growing need for integrated platforms that combine medical consultation, AI-powered guidance, and multi-specialty collaboration. Existing platforms usually offer isolated features either doctor booking, chat assistance, or disease prediction without providing a unified healthcare experience. CuraHub brings all these features together into a single ecosystem, offering multi-specialty doctor access, symptom-based AI guidance, machine learning disease prediction, real-time chat and video consultation, and a centralized panel for appointments, reviews, and payments.

The platform is designed to serve users from diverse medical backgrounds, including patients seeking quick help, individuals monitoring chronic conditions, and doctors looking for an efficient digital workspace. It promotes accessible and collaborative healthcare by enabling users to consult verified specialists, ask questions, access AI-driven suggestions, and receive early disease risk assessment. By using modern technologies such as Flutter, Firebase Authentication, Firestore, Storage, Stripe Payment Gateway, and integrated AI/ML models, CuraHub creates a dynamic, intelligent, and user-friendly environment. This project aims to simplify healthcare processes, making them more interactive, predictive, and accessible for users who want reliable medical guidance from their mobile device.

The primary goal of CuraHub is to build a platform where users can quickly find relevant doctors, book secure appointments, consult through chat or video, receive symptom-based AI assistance, and access early disease predictions all in one place.

1.1 Brief

CuraHub is a smart healthcare platform designed to assist patients by offering multi-specialty doctor access, AI-based symptom guidance, disease prediction, secure appointments, and real-time communication. Built with Flutter (Dart), Firebase, Node.js, and integrated ML models such as Regression, ANN, and CNN, it serves users with diverse medical needs. Featuring AI-driven support, video consultation, and Stripe-based online payments, the platform creates an interactive environment for accessible, predictive, and collaborative healthcare. The CuraHub system is organized into six primary modules.

1.1.1 User Registration and Patient Interaction

The User Registration and Patient Interaction module enables users to easily sign up on the platform using Firebase email authentication or Google login. Once registered, users can create personalized health profiles, explore medical departments, and connect with specialist doctors. They can also book appointments, consult through real-time chat or video call, and receive AI-based symptom guidance according to their selected department. This module supports smooth interaction and allows patients to stay connected with healthcare professionals.

Admins have full management control, including monitoring user accounts, appointments, and medical interactions. They can approve or remove doctors, manage consultation records, and handle patient feedback to ensure system reliability. Admins can also manage reviews, medical reports, transactions, and communication logs, ensuring a secure, organized, and high-quality healthcare experience across the platform.

1.1.2 Medical Cases and Smart Health Consultations

The Medical Cases module is a core component of CuraHub, offering users the opportunity to explore a wide range of health-related scenarios tailored to different medical categories and complexity levels. Users can review multiple cases, understand symptoms, read recommended treatments, and compare AI-generated insights with real doctor guidance. This interactive space encourages learning, awareness, and improvement in personal health decision-making.

A unique feature of this module is the Smart Consultation section, where one doctor can host a live session and the user joins as a patient seeking guidance. The environment simulates a real medical consultation, with safeguards such as privacy protection, secure data handling, and auto-disconnection if suspicious activity is detected, ensuring safety and integrity. Admins manage the addition and categorization of medical cases, review AI-generated results, and maintain the accuracy and quality of the health information shared in this space.

1.1.3 Symptom Analysis and AI Recommendations

The Symptom Analysis AI helps users identify potential health issues by analyzing symptoms, medical reports, or health concerns to provide smart suggestions and guidance. Users can review recommendations, compare past consultations, or consult doctors via chat or video. Admins oversee submissions, manage AI outputs, verify users, and ensure a secure and accurate healthcare environment.

1.1.4 Health Learning Hub

The Health Learning Hub is a dedicated space within CuraHub that acts as a comprehensive knowledge center for users looking to improve their understanding of medical conditions, preventive care, and wellness practices. It offers a wide range of educational resources, including downloadable health guides, interactive tutorials, and embedded video lessons across multiple medical specialties. Users can explore content, track their progress, and learn at their own pace, making it ideal for self-guided health education.

Each medical topic is supported with structured materials, allowing users to download guides and understand symptoms, treatment options, and preventive measures clearly. Upon completion of specific tutorials or learning paths, users may also receive a digital certificate acknowledging their achievement. Admins play a crucial role in this module by curating content, organizing tutorials by medical department or topic, and ensuring the materials remain accurate, current, and relevant for users of all knowledge levels.

1.1.5 Q&A Health Forum

The Q&A Health Forum module provides a collaborative space for users to ask medical and health-related questions and receive answers from certified doctors or the community. It encourages knowledge sharing and patient engagement, allowing users to interact and support each other in understanding symptoms, treatments, and wellness practices. Each answer can be upvoted by users, ensuring that the most accurate and helpful responses are highlighted.

In addition to viewing answers, users can comment on responses to request further clarification or engage in discussions about health concerns. The forum also tracks the total number of views and answers for each question, providing insight into common health topics. Admins actively monitor the forum to ensure discussions remain respectful, medically accurate, and on-topic. They manage inappropriate content and have the authority to remove posts or restrict users who violate the platform's guidelines, maintaining a safe and professional environment.

1.1.6 Real-Time Chat & Consultation

The Real-Time Chat module is designed to enable instant communication between patients and doctors, allowing them to send private messages, create consultation groups, and collaborate on health concerns in real time. Users can share medical reports, images, or documents by sending attachments. They can also view previous chat histories for ongoing consultations and delete messages as needed.

1.2 Relevance to Course Modules

CuraHub brings together a wide range of concepts I've learned throughout my Bachelor of Computer Science degree. This project is a reflection of how classroom knowledge can be transformed into a practical, real-world healthcare platform.

1.2.1 Mobile & Web App Development

The front end of CuraHub was built using Flutter with Dart, skills I gained through our mobile and web development courses. These courses helped me design a responsive and user-friendly interface that ensures a smooth experience across all devices[1].

1.2.2 Software Engineering Principles

The structure and planning of CuraHub heavily relied on principles learned in software engineering. Concepts such as requirement gathering, modular design, and the incremental development model were crucial in managing the platform's complexity and ensuring maintainability.

1.2.3 Database Management Systems

This course enabled me to design and implement the project's database efficiently. I used Firebase Firestore and Firebase Authentication to manage user profiles, doctor records, appointments, consultations, AI inputs, and other core components in a structured and optimized way.

1.2.4 Artificial Intelligence (AI)

AI played a role in the automated symptom analysis and disease prediction features within the platform. My understanding of AI concepts helped me integrate intelligent behavior into the system, such as generating smart health suggestions and preliminary diagnostic guidance to assist users.

1.2.5 Human-Computer Interaction (HCI)

HCI principles guided me in creating an intuitive and accessible user interface. From layout decisions to interaction flow, I applied usability techniques to ensure the platform feels engaging, especially for patients and users who may be less tech-savvy.

1.3 Project Background

The idea for CuraHub emerged from my own academic and practical journey as a computer science student. Like many others seeking healthcare services, I often found myself navigating multiple platforms just to access one complete healthcare solution one for booking appointments, another for consulting doctors, a separate one for health information, and yet another for AI-based guidance. This fragmented experience was not only time-consuming but also frustrating, especially for patients who may struggle with technology. That's when I began to think: What if there was one centralized platform that brought all these essential healthcare tools and support systems together in one place?

CuraHub was conceptualized to solve this very problem. It's a comprehensive, mobile- and web-based platform that caters to a wide range of healthcare needs. From providing multi-specialty doctor access and AI-powered symptom analysis to offering ML-based disease prediction and real-time chat/video consultation, the platform is built to support users at every stage. This fragmented experience was not only time-consuming but also frustrating, especially for patients who may struggle with technology. Whether someone is seeking preventive care, managing chronic conditions, or consulting specialists, the features of this platform aim to create a continuous and connected healthcare experience. CuraHub is not just a technical project—it is a solution born out of real problems that I and many others faced while accessing healthcare services

The inclusion of real-time chat functionalities, AI-based guidance, and patient-doctor interaction tools was inspired by how healthcare users often benefit most—through timely support and collaborative communication. Accessing healthcare doesn't have to be fragmented. With CuraHub, I wanted to build a digital space where patients can receive intelligent guidance, consult doctors, and manage their health efficiently. The AI and ML integration on the platform is another step toward making healthcare more predictive, accurate, and personalized[2].

Moreover, by enabling features like downloadable health reports, consultation tracking, and secure Stripe-based payments, the platform not only improves convenience but also ensures safety and reliability. Admin functionalities further ensure the platform remains organized, moderated, and resourceful, managing doctors, appointments, transactions, and patient feedback effectively. In essence, CuraHub is not just a technical project—it is a solution born out of real problems that I and many others faced while accessing healthcare services.

1.4 Literature Review

Table 1.1 summarizes gaps in existing healthcare platforms like Oladoc, Marham, Sehat Kahani, and WebMD, highlighting issues such as limited AI integration, lack of disease prediction, and minimal patient-doctor collaboration. CuraHub addresses these by offering an all-in-one solution with AI-powered symptom guidance, real-time chat/video consultation, multi-specialty access, and secure payment integration to enhance the digital healthcare experience.

Table 1.1: Literature Review

Websites	Weakness	Proposed Project Solution (CuraHub)
Oladoc	Focused mainly on doctor booking; lacks AI symptom analysis or disease prediction features.	Integrated AI symptom checker and ML models (Regression, ANN, CNN) for early diagnosis and intelligent guidance.
Marham	Limited real-time chat support and no predictive medical analytics.	Real-time chat/video consultation combined with AI-driven disease prediction and patient insights.
Sehat Kahani	Provides online consultations but lacks multi-department AI support and centralized patient history.	Multi-department platform with AI chat assistants, Firebase-backed health records, and unified patient history.
WebMD	Offers health information but does not provide real-time doctor interaction or appointment booking.	One-stop platform offering doctor search, appointments, secure messaging, and AI-based suggestions.

1.5 Analysis from Literature Review

The analysis of existing platforms and trends in digital healthcare reveals several gaps that CuraHub aims to address. Many current platforms offer individual features like doctor booking, health information, or basic consultation but fail to integrate these elements into a cohesive experience. Our literature review identifies limitations such as the lack of AI-powered symptom analysis, predictive disease models, real-time doctor-patient collaboration, and a centralized management system. CuraHub bridges these gaps by providing a unified platform that integrates AI-based symptom guidance, ML-driven disease prediction, real-time chat/video consultations, and a secure, multi-specialty healthcare ecosystem. Below is a detailed comparison based on our findings.

1.5.1 User Registration and Patient Interaction

While many healthcare platforms allow users to register and access basic services, they often lack seamless integration of interactive features that encourage patient engagement and collaboration. Platforms like Oladoc or Marham focus primarily on doctor booking but don't offer robust patient profiles or interaction tools. CuraHub enhances this by allowing users to easily register using Google or Email authentication and engage with doctors and other patients through secure consultations, health updates, and AI-guided symptom discussions. This module creates a space where users not only access healthcare services but can also track their health, receive guidance, and contribute to their wellness journey in meaningful ways. Admins can monitor interactions, ensuring accuracy and quality, making CuraHub more reliable, safe, and engaging.

1.5.2 Medical Cases & Consultations

Existing healthcare platforms often provide access to doctors or health information but lack interactive features that promote patient engagement and learning. These platforms typically do not allow users to explore medical cases, compare AI-generated suggestions, or discuss health concerns with doctors, which limits the overall experience. CuraHub addresses this by providing features such as reviewing medical case studies, receiving AI-assisted guidance, and participating in real-time consultations with doctors. The ability to explore multiple cases and engage in interactive consultations further enhances patient understanding and health management, setting CuraHub apart from conventional healthcare platforms[5].

1.5.3 Symptom Analysis AI

Traditional healthcare platforms offer basic symptom checkers but fail to integrate AI or real-time guidance. Many tools allow users to input symptoms, but there's little personalized support or predictive insights. CuraHub uses an AI-driven symptom analysis system to not only identify possible conditions but also assist users with intelligent recommendations in real time. The platform allows users to discuss their symptoms with doctors, receive AI suggestions, and track trends in health data, providing a more interactive and insightful healthcare experience.

1.5.4 Health Learning Hub

Many healthcare education platforms offer structured tutorials and health guides but lack personalized learning paths or practical interaction with real-world health scenarios. While these platforms focus on providing information, they do not allow users to test their understanding or engage with interactive case studies. CuraHub integrates tutorials across multiple medical specialties with AI-assisted symptom analysis and patient case simulations. Users can access downloadable health guides, track their progress, and improve their knowledge through interactive exercises, making it a more dynamic and comprehensive health learning experience.

1.5.5 Q&A Health Forum

Platforms like health forums provide spaces for users to ask and answer medical questions, but they often suffer from overwhelming content and difficulty in navigating the most relevant responses. CuraHub enhances the Q&A experience by offering a streamlined interface where users can see total views, answers, and comment on responses. Users can also engage in deeper discussions with doctors or other patients, making the Q&A process more interactive and engaging. Admins monitor the forum, ensuring the content stays accurate and respectful, and can intervene if needed. This module promotes a collaborative healthcare environment, making it easier for users to find reliable guidance and engage in meaningful discussions.

1.5.6 Real-Time Chat & Consultation

While many healthcare platforms allow users to communicate, they often lack comprehensive features for secure consultations and content sharing. CuraHub provides an integrated real-time chat system where patients can communicate privately with doctors, create group consultations, share medical reports or images, and maintain their chat history. The ability to block users, manage profiles, and delete messages ensures a safe and controlled communication environment.

1.6 Methodology and Software Lifecycle

For the development of the CuraHub project, I have chosen a modular, object-oriented, and incremental development approach. The reason for selecting this methodology is to ensure clear structure, flexibility, and the ability to scale the platform as it grows. This approach allows each module such as patient registration, AI symptom analysis, real-time consultations, and payment integration to be developed, tested, and improved independently, ensuring a smooth development process and easy debugging.

1.6.1 Modular Development Approach

The project is divided into individual modules such as User Registration, Medical Cases & Consultations, Symptom Analysis AI, Health Learning Hub, Q&A Health Forum, and Real-Time Chat & Consultation. Each module is developed as a self-contained unit, with its own functionality and features. This modular structure allows for focused development and testing on specific features without affecting the rest of the system. It also makes it easier to manage and modify the platform as needed, as changes can be made in one module without disrupting others.[6].

1.6.2 Object-Oriented Development (OOD)

The project leverages object-oriented programming (OOP) principles such as classes, objects, and methods to design the system. By designing the system using classes like User, Doctor, Appointment, MedicalCase, and Consultation, the code becomes more organized, reusable, and easier to maintain. OOP also enables better data encapsulation and abstraction, ensuring that only relevant data is exposed to other parts of the platform.

1.6.3 Agile Approach

The CuraHub adopts the Agile Software Development Model, specifically the Scrum framework, to manage the project's dynamic and evolving requirements. Agile enables the development team to work in iterative cycles called sprints, typically lasting 1–2 weeks, during which specific features such as doctor registration, AI chatbot integration, or appointment booking are designed, developed, and tested. The process includes regular sprint planning, daily stand-up meetings, sprint reviews, and retrospectives, ensuring continuous communication among team members and stakeholders. Agile promotes early delivery of working modules, quick adaptation to feedback, and seamless incorporation of changes, which is crucial for a user-centric healthcare platform. This model ensures that CuraHub remains flexible, responsive to user needs.

1.6.4 Rationale Behind the Selected Methodology

The combination of modular development, object-oriented programming, and an incremental approach was chosen for the following reasons: Modular development helps to break the project down into smaller, more manageable components, making the development process more organized and efficient. Object-oriented programming encourages code reusability, maintainability, and organization, which is essential in a dynamic mobile application like CuraHub that may require future updates and enhancements. The incremental approach ensures that progress is made continuously, with each phase producing a working product. This makes it easier to test and refine each feature as the project evolves. This methodology also allows for easier error detection and debugging since each module, such as User Registration, AI Symptom Analysis, Medical Consultations, Q&A Health Forum, Real-Time Chat, and Stripe Payments, is developed and tested separately before integration.

1.6.5 Software Development Life Cycle (SDLC)

For the Software Development Life Cycle (SDLC), the Incremental Model was selected. This model divides the CuraHub project into smaller, more manageable parts or increments, with each increment adding functionality to the platform, such as User Registration, Medical Cases & Consultations, AI Symptom Analysis, Q&A Health Forum, Real-Time Chat, and Stripe Payment Integration[9].

1.6.5.1 Planning and Requirements Gathering

During this phase, the requirements for each module were gathered and defined, along with the necessary technologies, tools, and features. This includes understanding the needs of users for features such as Q&A Health Forum, Medical Cases & Consultations, AI Symptom Analysis, Real-Time Chat & Consultation, and Stripe Payment Integration.

1.6.5.2 Design

Based on the gathered requirements, a detailed design for each module was created. This design defines the data structure, user interface, and functionality of each feature. The object-oriented design approach was used to define the structure of the CuraHub application and the relationships between the different modules, such as User Registration, Medical Cases & Consultations, AI Symptom Analysis, Q&A Health Forum, Real-Time Chat & Consultation, and Stripe Payment Integration.

1.6.5.3 Implementation

Each module is implemented in small, incremental stages. The first version of a module is developed and tested before moving on to the next one. The implementation phase follows the modular and object-oriented design in Flutter/Dart, ensuring the code is maintainable and organized. Modules include User Registration, Medical Cases & Consultations, AI Symptom Analysis, Q&A Health Forum, Real-Time Chat & Consultation, and Stripe Payment organized.

1.6.5.4 Testing

After each increment is completed, thorough testing is performed to ensure the feature works as expected. This includes unit testing, integration testing, and user acceptance testing (UAT) to verify the functionality and user experience. Testing occurs incrementally, allowing issues to be detected and addressed early in the development process. Modules tested include User Registration, Medical Cases & Consultations, AI Symptom Analysis, Q&A Health Forum, Real-Time Chat & Consultation, and Stripe Payment Integration[3].

1.6.5.5 Deployment and Maintenance

After each increment is tested, it is deployed to the CuraHub platform, ensuring that each new feature or update is fully functional before being released. Maintenance involves addressing any bugs, updating features, or adding enhancements to the system over time. Modules covered include User Registration, Medical Cases & Consultations, AI Symptom Analysis, Q&A Health Forum, Real-Time Chat & Consultation, and Stripe Payment Integration.

Chapter 2

PROBLEM DEFINITION

2 Problem Definition

A problem statement is a concise description of an issue to address or a condition to be improved upon. It identifies the gap between the current state and desired state of a process and product. For CuraHub, the problem is the lack of a unified digital healthcare platform for patients to consult doctors, access AI-assisted disease detection, manage appointments, and make payments efficiently.

2.1 Problem Statement

In the rapidly evolving world of digital healthcare, patients often struggle to find a cohesive and centralized platform that supports their medical consultation, disease detection, and healthcare management needs. Most existing platforms offer isolated features, such as doctor directories, appointment booking systems, or health information portals, but rarely combine all necessary components into a single, integrated system. As a result, users must navigate multiple disconnected tools and websites to manage their health effectively. This fragmentation not only hampers efficiency but also discourages continuous monitoring and timely medical intervention.

Moreover, many of these platforms lack intelligent features such as AI-powered disease prediction, personalized health suggestions, or symptom analysis. Communication tools provided are typically limited, offering no real-time chat or video consultation system to facilitate quick help or collaborative decision-making. Additionally, users cannot easily track their medical history, store health records in one place, or receive reminders for follow-ups. Admin-level control is often minimal, leaving management of doctors, appointments, and reviews inefficient, while user-level features such as privacy controls and interaction management are usually non-existent.

The absence of a unified, AI-powered healthcare ecosystem results in scattered experiences, reduced engagement, and missed opportunities for early diagnosis, efficient communication, and effective health management. These shortcomings highlight the urgent need for a comprehensive, interactive, and intelligent platform tailored to the modern-day challenges of healthcare access and management[4].

CuraHub aims to resolve these issues by offering a centralized platform that includes real-time chat and video consultation, AI-assisted symptom analysis and disease prediction, Q&A forums with doctors, a resource-rich health education section, appointment scheduling, and Stripe-based payment integration all integrated into one system.

2.2 Deliverables Requirements

2.2.1 CuraHub Platform

The project aims to develop a fully functional mobile/app-based healthcare platform offering a range of features to enhance user engagement and accessibility. Users can register, create profiles, and authenticate via email or social logins. They can consult doctors, view medical specialists' profiles, and access AI-assisted disease prediction and symptom analysis. The platform also includes a Q&A Health Forum for medical inquiries, appointment scheduling, and real-time chat or video consultation with doctors. Additionally, an admin panel allows for managing users, doctors, appointments, reviews, AI model configurations, and content moderation.

2.2.2 Documentation

User Documentation: A comprehensive guide for users explaining how to use the platform effectively, including features such as account management, doctor consultation booking, AI symptom analysis, real-time chat and video consultation, accessing medical resources, and managing appointments.

Technical Documentation: Detailed documentation outlining the CuraHub architecture, Firebase database schema, AI model API integrations, Stripe payment integration, Flutter/Dart code structure, and deployment process.

Admin Guide: Instructions for administrators on how to manage users, doctors, appointments, reviews, AI model configurations, content moderation, and overall system maintenance tasks in CuraHub.

2.2.3 Testing Reports

Comprehensive testing reports detailing functionality tests (unit and integration tests), performance tests, and user acceptance testing (UAT). Bug reports and resolutions for all identified issues during the testing phases, covering modules such as User Registration, Medical Consultations, AI Symptom Analysis, Q&A Health Forum, Real-Time Chat & Video Consultation, and Stripe Payment Integration.

2.2.4 Deployment Setup

A fully deployed and live version of the CuraHub platform is hosted on a secure server with all necessary configurations for smooth operation. Documentation of the deployment process includes server setup, domain management, Firebase configuration, Stripe integration, AI model API deployment, and backup protocols.

2.3 Development Requirements

2.3.1 Technologies and Tools

Frontend

The frontend will be built using Flutter with Dart for interactivity and functionality. Material Design and Flutter widgets will ensure a responsive and visually appealing design, while real-time communication features like chat and video consultation will be supported through Firebase Realtime Database and WebSockets.

Backend

The backend will be developed using Firebase for authentication, database management, and real-time functionalities. Node.js will be used for server-side operations such as Stripe payment integration and AI model API calls. Third-party APIs will be integrated for real-time consultation and AI-powered disease prediction. OAuth 2.0 will be employed for secure login options, such as Google and Email authentication. To ensure security, the system will implement input validation, encryption, secure data storage, and access control practices.

2.3.2 Server and Hosting Requirements

CuraHub will be deployed using Firebase Hosting for the mobile/web app backend and Node.js server for handling API calls and Stripe payment processing. Proper server configuration will ensure smooth handling of concurrent users, real-time chat, and video consultations. SSL certificates and Firebase security rules will be implemented for secure data transmission and user privacy. Cloud Storage will manage medical reports and images securely, while real-time updates will be handled through Firebase Cloud Messaging (FCM). Regular monitoring and logging will ensure system reliability and quick issue resolution.

2.3.3 Performance and Scalability

CuraHub will implement load balancing and caching mechanisms to ensure optimal performance, especially during peak usage periods. Firebase database optimization strategies will be applied to handle large volumes of data, such as user profiles, medical consultations, AI model logs, and appointment records. Real-time processing capabilities for chat, video consultations, and AI symptom analysis will be maintained with minimal latency to ensure smooth user experience.

2.3.4 Security and Privacy

CuraHub will implement strong user authentication methods to prevent unauthorized access. Role-based access control (RBAC) will ensure differentiated access rights between patients, doctors, and administrators. Sensitive user data, including medical records, passwords, and payment information, will be securely stored with encryption and Firebase security rules. GDPR-compliant and HIPAA-aligned data handling practices will be followed to ensure complete user privacy and confidentiality[5].

2.3.5 Development Milestones

The development milestones for CuraHub are divided into five phases. Phase 1 involves gathering requirements, creating user stories, and designing the initial platform wireframes. In Phase 2, core features such as user authentication, doctor profiles, appointment booking, AI symptom analysis, and the Q&A Health Forum will be developed. Phase 3 focuses on implementing real-time features like chat, video consultations, and AI-powered disease prediction modules. Phase 4 includes testing, bug fixing, and the deployment of the beta version. Finally, Phase 5 covers the final deployment and delivery of the platform, along with providing user documentation, admin guides, and training materials.

2.3.6 Collaboration and Version Control

CuraHub uses Git/GitHub for source code management, version tracking, and collaborative development among the three team members. This ensures smooth integration of features, proper code versioning, and effective teamwork throughout the development process.

2.4 Current System

At present, there is no single system that fully integrates the features necessary for a comprehensive healthcare collaboration platform like CuraHub. Existing systems, such as traditional hospital management software, telemedicine apps, and health forums, provide valuable services but fall short in combining essential tools for real-time consultations, AI-enhanced disease prediction, and collaborative patient-doctor interactions. Most current systems focus on isolated components like appointment booking, health records, or discussion boards but do not seamlessly integrate them into a cohesive platform that can support interactive health monitoring and personalized patient care at scale.

Existing telemedicine and e-health platforms typically provide static services such as video calls or document sharing, lacking interactivity and the dynamic elements required for effective patient engagement. Traditional hospital management systems often struggle with scalability and personalized healthcare delivery, as they mostly serve standardized workflows. Furthermore, Q&A health forums on these platforms tend to be static, where users post questions and answers without much real-time interaction, leaving room for more engaging features like live chat, AI-assisted symptom analysis, and multi-department collaboration.

In contrast, the proposed CuraHub system aims to combine the best aspects of these existing systems while addressing their key shortcomings. It will offer dynamic, interactive features, a real-time communication framework, AI-powered disease prediction and guidance, and collaborative tools for doctors and patients to foster effective healthcare delivery[6].

Chapter 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

The requirement of identifying techniques for creating a use case diagram for the CuraHub involves a structured process. It includes stakeholder analysis to understand user needs, followed by identifying and describing use cases, establishing relationships among them, and finally constructing the use case diagram to visualize system interactions. This approach ensures that all functional requirements are clearly defined and aligned with stakeholder expectations. It also provides a clear blueprint for developers to design and implement the system effectively.

3.1 Use Case 1

Figure 3.1 This use case diagram represents the core user account operations in the CuraHub platform. It includes actions performed by both users (doctors and patients) and administrators. Users can register, log in, log out, reset passwords, and verify emails. Admins are responsible for managing user accounts through the web dashboard. Relationships like <<include>> and <<extend>> illustrate dependent and optional processes within the system.

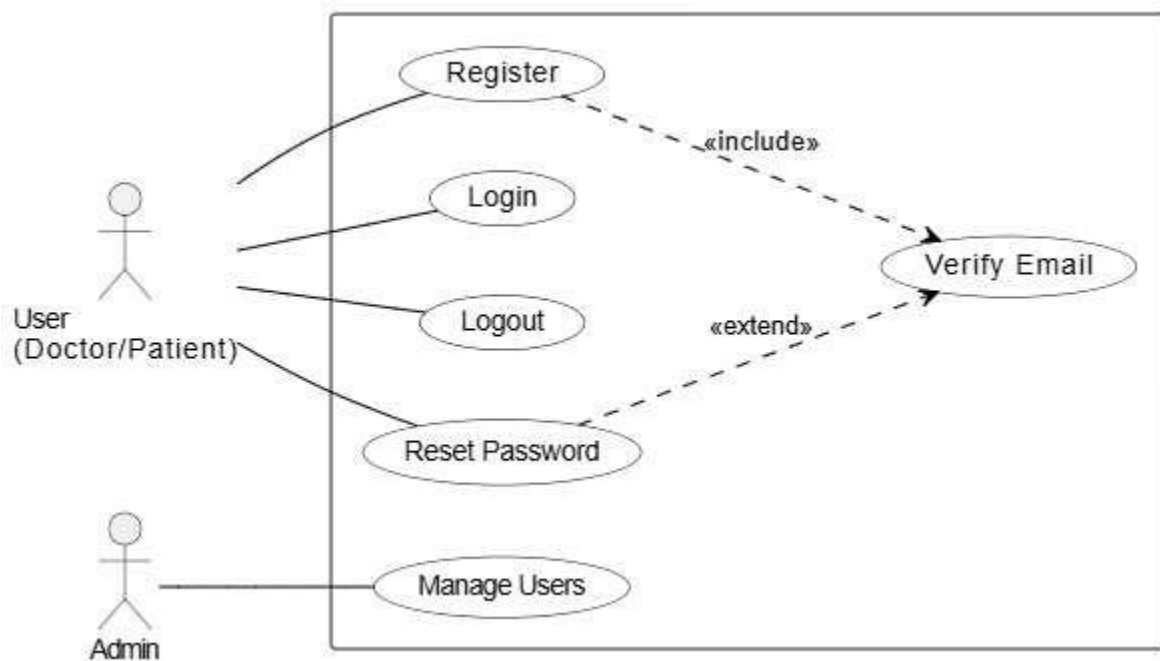


Figure 3.1: User Management

3.1.1 Use Case 1 Description

Table 3.1 This use case describes the process by which users (doctors or patients) register for an account, log in, reset their passwords, verify emails, and log out, while administrators manage user accounts. It ensures secure access and account management within the CuraHub mobile and web platform

Table 3.1: Authentication, Registration, and Posting

Use Case ID	UC-1
Actors	Primary Actor: User (Doctor/Patient) Secondary Actor: Admin
Description	This use case describes the process by which patients or doctors register for an account, log in to access services, reset forgotten passwords, verify email addresses, and log out securely. Administrators are responsible for managing user accounts through the admin portal.
Trigger	The user accesses the app to register, log in, or recover their account. The admin accesses the portal to manage user data.
Preconditions	The user has access to the CuraHub mobile app. The admin has access to the web admin panel with valid credentials.
Post conditions	A user account is created and activated. The user is authenticated into the system. Password is reset successfully. The admin has updated or managed user records.
Normal Flow	1. Register i. User navigates to the registration screen. ii. User enters required details (name, email, password, role). iii. System sends a verification email. iv. User clicks on the verification link. v. System activates account. 2. Login i. User opens the app and navigates to the login page. ii. User enters valid credentials. iii. System authenticates the user and redirects to the dashboard.

	3. Logout i. User clicks on "Logout". ii. System logs the user out and returns to login screen. 4. Reset Password i. User selects "Forgot Password". ii. Enters registered email. iii. System sends reset link via email. iv. User resets password through the link. 5. Manage Users (Admin) i. Admin logs in to the web portal. ii. Navigates to User Management section. iii. Views, approves, edits, or deletes user accounts.
Alternative Flows	1. Register – Email Already Used i. User submits the registration form. ii. System identifies email in use. iii. Displays error message to use a different email. 2. Login – Incorrect Credentials i. User enters incorrect login info. ii. System shows error and prompts retry.
Exceptions	1. Verification Email Not Received i. User does not receive email due to spam or server issues. ii. User can request resending verification email. 2. Password Reset Link Expired i. User attempts to use an expired link. ii. System prompts to request a new one.
Business Rules	- Usernames/emails must be unique. - Passwords must be at least 8 characters long and secure. - Accounts must be verified before login. - Admins are the only actors authorized to manage users.
Assumptions	- Users have internet access and valid email addresses. - Users have internet, email, and agree to platform policies.

3.2 Use Case 2

Figure 3.2 This use case diagram illustrates interactions related to browsing and managing doctor profiles and medical departments in the CuraHub platform. Users (doctors/patients) can view and update doctor profiles and browse departments.

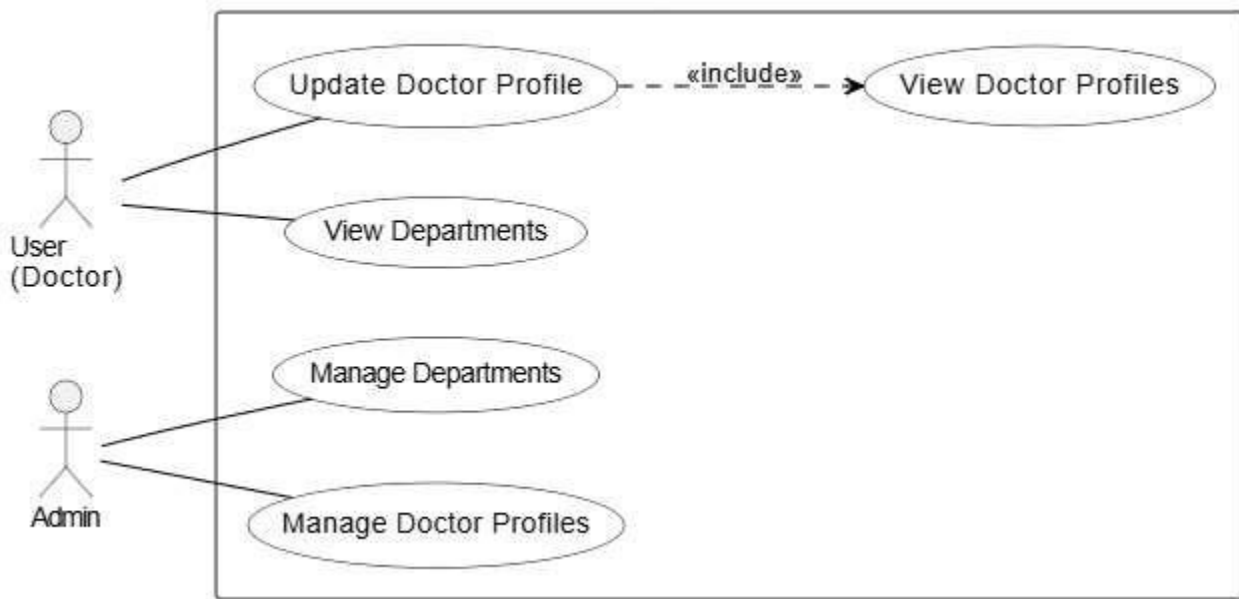


Figure 3.2: Doctor & Department Management

3.2.1 Use Case 2 Description

Table 3.2 This use case table illustrates interactions related to browsing and managing doctor profiles and medical departments in the CuraHub platform. Users (doctors/patients) can view and update doctor profiles and browse departments, while the admin manages doctor profiles and department listings.

Table 3.2: Doctor & Department Management

Use Case ID	UC-2
Actors	Primary Actor: User (Doctor/Patient) Secondary Actor: Admin
Description	This use case involves viewing and managing doctor profiles and medical departments. Users can browse departments and view or update doctor profiles. Admins manage department records and doctor data via the admin portal.

Trigger	The user chooses to explore doctors or departments. The admin chooses to manage doctor profiles or departments.
Preconditions	User is logged in. Admin has authenticated access to the admin panel.
Postconditions	Departments are displayed. Doctor profiles are viewed or updated. Admin successfully manages doctor or department data.
Normal Flow	1. View Doctor Profiles i. User navigates to the doctor section. ii. System displays categorized doctor listings with filters. 2. Update Doctor Profile i. Logged-in doctor selects "Edit Profile". ii. Updates profile information and saves changes. iii. System validates and updates the profile. 3. View Departments i. User accesses the department page. ii. System shows categorized departments. 4. Manage Departments (Admin) i. Admin logs into the admin panel. ii. Admin adds, edits, or deletes departments. 5. Manage Doctor Profiles (Admin) i. Admin navigates to the doctor management section. ii. Views, verifies, edits, or removes doctor accounts.
Alternative Flows	1. Update Profile - Invalid Data i. Doctor submits profile with missing fields. ii. System prompts for correction. 2. Manage Departments - Duplicate Entry i. Admin tries to add a department with an existing name. ii. System shows a duplication warning.
Exceptions	1. Doctor Profile Fetch Failed i. Network issue or server error. ii. System displays an error message.
Business Rules	Department names must be unique. Only verified doctors can update their

	profiles. Admin must authenticate for management actions.
Assumptions	Users have internet access and are logged in. Admins have sufficient privileges to manage platform data.

This table outlines the interactions involved in managing and viewing doctor profiles and medical departments within the CuraHub platform. It captures the roles of both users and administrators in accessing and modifying healthcare data. The use case ensures that patients can browse medical professionals and departments, while admins maintain accurate records.

3.3 Use Case 3

Figure 3.3 This use case diagram depicts the appointment and schedule management functionalities within CuraHub. Users (patients and doctors) can book, view, and cancel their appointments, while administrators oversee appointment management and scheduling to ensure smooth operation and availability. The system supports coordinated interaction between users and admins for efficient healthcare service delivery. It captures the roles of both users and administrators in accessing and modifying healthcare data. The use case ensures that patients can browse medical professionals and departments, while admins maintain accurate records.

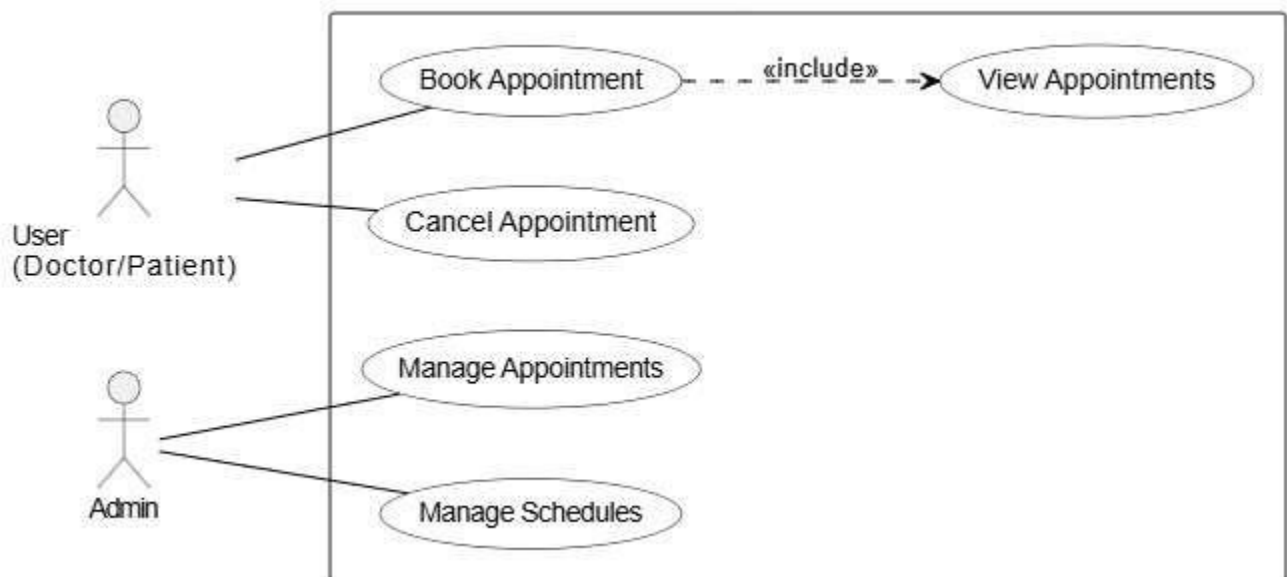


Figure 3.3: Appointments and Scheduling

3.3.1 Use Case 3 Description

Table 3.3 This table details the processes involved in appointment booking, viewing, cancellation, and schedule management on the CuraHub platform. It highlights the roles of patients, doctors, and administrators in managing appointments and schedules, ensuring timely and organized healthcare services. The use case addresses user interactions and administrative controls to maintain appointment integrity and optimize healthcare access.

Table 3.3: Appointments and Scheduling

Use Case ID	UC-3
Actors	Primary Actor: User (Doctor/Patient)
	Secondary Actor: Admin
Description	This use case describes how users book, view, and cancel appointments, while administrators manage appointments and doctor schedules. The system facilitates appointment handling for efficient healthcare delivery.
Trigger	User initiates booking, viewing, or cancellation of appointments; Admin manages appointments and schedules.
Preconditions	User is logged into the mobile app or web portal. Admin has appropriate privileges. Schedules exist for doctors.
Postconditions	Appointments are booked, viewed, or canceled; schedules and appointments are updated accordingly.
Normal Flow	1. Book Appointment: User selects doctor, date, time, and confirms booking. 2. View Appointments: User or doctor views upcoming appointments. 3. Cancel Appointment: User cancels an existing appointment. 4. Manage Appointments (Admin): Admin views, updates, or deletes appointments. 5. Manage Schedules (Admin): Admin updates doctors' availability and schedules.
Alternative Flows	1. Booking Conflicts: System notifies user if the selected slot is unavailable. 2. Cancellation Deadlines: User is notified if cancellation is too late or restricted.
Exceptions	1. System failure during booking or cancellation prompts retry.

	2. Unauthorized access attempt by non-privileged users is blocked.
Business Rules	Appointments must be booked within available schedule slots. Users cannot double-book overlapping appointments. Admins must approve schedule changes. Cancellations may have policies or fees.
Assumptions	Users have stable internet connection. Doctors maintain up-to-date schedules. Admins monitor system for conflicts and errors.

3.4 Use Case 4

Figure 3.4 This use case diagram illustrates the chat functionalities within CuraHub, showing how users (doctors and patients) can start, send, receive, and end chat conversations. Administrators have the ability to manage chat records for monitoring and support purposes. The system ensures real-time communication while maintaining chat history integrity.

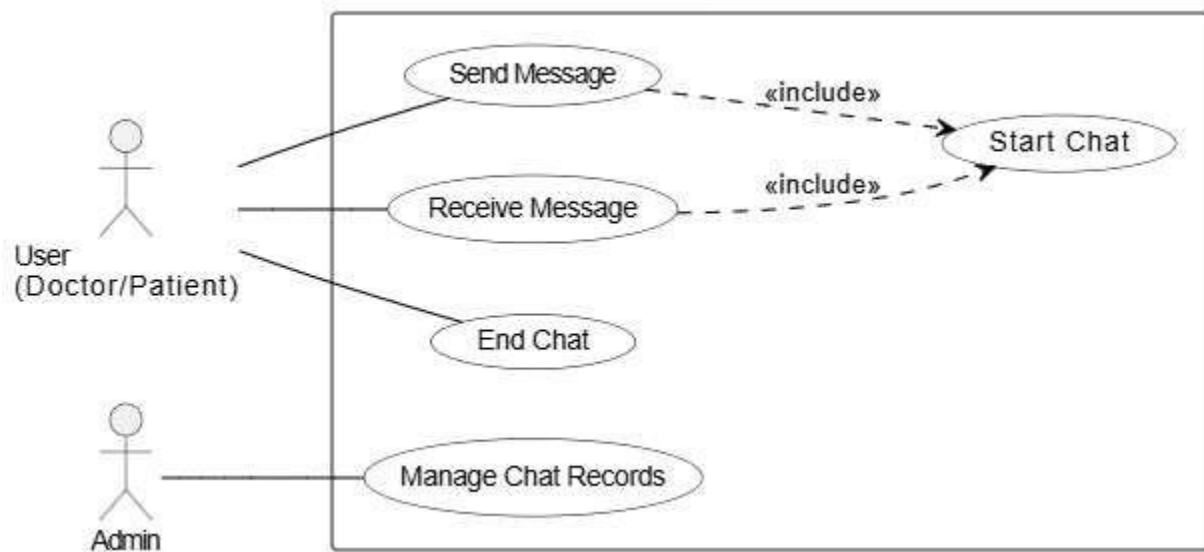


Figure 3.4: Chat Interaction

3.4.1 Use Case 4 Description

Table 3.4 This use case details the process of real-time chat communication between users (doctors and patients) and the administrative management of chat records. It highlights the sequential flow of starting chats, sending and receiving messages, ending conversations, and administrative oversight to ensure quality and compliance.

Table 3.4: Chat Interaction

Use Case ID	UC-4
Actors	Primary Actor: User (Doctor/Patient)
	Secondary Actor: Admin
Description	This use case describes how users initiate and participate in chat conversations, including sending and receiving messages. Administrators manage chat records for auditing and support.
Trigger	User initiates a chat session or sends/receives messages; Admin reviews or manages chat logs.
Preconditions	User is logged into the system. Chat service is active and connected. Admin has permission to access chat records.
Postconditions	Chat sessions are logged, messages delivered in real-time, and chat records maintained.
Normal Flow	1. Start Chat: User initiates a chat session. 2. Send Message: User sends a message; system delivers it. 3. Receive Message: User receives messages from another participant. 4. End Chat: User ends the chat session. 5. Manage Chat Records: Admin accesses and manages chat logs for review or compliance.
Alternative Flows	1. Message Delivery Failure: System retries or notifies the user. 2. Unauthorized Access: Admin attempts to access restricted chat records are denied.
Exceptions	1. Network Disconnection interrupts chat. 2. System errors during message transmission prompt error handling.
Business Rules	Only authenticated users can start or participate in chats. Chat records must be securely stored. Admins have exclusive rights to manage chat records. Chats are timestamped and user-identified.
Assumptions	Users have reliable internet access. System supports real-time message delivery.

3.5 Use Case 5

Figure 3.5 This use case diagram represents the voice/video call interactions within CuraHub. Users (doctors and patients) can initiate, receive, and end calls seamlessly. Meanwhile, administrators oversee call records management to ensure quality control and compliance with system policies.

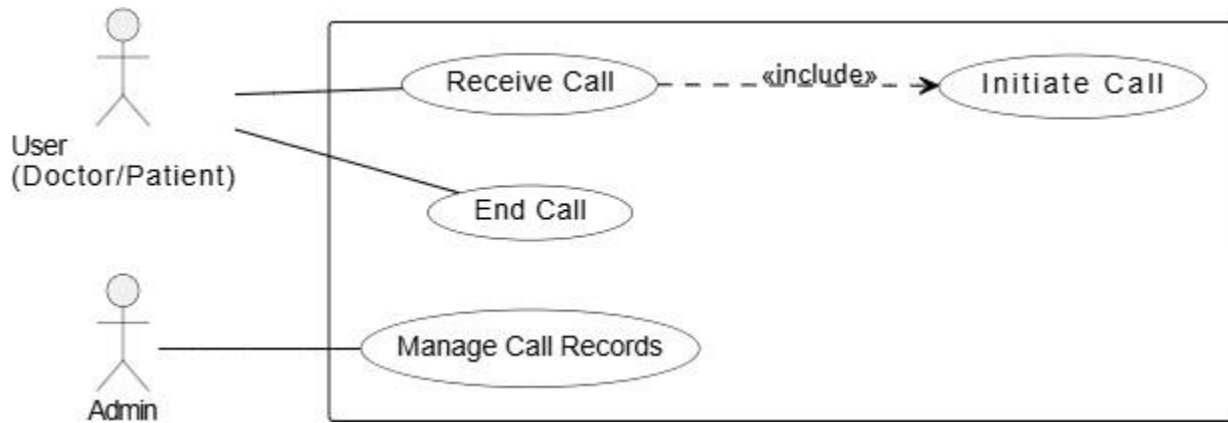


Figure 3.5: Call Functionality

3.5.1 Use Case 5 Description

Table 3.5 This use case describes the process of initiating and managing calls between users, including call reception and termination. It also covers the administrative function of managing records.

Table 3.5: Call Functionality

Use Case ID	UC-5
Actors	Primary Actor: User (Doctor/Patient) Secondary Actor: Admin
Description	This use case details how users make and receive calls, terminate conversations, and how admins manage call logs for monitoring and compliance.
Trigger	User initiates or receives a call; Admin accesses call records for management.
Preconditions	User is logged in and has access to calling features; system supports call

	functionality; Admin has proper permissions.
Postconditions	Call sessions are established or terminated; call records are updated and stored securely.
Normal Flow	1. Initiate Call: User starts a call to another user. 2. Receive Call: User receives an incoming call. 3. End Call: User ends the active call.4. Manage Call Records: Admin reviews and manages stored call records.
Alternative Flows	1. Call Rejected: User declines an incoming call. 2. Call Dropped: Unexpected disconnection handled gracefully. 3. Unauthorized Access: Admin attempts to access restricted call records denied.
Exceptions	1. Network failure during call. 2. System errors during call setup or termination.
Business Rules	Only authenticated users can place or receive calls. Call records must be securely stored and accessible only to admins. Calls must be logged with timestamps and participant details.
Assumptions	Users have sufficient network connectivity. System supports real-time voice/video communication. Admins are responsible for managing call data compliance.

3.6 Use Case 6

Figure 3.6 This use case diagram illustrates the payment process within CuraHub. Users can make payments and request refunds, with payment verification included as a critical step. Administrators manage all transaction records to ensure financial accuracy and transparency. This use case diagram illustrates the payment process within CuraHub. Users can make payments and request refunds, with payment verification included as a critical step. Administrators manage all transaction records to ensure financial accuracy and transparency. The system also sends automated payment confirmations and alerts for failed transactions. Users can track their payment history for better clarity. Admins can review refund requests, approve or reject them, and generate financial reports for auditing purposes.

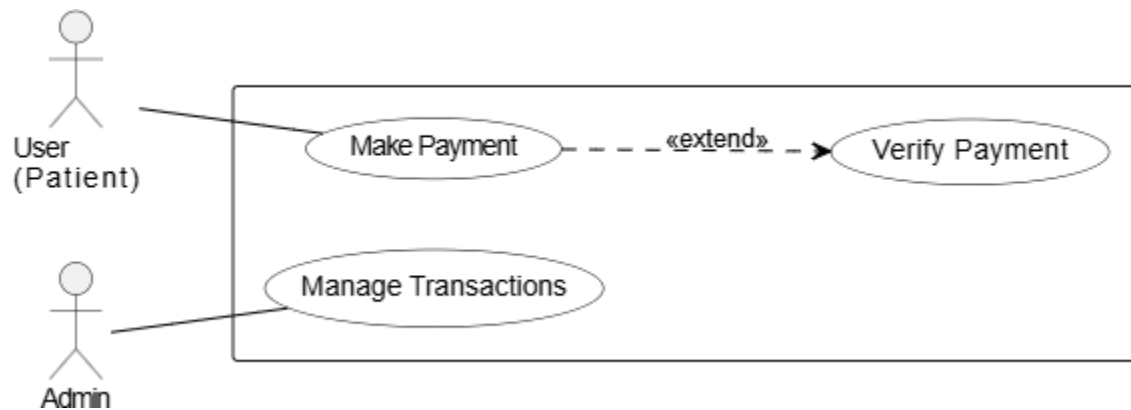


Figure 3.6: Payment Management

This use case captures the complete lifecycle of payments made by users, including verification and refunds. It also defines the administrative role in managing and auditing transaction records for accountability and security.

3.6.1 Use Case 6 Description

Table 3.6 This section provides a detailed overview of the Real-Time Consultation & Chat use case in CuraHub, highlighting the interactions between patients, doctors, and the system, including symptom-based discussions, case-related messaging, and group health consultations. It defines the roles of each actor such as patients, doctors, and administrators and outlines the system’s behavior to ensure secure, confidential, and collaborative communication throughout the healthcare process.

Table 3.6: Payment Management

Use Case ID	UC-6
Actors	Primary Actor: User (Doctor/Patient)
	Secondary Actor: Admin
Description	This use case describes how users make payments, verify transactions, and request refunds, while administrators oversee and manage all payment transactions within the system.
Trigger	User initiates a payment or refund; Admin accesses transaction records for management.
Preconditions	User is logged in and has a valid payment method; system supports secure payment processing; Admin has transaction management permissions.

Postconditions	Payment is processed and verified; refunds are handled as necessary; transaction records are updated and securely stored.
Normal Flow	1. Make Payment: User initiates a payment for services. 2. Verify Payment: System verifies payment authenticity and status. 3. Refund Payment (optional): User requests a refund; system verifies and processes refund. 4. Manage Transactions: Admin views and manages payment records.
Alternative Flows	1. Payment Failure: Payment is declined or unsuccessful. 2. Refund Denied: Refund request does not meet policy criteria. 3. Unauthorized Access: Admin attempts to access restricted transaction data.
Exceptions	1. Payment gateway downtime. 2. Incorrect payment details entered. 3. System errors during transaction processing.
Business Rules	All payments must be verified before confirmation. Refunds require verification and approval. Transaction records must be securely stored and accessible only to authorized personnel.
Assumptions	Users have valid payment methods and connectivity. System integrates with reliable payment gateways. Admins ensure compliance with financial regulations.

3.7 Functional Requirements

Functional requirements define the specific behavior or functionality that the CuraHub healthcare platform must exhibit to meet the needs of patients, doctors, and administrators. These requirements describe what the system should do, typically expressed in terms of actions, tasks, or operations that users can perform such as booking consultations, submitting medical cases, chatting with doctors, or managing patient records. They form the foundation for system design and development, guiding the creation of use cases, system architecture, and user interfaces within CuraHub.

3.7.1 User Registration and Authentication

The CuraHub platform must allow new users—patients, doctors, and admins—to register through traditional email-password credentials, with support for secure social logins such as Google using OAuth 2.0. This ensures easy access while maintaining strong security standards. Upon registration, users should be able to manage their profiles, update personal information, upload medical details (if required), and reset passwords whenever needed.

3.7.2 Medical Cases & Consultations Module

Users should be able to view, submit, and manage medical cases within the CuraHub platform. These cases can be categorized based on the medical specialty or complexity level. Patients can submit their symptoms or reports, while doctors can review these cases, provide expert feedback, and offer medical recommendations. The system should also support additional features such as case-based discussions and online consultation sessions, enabling real-time medical guidance and collaborative diagnosis.

3.7.3 Real-Time Chat System

A core feature of CuraHub is its real-time chat module, which must support one-to-one messaging, group chats, and medical community discussion channels. It should include features like emoji support, medical file sharing (such as reports, prescriptions, or images), typing indicators, and instant notifications. The system should also ensure patient and doctor safety with features like report or block options, ensuring secure and responsible communication within the healthcare environment.

3.7.4 Q&A Health Forum

The forum section enables users to post health-related questions, provide answers, and engage in medical discussions. Each post can be liked, disliked, or marked as “resolved.” This module promotes knowledge-sharing and acts as a support system for patients seeking guidance. Users will also have a reputation or point system based on their activity and helpful contributions, ensuring an engaging and supportive healthcare community. The system will highlight verified answers from doctors for reliable guidance. Users can follow topics or threads to receive updates, improving engagement. Moderation tools and AI-based content checks will maintain a safe and trustworthy discussion environment.

3.7.5 Bug Detection and Reporting

Users should be able to report issues or errors they encounter while using various features of the CuraHub platform, such as medical consultations, chat, or the Q&A Health Forum. The system should include a reporting form with fields such as description, screenshot upload, severity level, and affected module. Admins will receive notifications and manage these reports via a dedicated backend interface, ensuring quick resolution and smooth platform performance.

3.7.6 Learning Section

This module will provide structured health-related learning materials and downloadable resources for users. The content will be organized into beginner, intermediate, and advanced levels, covering topics such as disease awareness, preventive healthcare, first-aid guidance, and wellness education. CuraHub's learning section ensures that users can easily access reliable medical information, improve their health knowledge.

3.7.7 Medical Community

CuraHub supports social-style posting features, allowing registered users—patients, doctors, or healthcare professionals to share health updates, medical cases, tips, or informative content within the community. Users can post status updates with optional tags or categories, react to posts with likes or comments, and follow other users to receive a personalized feed. This functionality promotes peer-to-peer learning, community engagement, and visibility of individual contributions across the healthcare ecosystem, fostering collaboration and knowledge sharing among users.

3.7.8 Admin Panel and Content Moderation

Administrators in CuraHub will have access to a secure dashboard where they can manage users, medical cases, Q&A forum posts, bug reports, and uploaded content. Admins can suspend or delete users, review flagged content, and generate platform statistics. This module ensures proper moderation, maintains platform integrity, and supports a safe and organized healthcare community.

3.8 Non-Functional Requirements

Non-functional requirements describe the aspects of CuraHub that are not directly related to its core functionality but are essential for overall effectiveness, performance, and user experience. These include system attributes such as performance, reliability, usability, and security, ensuring the platform delivers a robust, responsive, and user-friendly healthcare collaboration experience.

3.8.1 Scalability

CuraHub should be scalable to accommodate a growing number of users, doctors, and medical content. The system architecture should support both horizontal and vertical scaling, ensuring that new features, increased patient consultations, or higher platform traffic do not affect stability or performance.

3.8.2 Usability

The CuraHub user interface should be intuitive, visually appealing, and accessible to users with varying levels of technical expertise, including patients and healthcare professionals. Accessibility standards such as keyboard navigation, screen reader compatibility, and adjustable text sizes will be prioritized. A consistent design system will be applied across all modules to maintain familiarity and ease of use.

3.8.3 Performance

CuraHub must maintain optimal response times, even under high traffic from multiple patients, doctors, and consultations. Features like real-time chat, AI-powered disease suggestions, and dynamic content loading should execute seamlessly without noticeable lag. Efficient backend handling, Firebase optimization, and caching mechanisms will be implemented to meet these performance expectations. To ensure smooth user experience, real-time video consultations will be supported through low-latency streaming protocols. AI/ML model predictions (Regression, ANN, CNN) will be deployed on optimized servers or cloud functions to deliver quick and accurate results. The system should handle concurrent access to medical records and appointment schedules without performance drops. Load balancing on the Node.js server will distribute traffic evenly during peak hours. Elasticsearch indexing will improve query performance for searching doctors, symptoms, and forum posts. Cloud Functions will manage heavy backend tasks, preventing delays in the mobile app.

3.8.4 Security and Privacy

Robust security protocols will be enforced in CuraHub, including user authentication via Firebase, role-based access control for patients, doctors, and admins, data encryption, and protection against common web vulnerabilities. Sensitive medical and personal data will be securely stored and transmitted, ensuring privacy and compliance with data protection standards.

3.8.5 Maintainability

The CuraHub codebase will follow modular design principles, clean coding standards, and thorough documentation to allow easy updates, debugging, and addition of new features. Version control through Git will assist the team of developers in collaboration and ongoing maintenance of the platform[7].

3.8.6 Compatibility

CuraHub will be compatible across multiple browsers (Chrome, Firefox, Edge) and devices (desktop, tablet, mobile). Responsive design using Flutter's built-in widgets and adaptive layouts will ensure consistent behavior and user experience across various screen sizes.

Chapter 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

The design and architecture of CuraHub focus on building a modular, scalable, and interactive mobile/web-based healthcare platform. The frontend, developed in Flutter using Dart, ensures responsive and user-friendly interactions, while the backend, powered by Firebase and Node.js, handles secure data processing, real-time communication, AI integration, and structured module management. This cohesive design supports seamless collaboration between patients and doctors, efficient health consultation, and effective content and appointment management across all features of the platform[8].

4.1 System Architecture

Diagram 4.1 shows how CuraHub connects patients, doctors, and administrators through mobile and web interfaces. At its core, the platform supports features like user registration, medical case submissions, AI-powered disease detection, real-time consultations, a Q&A health forum, and secure messaging.

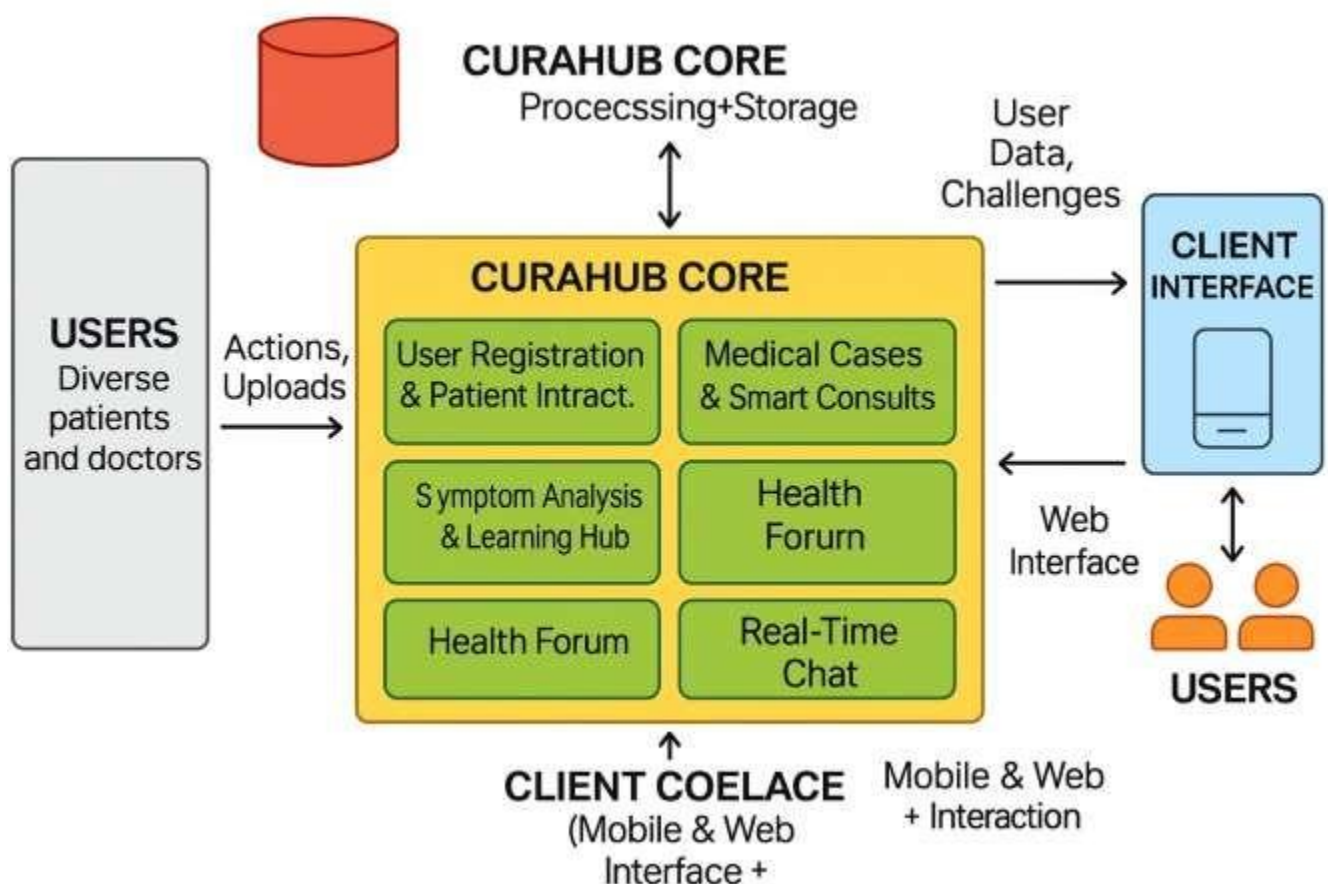


Figure 4.1: CCH architecture

The System Architecture of CuraHub is designed to ensure seamless integration between user interaction, healthcare processing, and secure data storage. It follows a layered architecture composed of three primary tiers: the Frontend Layer, the Backend Layer, and Database Layer. The Frontend Layer serves as the user-facing interface, where patients, doctors, and administrators interact with the platform. This layer includes modules like the user registration and patient interaction interface, symptom analysis and learning hub, medical cases and smart consults, health forum, and real-time chat interface. All features are optimized to provide an easy, intuitive, and responsive experience for diverse users.

4.2 Data Representation

The CuraHub adopts a structured and scalable approach to data representation, primarily utilizing Firebase Firestore and Realtime Database to store and manage healthcare-related information. This structured methodology ensures data integrity, consistency, and optimized performance across the platform's interconnected healthcare modules. Given the diverse and feature-rich nature of the platform—which includes patient registration, symptom analysis, medical cases, smart consultations, real-time chat, health forums, and learning resources—CuraHub relies on well-defined collections and documents to efficiently support data storage, retrieval, and manipulation.

Each entity within the system such as users, chat messages, patient records, medical case reports, or forum posts represented through Firestore collections, where documents contain structured fields that define attributes and relationships. This allows clear mapping of interactions between patients, doctors, and CuraHub's various healthcare services while maintaining a consistent and logically organized data flow. Additionally, Firebase's querying capabilities are used to handle filtering, real-time updates, and data synchronization, which is essential for supporting fast symptom responses, smart consults, and live chat communication.

While CuraHub primarily uses structured and semi-structured Firestore documents, it may also incorporate nested JSON objects for storing flexible data such as symptom analysis results, AI-generated medical insights, dynamic health reports, or variable-length consultation histories. This hybrid approach provides a balance between structure and flexibility, offering both the organized document model of Firestore and the adaptability needed for evolving medical data. Through this thoughtful data representation design, CuraHub ensures robust functionality, scalability, and a seamless healthcare experience for both patients and doctors.

4.3 Process Flow

The CuraHub process begins with user registration and authentication, where patients and doctors sign in to access personalized healthcare features. Once authenticated, users can explore services such as symptom analysis, medical case submission, smart consultations, health forums, and real-time chat. Patients can upload symptoms, interact with AI for preliminary analysis, and request consultations with doctors.

4.3.1 Activity Diagrams

Activity diagrams are essential for visualizing the dynamic behavior of CuraHub, providing a clear representation of how patients, doctors, and system components interact throughout various healthcare processes. These diagrams help illustrate the flow of activities such as user registration, symptom analysis, consultation requests, medical case handling, health forum participation, and real-time chat communication. By capturing both sequential and parallel processes, activity diagrams make it easier to understand how user actions transition from one stage to another within the platform.

Because CuraHub integrates multiple modules, AI-based symptom analysis, case submission, smart consultation, health forums, and chat activity diagrams play a crucial role in showing how these modules coordinate with each other. They highlight decision points, branching paths, and automated responses from the AI engine or Firebase backend. This visual clarity improves communication between developers, designers, and stakeholders, ensuring everyone understands how the healthcare workflow operates from start to finish[9].

4.3.1.1 User Registration and Patient Interaction Activity Diagram

The Admin Activity Diagram shows how the admin logs into CuraHub, accesses the dashboard, and manages users, doctors, and system activities. It represents the admin's overall control of platform operations. This diagram effectively represents the comprehensive role of the admin in managing CuraHub, ensuring seamless healthcare service delivery, data integrity, and a secure, compliant environment for both patients and doctors. Furthermore, the admin oversees Stripe payment transactions, refunds, and financial reports to ensure accurate and secure processing. Notifications and alerts allow the admin to track system activities in real-time, maintaining smooth platform operations.

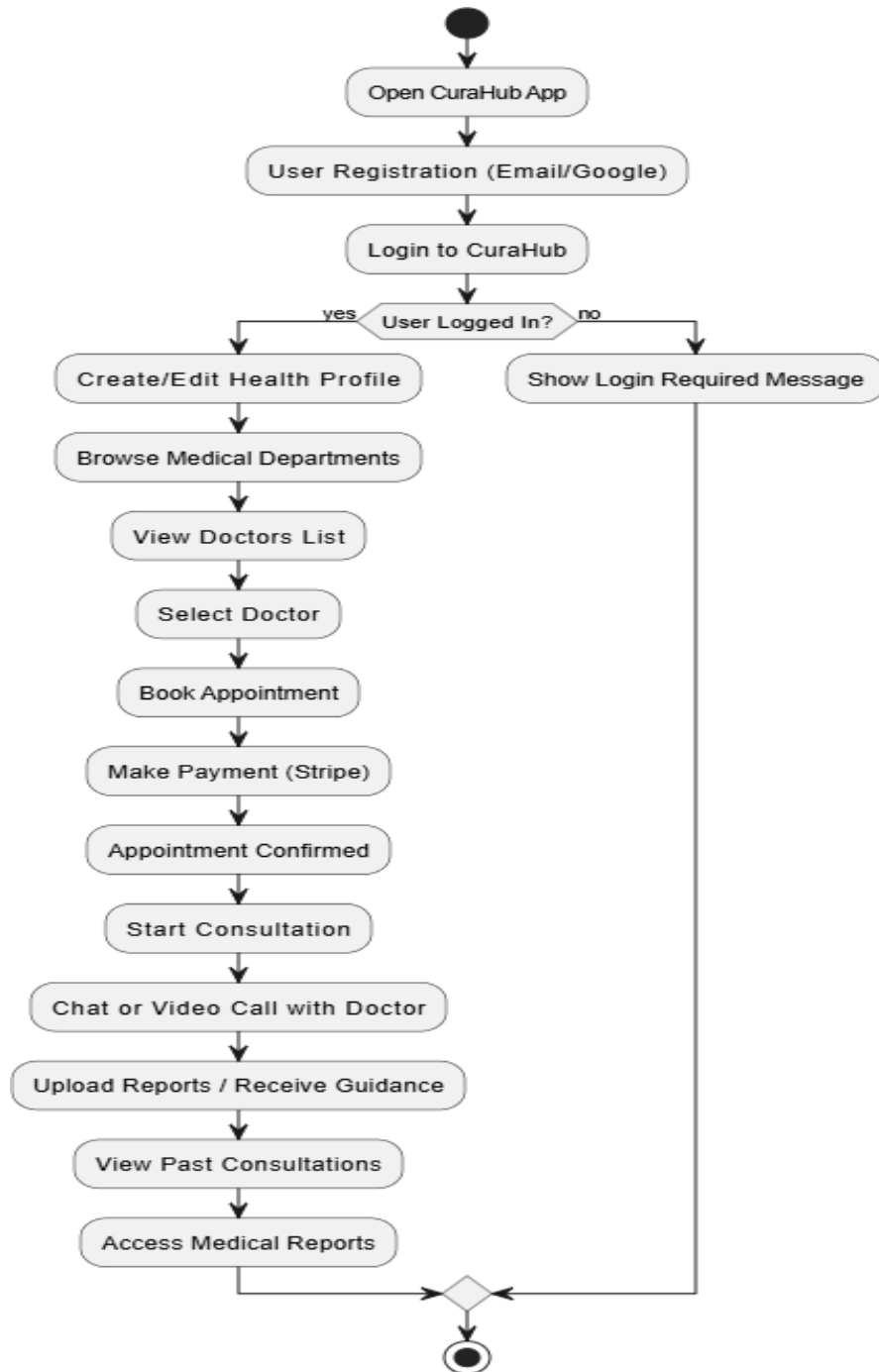


Figure 4.2: User Registration and patient interaction activity diagram

The Activity Diagram illustrates the administrator’s essential tasks within the CuraHub platform, starting with logging in and accessing the admin dashboard. Once logged in, the admin can manage patient and doctor registrations, verify credentials, and oversee all consultation requests and appointments. The admin can also monitor AI/ML symptom analysis results, track disease prediction logs.

4.3.1.2 Medical Cases and Consultation Activity Diagram

The Medical Cases and Consultation activity diagram illustrates the sequence of actions a patient takes to interact with the healthcare services on CuraHub. It begins with user registration and login via Firebase authentication. Once logged in, the patient can access the medical consultation module, where they can select a department, choose a doctor, and book appointments for chat or video consultations. The diagram highlights the decision point where the system verifies if the user is authenticated, ensuring that only registered users can access consultation features. Additionally, the diagram shows the option for patients to submit detailed medical cases, upload relevant reports or images, and provide symptom descriptions, enabling doctors to review them before the consultation.

The activity flow emphasizes AI-assisted symptom analysis, where the system provides preliminary suggestions and predicts possible conditions using ML models like Regression, ANN, and CNN. It also visualizes the consultation process, including doctor responses, scheduling follow-ups, and adding notes to medical records. Patients can leave feedback or rate the consultation, which is recorded for quality management and improvement.

Furthermore, the enhanced activity diagram also demonstrates how CuraHub integrates real-time notifications and reminders throughout the patient journey. Once a consultation is booked, the patient receives automated alerts about appointment confirmations, upcoming sessions, or recommended next steps based on their submitted reports. This ensures continuous engagement and helps reduce missed appointments. The system also updates the patient's dashboard with their case status, doctor replies, and AI-generated recommendations, allowing them to monitor their health interactions easily and efficiently.

Additionally, the diagram reflects the collaborative flow between doctors and the system backend. After reviewing patient submissions, doctors can request additional information, prescribe medication, or suggest diagnostic tests, all of which are tracked within the platform. The activity diagram highlights how each interaction contributes to the patient's digital medical record, offering a centralized history of consultations, treatments, and AI insights. This holistic workflow supports better decision-making, encourages proactive patient involvement, and enhances overall healthcare delivery on CuraHub.

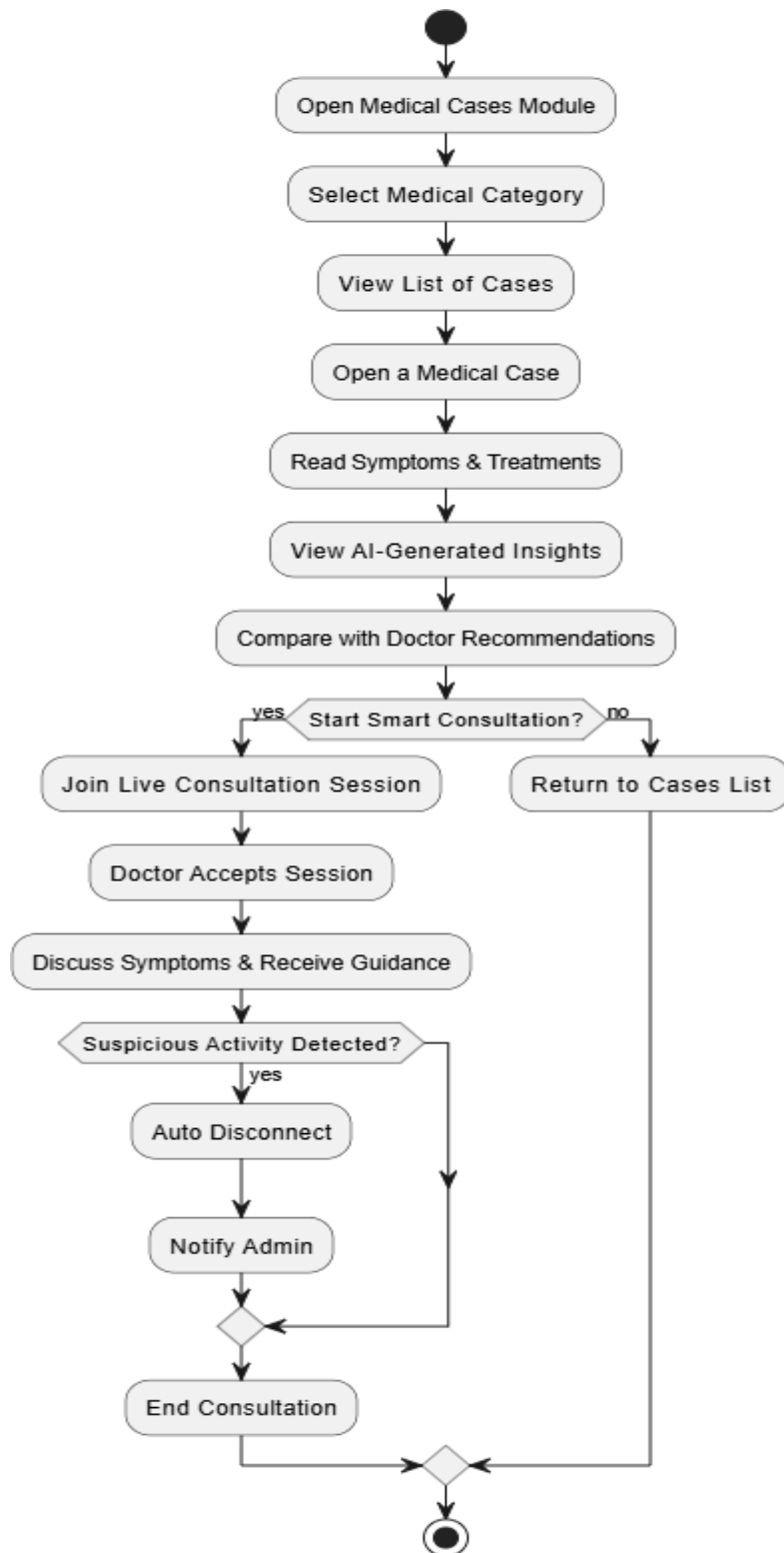


Figure 4.3: Medical Cases and Consultation Activity Diagram

4.3.1.3 System Analysis and AI Recommendation Activity Diagram

The Medical Cases and Consultation activity diagram outlines the patient journey for interacting with medical consultation services on CuraHub. After logging in, patients can browse available departments or specialists, select a doctor, and submit a medical case by providing symptom details, medical history, and any relevant reports or images. For consultations, the diagram highlights the booking and session process, where patients can schedule chat or video appointments and receive responses from doctors.

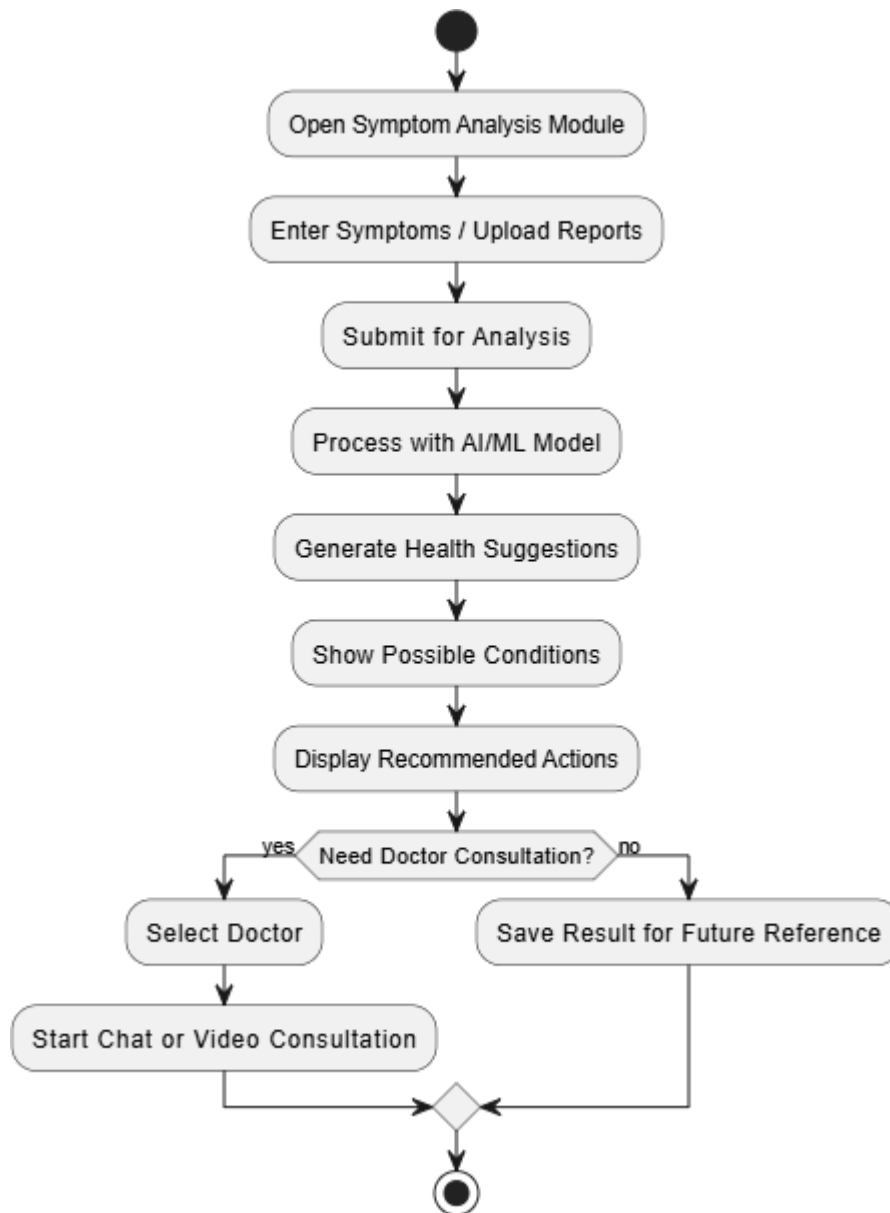


Figure 4.4: System Analysis and AI Recommendation Activity Diagram

4.3.1.4 Q&A Health Forum Activity Diagram

The Q&A Health Forum activity diagram demonstrates how patients and users interact with the community-driven health forum on CuraHub.

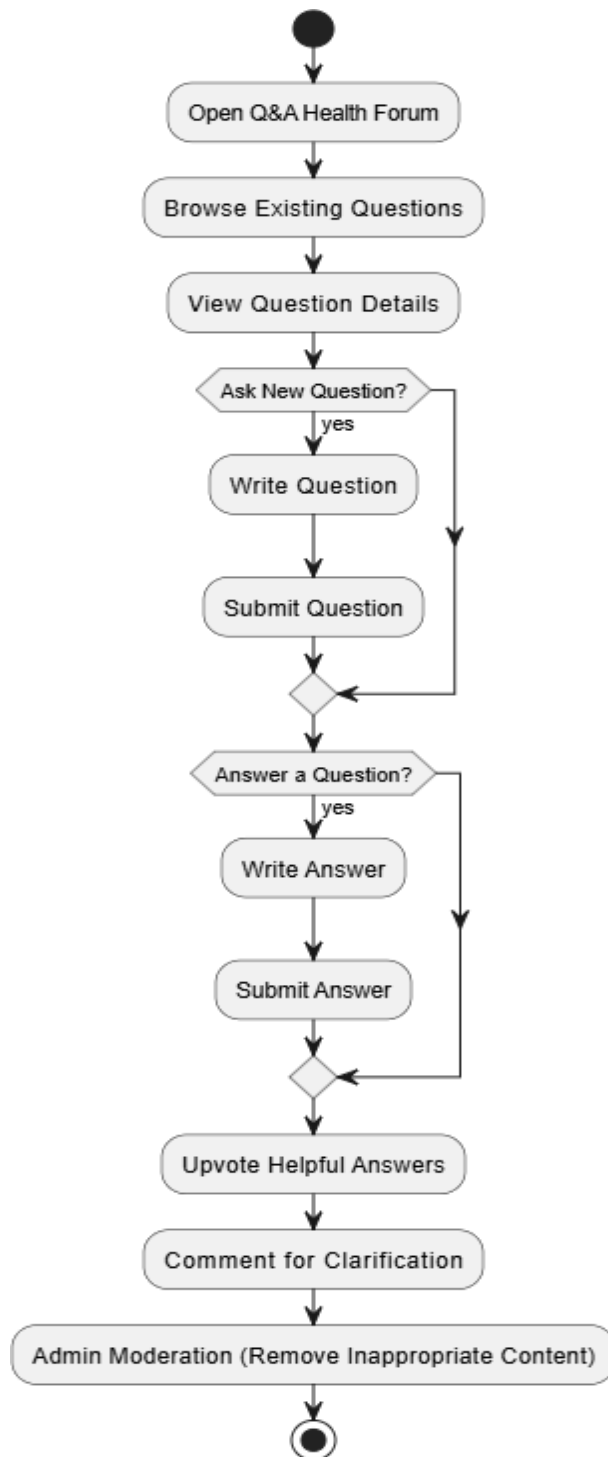


Figure 4.5: Q&A Health Forum Activity Diagram

4.3.1.5 Real-Time Chat Activity Diagram

The Real-Time Chat activity diagram illustrates the interactive communication flow within the CuraHub healthcare platform, showcasing how patients and doctors engage in seamless medical communication. The Real-Time Chat activity diagram depicts the dynamic communication workflow within the CuraHub healthcare platform, highlighting the seamless exchange of messages between patients and doctors. It visually demonstrates how real-time interactions are initiated, processed, and delivered to ensure smooth, instant, and effective medical communication. It showcases how patients and doctors initiate and exchange messages in a smooth, uninterrupted flow. The diagram further illustrates the system's handling of message delivery, status updates, and real-time synchronization, ensuring fast, reliable, and effective medical communication between users. The Real-Time Chat activity diagram depicts the dynamic communication workflow within the CuraHub healthcare platform, highlighting the seamless exchange of messages between patients and doctors.

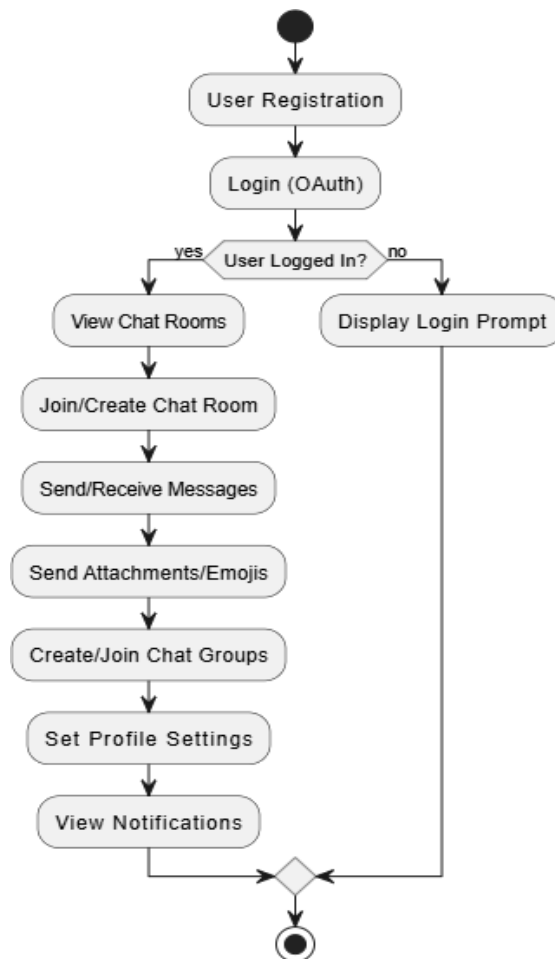


Figure 4.6: Real-Time Chat Activity Diagram

4.4 Design Models

4.4.1 Sequence Diagram

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time-focused, and they show the order of the interaction visually by using the vertical axis of the diagram to represent the time at which messages are sent.

4.4.1.1 Sequence Diagram for AI Symptom Checker

The AI Symptom Checker Sequence Diagram for CuraHub shows the step-by-step interaction between the user, the application interface, the backend server, and the AI analysis module. The sequence begins when the user opens the symptom checker and enters their symptoms. The backend receives the input and sends it to the AI model.

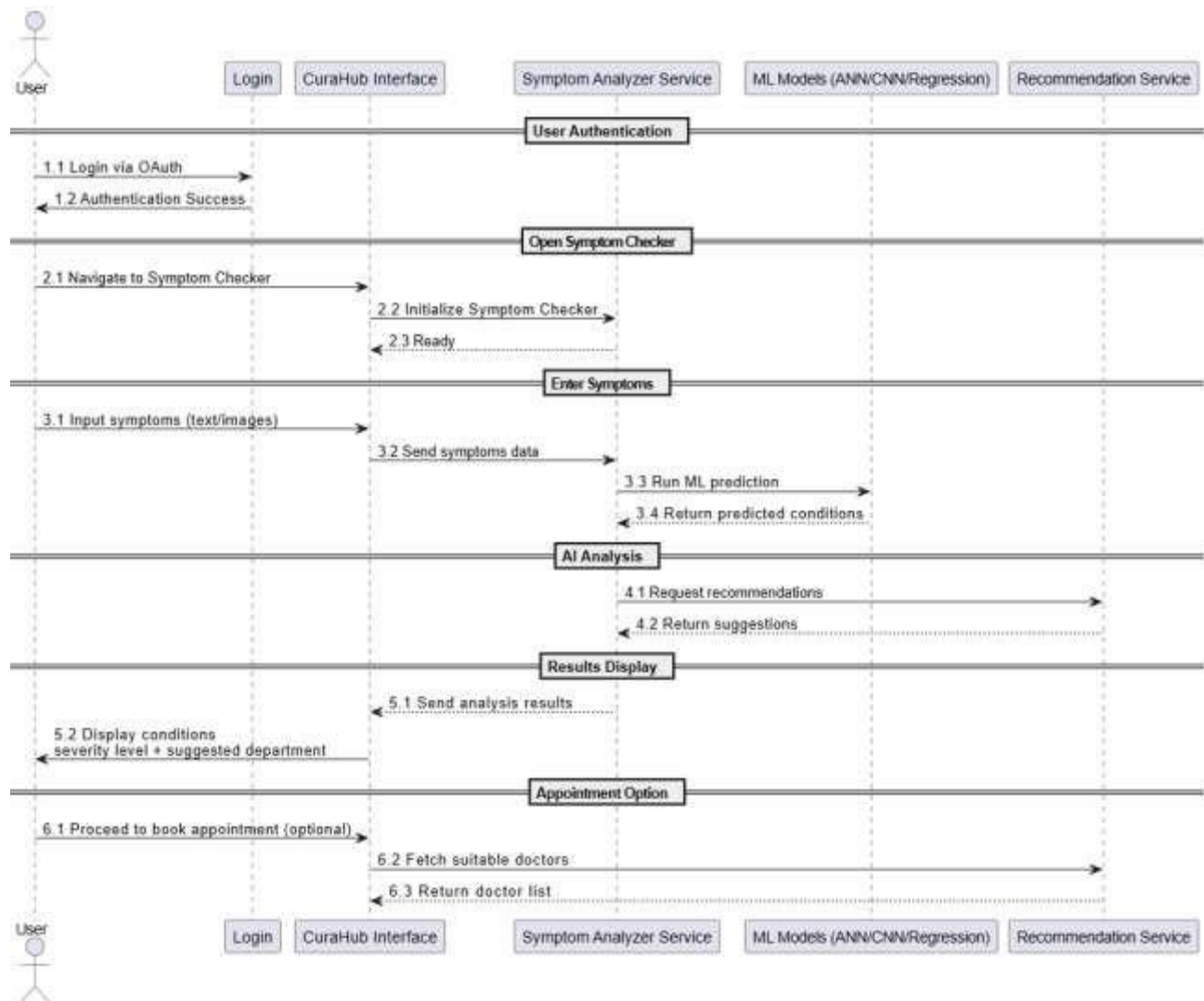


Figure 4.7: Sequence Diagram for AI Symptom Checker

4.4.1.2 Sequence Diagram for Appointment Booking and Scheduling

The Appointment Booking and Scheduling Sequence Diagram illustrates the flow of actions when a patient interacts with the consultation booking feature of CuraHub. It begins with the patient logging in through Firebase authentication. Once authenticated, the system fetches the user profile and displays available doctors, departments, and available time slots. The patient selects a doctor and preferred time, and the booking request is submitted through the appointment controller and stored securely in the database.

The system confirms successful appointment creation and updates the patient and doctor dashboards in real time. Notifications are sent to both the patient and the doctor to ensure they are informed about the scheduled consultation.

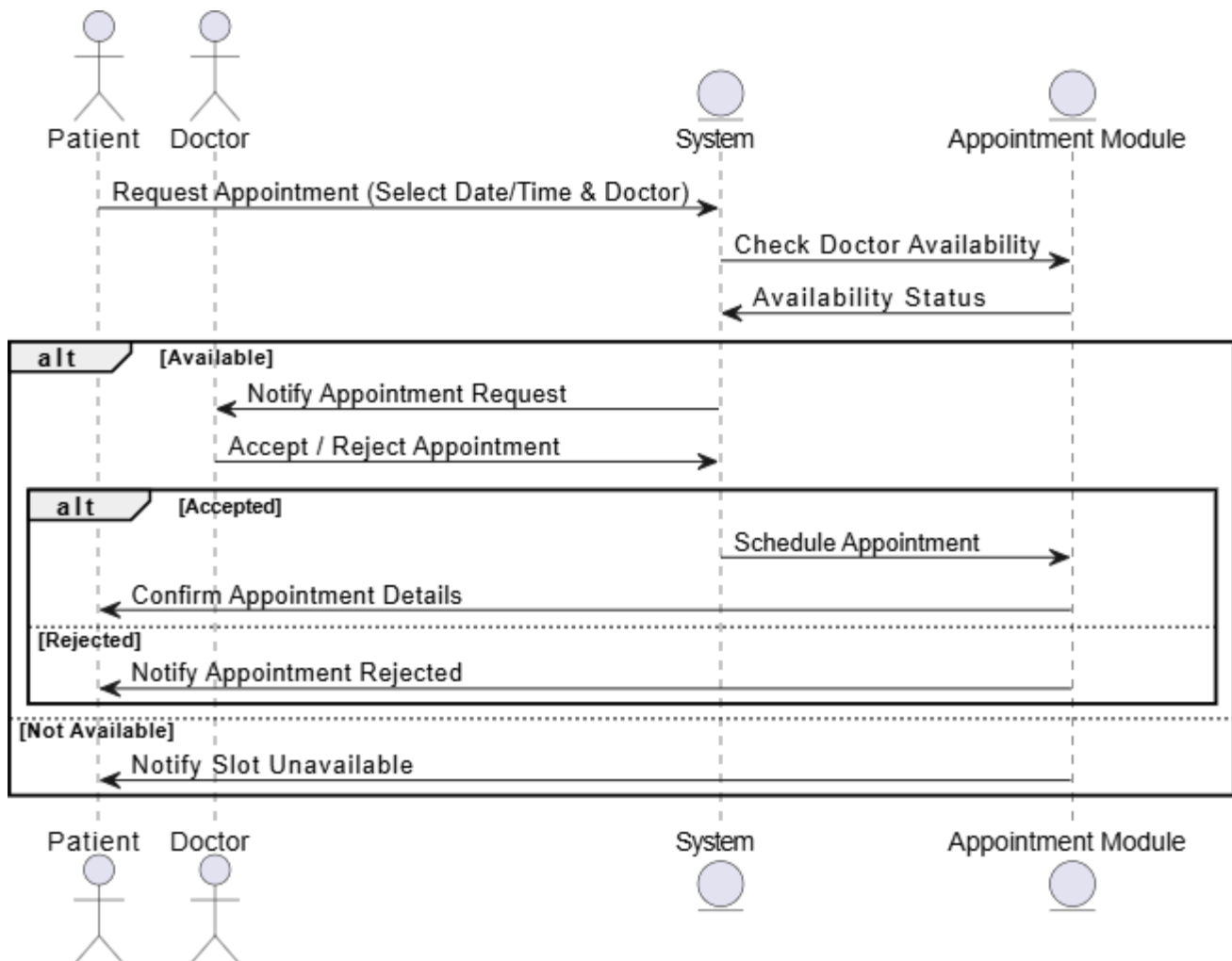


Figure 4.8: Sequence Diagram for appointment booking and scheduling

4.4.1.3 Sequence Diagram for Medical Consultation

The Medical Consultation Sequence Diagram illustrates the interaction flow when patients engage in consultations with doctors through CuraHub. After successful registration and login, patients access the consultation interface, where they can submit medical cases, describe symptoms.

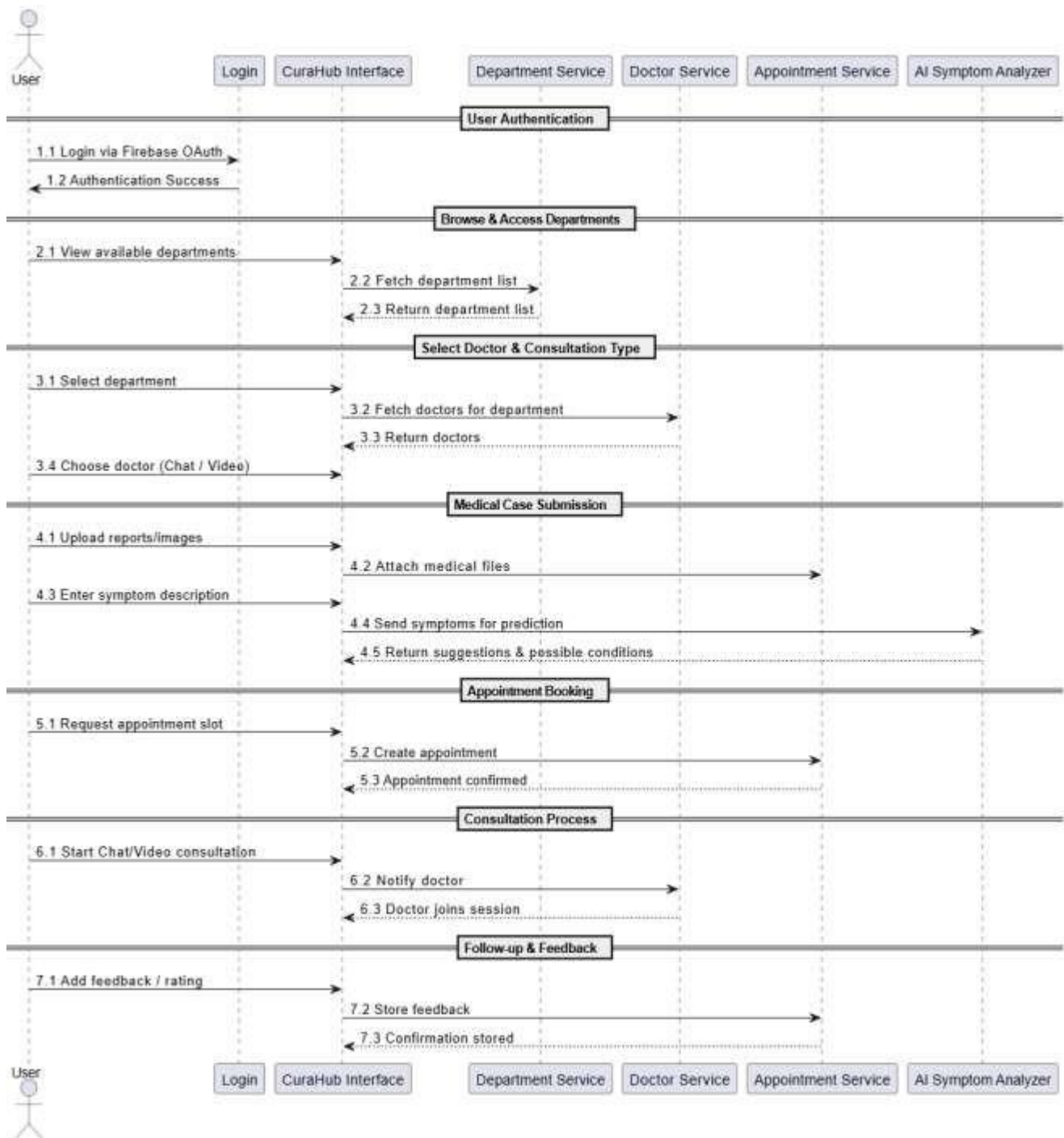


Figure 4.9: Sequence diagram for Medical Consultation

4.4.1.4 Sequence diagram for Medical Records Management

The Medical Records Management Sequence Diagram outlines the step-by-step interactions between patients, doctors, and the system as they access and manage medical records within CuraHub. Upon successful registration and login, users navigate to the Medical Records module.

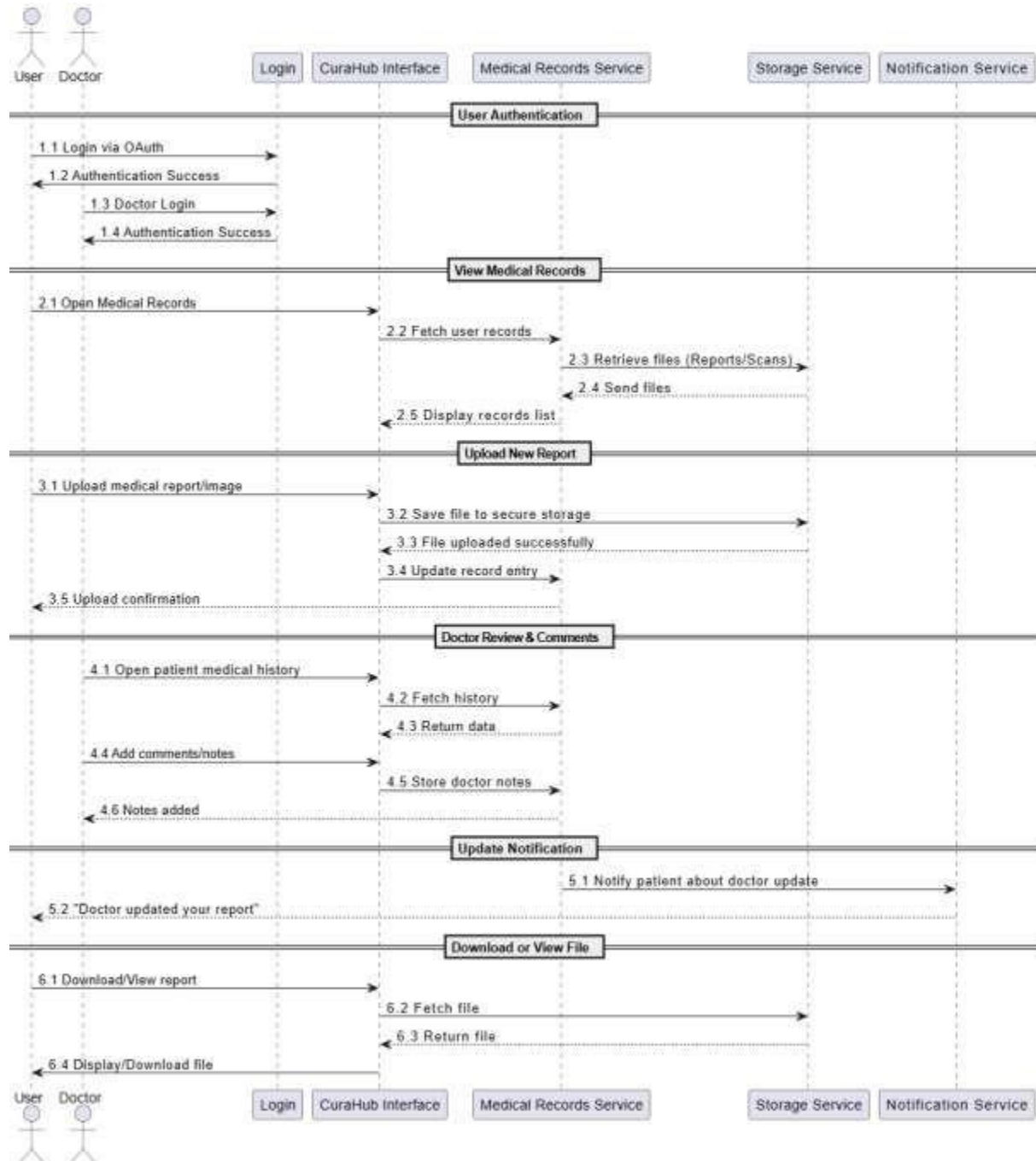


Figure 4.10: Sequence diagram for Medical Records Management

4.4.1.5 Sequence diagram for real-time chat

The Real-Time Chat Sequence Diagram illustrates the interactive flow between a patient, doctor, the chat interface, and the backend system that powers instant communication on CuraHub. Once authenticated, users can access existing chat sessions with doctors or initiate new private chats for medical consultation.

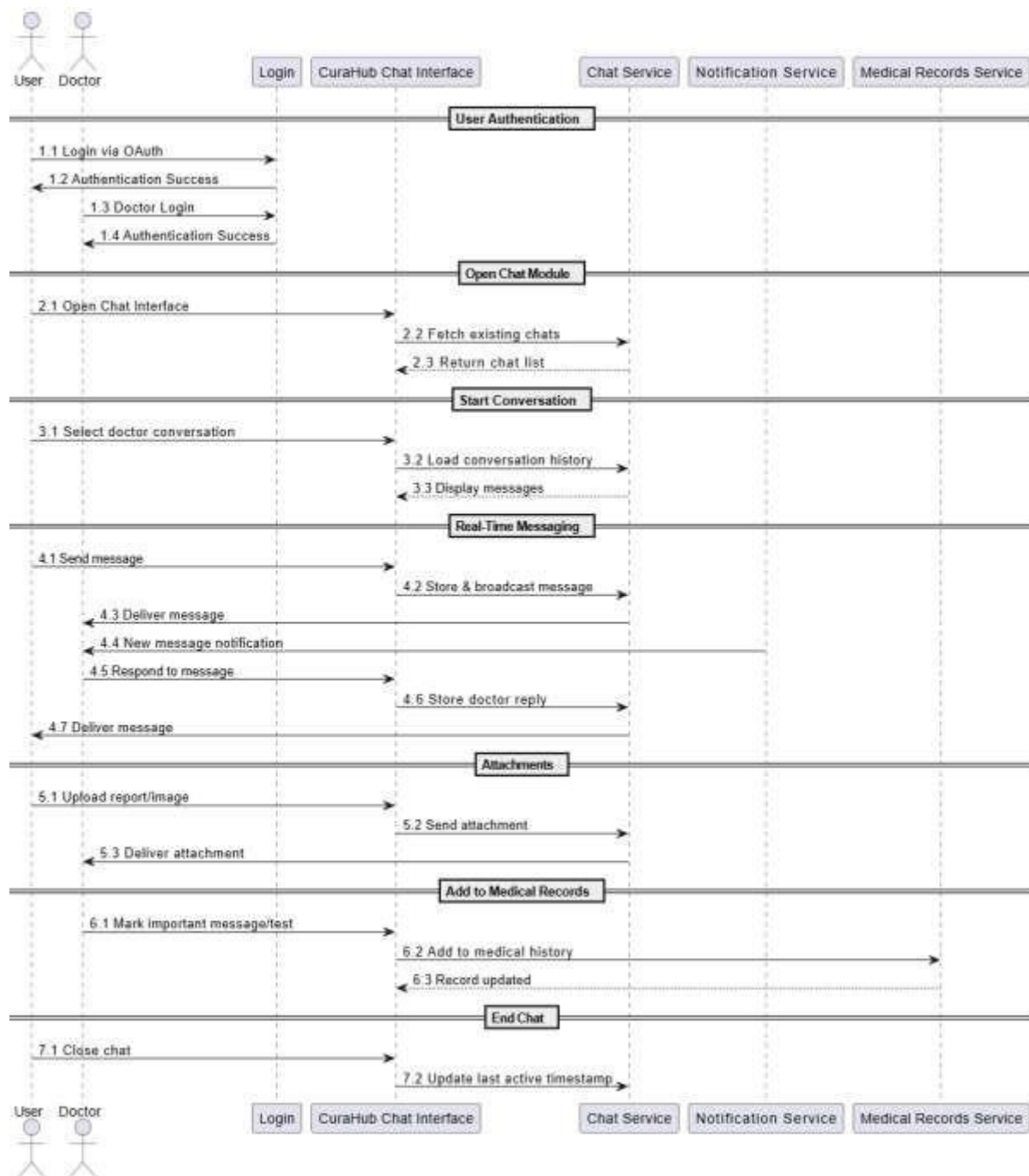


Figure 4.11: Sequence diagram for real-time chat

4.4.2 Class Diagram

A class diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships. It is widely used in object-oriented systems to visualize, document, and construct executable code. Class diagrams describe the static view of an application and are crucial for modeling system functionalities and collaborations.

4.4.2.1 Class Diagram

The CuraHub class diagram provides a comprehensive representation of the system's object-oriented architecture designed for a mobile and web-based healthcare platform. At its core lies the User class, which encapsulates essential attributes such as `userId`, `password`, `name`, and `email`, along with key methods including `register()`, `login()`, `viewDepartments()`, `bookAppointment()`, and `chatWithDoctor()`. Users interact with multiple modules such as Chat, Call, Appointment, Payment, DiseaseDetection, AIChatbot, Doctor, and Department, each represented by a distinct class. These classes define domain-specific attributes and functions—for example, the DiseaseDetection class handles `analyzeSymptoms()` and `generateReport()` methods, while the Chat and Call classes manage real-time messaging and video/audio communication between patients and doctors.

The Payment class manages transaction processing through methods like `processPayment()` and `generateReceipt()`, ensuring secure financial handling, whereas the Appointment class enables booking, cancellation, and status updates for consultations. The AIChatbot class supports automated patient guidance by responding to queries and providing health tips. The Doctor class includes attributes for `doctorId`, `name`, `specialization`, `qualifications`, and `availability`, along with methods to `viewProfile()`, `respondToChat()`, and `acceptAppointment()`. The Department class organizes doctors into specialties and provides methods like `listDoctors()` and `getChatbot()` for structured access[10].

The Admin class stands out with elevated privileges, including `login()`, `approveDoctor()`, `manageDepartments()`, and `viewAnalytics()`, allowing oversight of the entire system. Each module is associated with the User class, highlighting that users are the primary actors who generate, access, and interact with medical data, while the Admin supervises and manages these interactions. Relationships are clearly defined, such as one-to-many associations between users and their appointments, chats, payments, or medical cases, ensuring scalability and efficient data management.

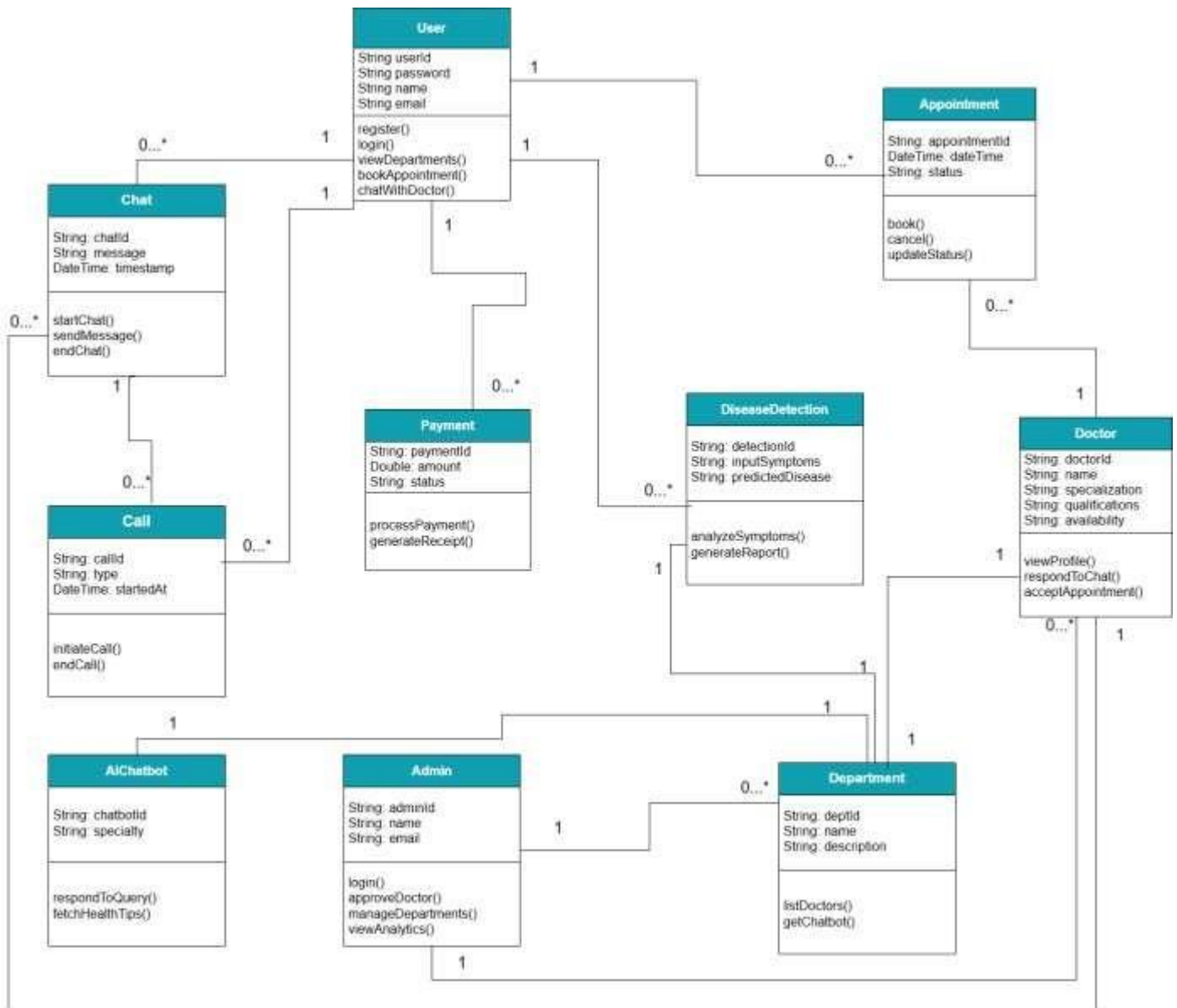


Figure 4.12: Class Diagram

User Management: The User Management module in Curahub handles the complete authentication and identity management of patients using the platform. The *User* class acts as the primary entity, containing key attributes such as userID, name, email, password, age, and medical history reference IDs. It provides essential methods that allow users to register, log in, log out, update their profiles, and view their health-related information.

Since Curahub is a healthcare platform, the module ensures strict security and privacy, leveraging Firebase Authentication to manage credentials securely. The Registration component validates user input, checks email uniqueness, and ensures that only legitimate user accounts are created. The Login component focuses on verifying user credentials and granting secure access to the system.

Disease Detection and Health Assessment Module: The Disease Detection module empowers users to receive intelligent, data-driven health assessments based on their symptoms. The *DiseaseDetection* class includes core attributes such as detectID, inputSymptoms, medicalHistoryReference, predictionResult, and riskLevel, representing the outcome of each diagnostic request. It provides methods to analyze symptoms, run machine-learning prediction models (Regression, ANN, and CNN), generate structured reports, and recommend relevant medical departments or specialists. This module enhances healthcare decision-making by offering preliminary screening and insights before a user consults a doctor. It supports early risk assessment, guides patients toward the correct medical department, and increases awareness about potential medical concerns.

Appointment Management: The appointment management module handles scheduling and tracking of patient-doctor consultations. Functions include booking, updating, canceling, and confirming appointments. This module ensures smooth and organized consultation management and helps both patients and doctors manage their schedules effectively.

AI Assistant: The AI chat assistant module provides intelligent support to users through real-time chat. The class includes attributes such as assistantID, department, userQuery, and response. Functions include analyzing user questions, providing recommendations, and guiding users to appropriate services or specialists. This module helps users get immediate, AI-driven guidance and enhances the overall healthcare experience in CuraHub.

Doctor Module: The Doctor class handles information related to medical professionals, including doctorID, name, specialization, qualifications, and availability. Doctors can view patient profiles, respond to chats, participate in video calls, and accept or decline appointments. This module supports medical practitioners in managing their consultations effectively. It ensures that patients interact only with verified, specialized doctors, enhancing trust and reliability across the platform. Doctors are also linked with their respective departments, maintaining structured medical categorization.

Department Management: The Department class organizes medical specialties such as Cardiology, Nephrology, Endocrinology, Oncology, and more. Each department holds deptID, name, and description, and connects to both doctors and AI chatbots. This module ensures structured categorization of healthcare services, allowing users to navigate easily based on their medical needs. It also supports department-level analytics and recommendations through linked AI modules.

Real-Time Calling: The Call class supports audio or video calls between users and doctors. Attributes include call type, and start time. Functions allow the system to initiate and end calls securely. This module is essential for face-to-face remote consultations, enabling doctors to examine symptoms more accurately while providing patients with interactive, real-time medical support. Calls complement chats and appointments, ensuring comprehensive virtual healthcare.

Payment Module: The PaymentModule handles all transactions within CuraHub using Stripe. The class includes attributes such as paymentID, userID, amount, paymentMethod, and status. Functions include processing payments, confirming transactions, and storing payment records. This module ensures secure, reliable, and smooth payment experiences for consultations and other services.

Chapter 5

IMPLEMENTATION

5 Implementation

The implementation phase of CuraHub involves translating the design into a functional, fully operational healthcare platform using modern tools, frameworks, and technologies. This chapter outlines the step-by-step development of the system, describing the chosen backend and frontend technologies, database schema, API integration, and module-wise coding strategies. Flutter and Dart were used for the mobile frontend, while Firebase handles backend services including authentication, real-time database, and storage. Node.js is used for API-based services such as payment integration (Stripe) and AI model communication. Each module, including user registration, medical case submissions, AI-powered disease detection, real-time consultations, and the Q&A health forum, is developed independently and later integrated into a seamless user experience through proper routing, middleware, and admin management.admin.

5.1 Algorithm

This section outlines the key algorithms and logic used across the major modules of CuraHub. These algorithms are designed to handle user interactions, medical case submissions, AI-powered disease detection, real-time doctor-patient consultations, Q&A health forum discussions, and payment or notification workflows with clarity and efficiency.

5.1.1 User authentication

Function: Handles user registration/login via Google, Email, or other OAuth providers.

Input: Username (string), password (string) / OAuthProvider (Google, Email)

Output: User login or new account creation

1. BEGIN
2. IF user clicks "Login with OAuthProvider" THEN
3. Redirect to OAuthProvider authentication page
4. Receive access token and user data
5. IF user exists in CuraHub database THEN
6. Log the user in
7. ELSE
8. Create a new user in CuraHub database with retrieved data
9. END

5.1.2 Social Posting

Function: Allows users (patients, doctors, or healthcare contributors) to create, share, and interact with posts.

Input: User content (text, optional image or medical document)

Output: Medical update submitted, shown in community feed, and interactions recorded.

Pseudocode

1. BEGIN
2. IF user is authenticated THEN
3. Display update/case submission form
4. IF form is submitted THEN
5. Save update to database with content, optional image, and user ID
6. Show update in community feed
7. ENDIF
8. FOR each each update displayed DO
9. Enable reactions, feedback, and discussion options
10. Record each interaction in the system
11. END

5.1.3 Interactive Health Consultations

Function: Submit medical cases for AI-assisted.

Input: Case details and symptoms.

Output: Feedback status with recommendations.

Pseudocode

1. BEGIN
2. Display all consultation cases
3. WAIT for user selection or new case submission
4. Show case details and input form
5. IF user submits suggestion THEN
6. Send to AI for analysis and RECEIVE feedback
7. Validate submission and update user progress
8. ELSE provide hints or guidance
9. END

5.1.4 Live Consultation Simulation

Function: Enables two users to join a real-time session.

Input: Host and participant info

Output: Shared session with interaction and tracking

Pseudocode

1. BEGIN
2. Host initiates live session
3. Generate secure access link for participant
4. Load shared interface for both users
5. Start session timer
6. IF participant navigates away from interface THEN
7. Trigger warning or temporarily pause session
8. END

5.1.5 Real-Time Chat & Messaging Algorithm

Function: real-time communication with private chats, group rooms, file sharing, and moderation.

Input: Messages, attachments, emojis, user actions

Output: Real-time communication

Pseudocode

1. BEGIN
2. User creates or joins chat room
3. User sends a message
4. Store message in database
5. Send it to other users in the chat
6. Allow attachments and emoji support
7. END

5.2 External APIs

Table 5.1 provides a detailed overview of the external APIs integrated into the CuraHub mobile application. These APIs play a crucial role in enabling core functionalities such as user authentication, data storage, real-time communication, media handling, notifications, location services, and AI-powered health assistance. Firebase services—including Authentication, Firestore, Realtime Database, Storage, and Cloud Messaging—form the backbone of the app by managing user login, securely storing health records and chat data, enabling instant messaging, handling media uploads, and delivering push notifications. Additionally, Google Maps API supports location-based features such as identifying nearby hospitals and emergency assistance. The Google Gemini API is used to provide intelligent health suggestions and AI-driven chat responses within the app. Together, these APIs enhance the overall performance, reliability, and intelligence of CuraHub, ensuring a seamless and efficient user experience.

These APIs also enable scalability, allowing the app to handle a growing number of users and medical data without performance degradation. By leveraging cloud-based services, CuraHub ensures high availability, security, and minimal latency in delivering real-time updates. Moreover, the integration of AI and location-based APIs allows the platform to offer personalized and context-aware healthcare recommendations to patients, making it a comprehensive digital health solution.

Additionally, Google Maps API supports location-based features such as identifying nearby hospitals and emergency assistance. The Google Gemini API is used to provide intelligent health suggestions and AI-driven chat responses within the app. Together, these APIs enhance the overall performance, reliability, and intelligence of CuraHub, ensuring a seamless and efficient user experience. The Google Gemini API is used to provide intelligent health suggestions and AI-driven chat responses within the app. Together, these APIs enhance the overall performance, reliability, and intelligence of CuraHub, ensuring a seamless and efficient user experience. Firebase services—including Authentication, Firestore, Realtime Database, Storage, and Cloud Messaging—form the backbone of the app by managing user login, securely storing health records and chat data, enabling instant messaging, handling media uploads. These APIs play a crucial role in enabling core functionalities such as user authentication, data storage, real-time communication, media handling, notifications, location services, and AI-powered health assistance. Firebase services—including Authentication, Firestore, Realtime Database, Storage, and Cloud Messaging.

Table 5.1: External APIs

Name of API	Description of API	Purpose of Usage	Function/Class Name Where Used
Firebase Authentication API	Firebase Auth API for user login, signup & secure access	User authentication using Email/Password, Phone, or OTP	AuthService, LoginScreen, SignupScreen
Firebase Firestore API	NoSQL cloud database API provided by Firebase	Stores user profiles, consultations, chat history, appointments	FirestoreService, ChatRepository, ConsultationService
Firebase Realtime Database API	Real-time cloud database by Firebase	Used for instant chat updates, live typing indicators	RealtimeDBService, ChatScreen
Firebase Storage API	Cloud file storage API	Uploading and retrieving profile images, medical documents, chat attachments	StorageService, ProfileController, UploadService
FCM – Firebase Cloud Messaging API	Push notification API by Firebase	Sends alerts for new messages, doctor replies, and reminders	NotificationService, FCMService
DeepSeek API (AI)	DeepSeek’s advanced generative AI	Provides AI-powered symptom checking, intelligent health recommendations, and real-time chatbot responses	AIService, DeepSeekHandler, ChatbotScreen

5.3 User Interface

The CuraHub user interface (UI) is designed for simplicity and ease of use, with a clean and intuitive layout that enhances user engagement.

5.3.1 Splash Screen

The screen shown is the Splash Screen for the "Cura Hub" mobile application. It features a simple, highly branded design with a bright blue background contrasted against a white, stylized logo and the app's name centered prominently. This screen serves the crucial function of displaying the app's identity and professional aesthetic to the user while simultaneously acting as a visual.



Figure 5.1: Splash Screen

5.3.2 Entry Screen

This screen, identified as the Role Selection/Welcome Screen for Cura Hub, serves as the critical entry point following the initial splash screen. It maintains the app's professional branding with the centered logo and name, welcoming users with a concise tagline. The screen's primary objective is to differentiate user types, presenting clear options for both Login and Sign Up based on whether the user is acting as a Doctor or a Patient.

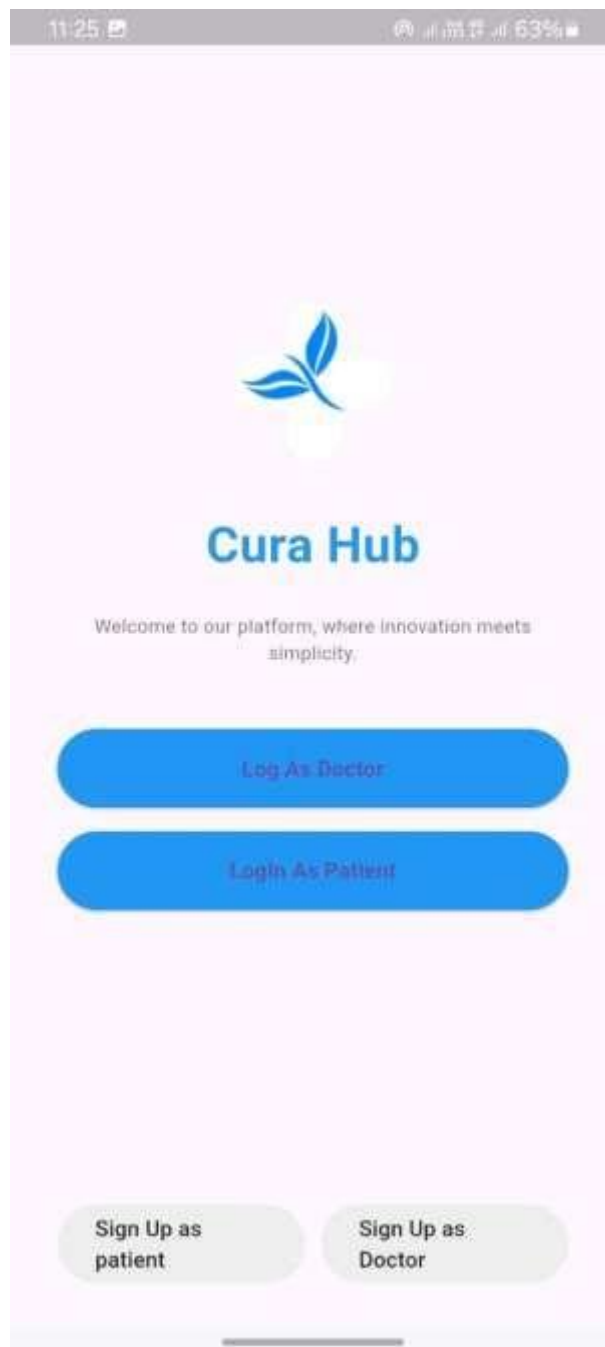


Figure 5.2: Entry Screen

5.3.3 User Sign up

This screen is the comprehensive New Account/Registration Form for the Cura Hub application, designed to collect all necessary personal details from a new user. It prompts for critical inputs, including Full Name, Password (with confirmation), Email, Mobile Number, and Date of Birth.

11:25 63%

Log In

Welcome
Please log in to continue accessing your health dashboard.

Email or Mobile Number

Password

[Forget Password](#)

Log In

or sign up with

Don't have an account? [Sign Up](#)

Figure 5.3: Login Screen

5.3.4 Doctor Sign up

This screen is the Doctor Sign Up Screen for the Cura Hub application, specifically designed to register medical professionals onto the platform. The form collects basic information such as the doctor's name, age, phone number, email, and password, but its key distinction lies in the sections dedicated to uploading professional documentation. These mandatory fields include a DMC Scan (likely for medical council registration), a Degree Scan, and a required Selfie Capture for visual identity verification, ensuring the legitimacy and certification of the doctors joining the service.

11:25 AM 63%

←

New Account

Full name

Enter your full name

Password

Enter your password

Retype Password

Retype your password

Email

example@example.com

Mobile Number

03XX-XXXXXXX

Date Of Birth

DD / MM / YYYY

By continuing, you agree to [Terms of Use](#) and [Privacy Policy](#).

Sign Up

or sign up with

Already have an account? [Log in](#)

Figure 5.4: Doctor Sign Up

5.3.5 Patient Home Screen

This screen is the main Patient Dashboard or Home Screen, which serves as the central hub for users after they have successfully logged in, greeting the user personally at the top. It features a persistent bottom navigation bar for accessing core features and includes a central search bar for finding specific doctors or medical specialties. A distinctive feature is the prominent, purple "Meet Your AI Doctor" button, suggesting the integration of a virtual health assistant.



Figure 5.5: Patient Home Screen

5.3.6 Booking Appointment

This screen is a Date Selection Modal displaying an interactive calendar for November 2025, which allows the user to easily pick a specific date (currently set to Saturday, November 15) before proceeding with the "Book Appointment" or "Chat with Doctor" actions visible beneath the pop-up.

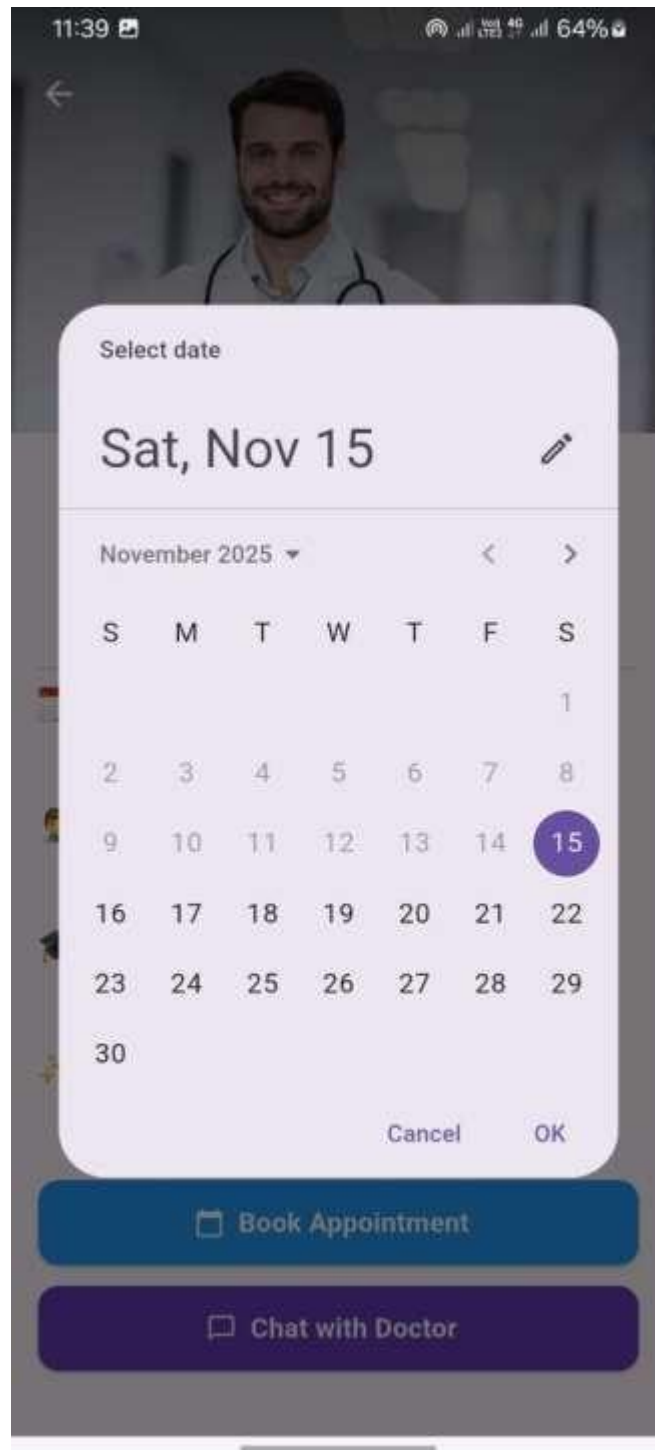


Figure 5.6: Booking Appointment

5.3.7 Select Appointment Time Screen

This screen combines the Doctor Profile View for "kashan" (General) with an active Time Selection Modal to facilitate appointment booking. The profile section shows the doctor's name, specialization, a rating status (N/A), and patient count (0 Patients), along with dedicated sections for Schedule (currently "Not Available") and Profile information. The overlaying modal prominently displays the title "Select Appointment Time" and uses a scrollable picker interface to allow the user to choose the precise hour and minute before confirming the time with the blue "Confirm Time" button.

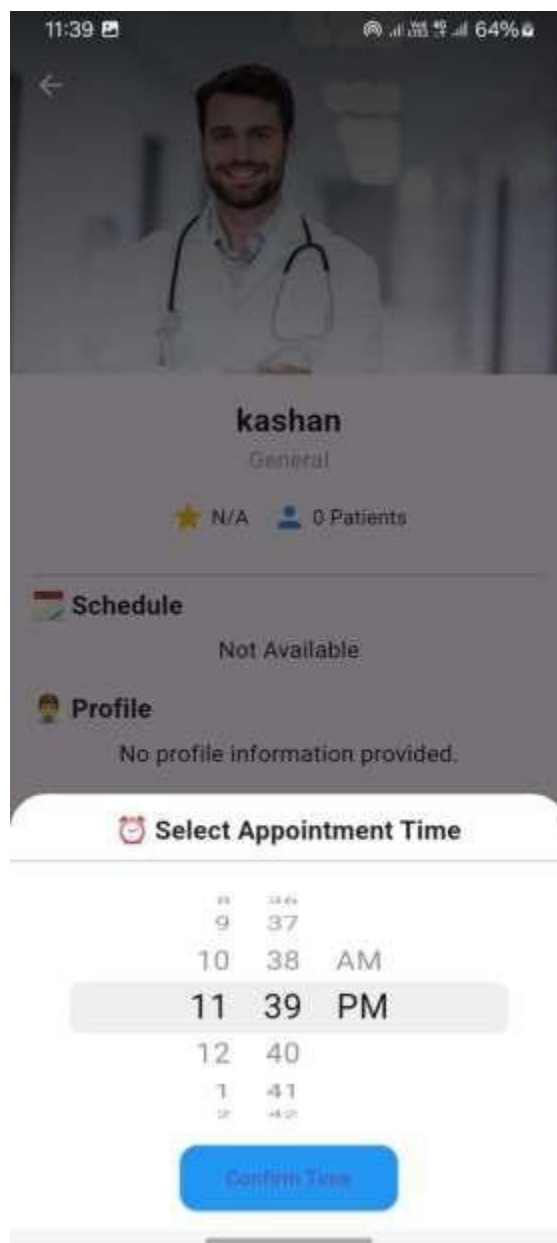


Figure 5.7: Select Appointment Time screen

5.3.8 Favorite Doctor Screen

This screen is the dedicated Favorites Screen, accessible via the heart icon in the bottom navigation bar, and it displays a curated list of doctors saved by the user. Each entry in the list shows the doctor's profile picture, name (e.g., Haseeb, Dr Zamara Tauseef), and a status line (N/A, likely for rating or specialty). The primary function of the screen is to allow easy access to these preferred professionals, with a red trash can icon next to each name providing a simple mechanism to remove the doctor from the saved list.

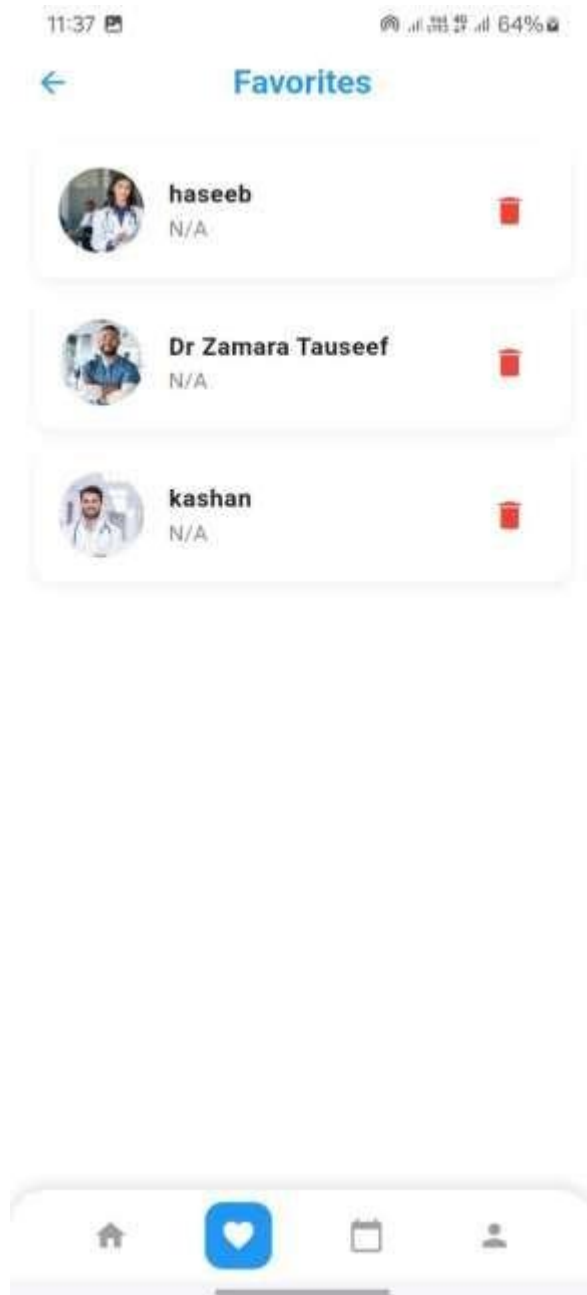


Figure 5.8: Favorite Doctor screen

5.3.9 Appointment Management Screen

This screen, titled "My Appointments," is the central hub for managing a user's scheduled doctor visits and is organized using three primary filter tabs: Completed, Upcoming (currently selected), and Cancelled. The main area displays a list of appointment cards, with each entry showing the doctor's name, the date and time of the appointment, and the current Status: Booked date. Additionally, users are provided with a clear "X" icon on each card, likely indicating a function to quickly cancel or remove an upcoming appointment directly from this view. The selected Upcoming tab ensures that the user's immediate and future schedule is prioritized and easily accessible.

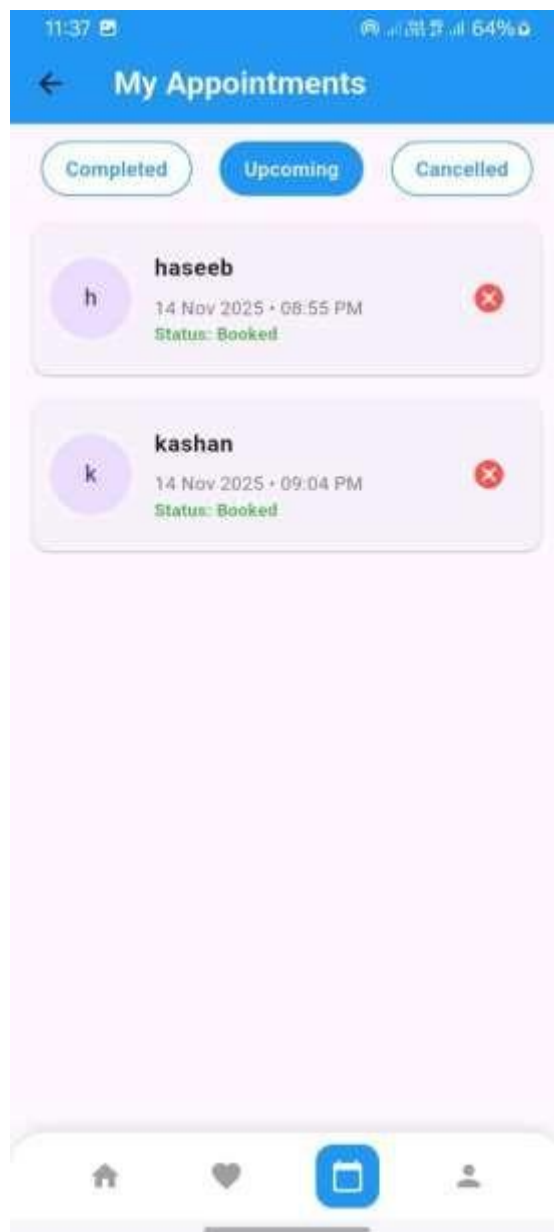


Figure 5.9: Appointment Management screen

5.3.10 Setting Screen

This screen, titled "Settings," is the dedicated control panel for managing essential app features, presenting three primary configurable options to the user. These options include Notification Setting (to control alerts), Password Manager (for security and credential updates), and a direct link to Delete Account (for complete account removal). This page provides the user with centralized access to modify core aspects of their application experience and security preferences.

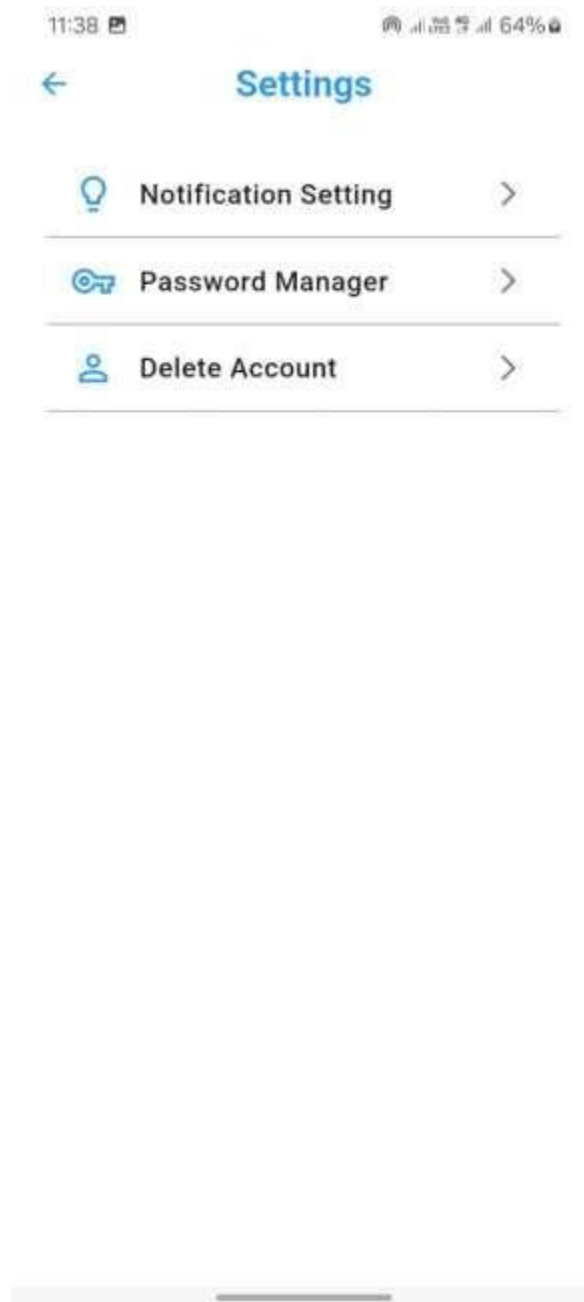


Figure 5.10: Setting screen

5.3.11 Notification Settings Screen

This screen is the dedicated Notification Setting Screen, which provides the user with granular control over the types of alerts they receive from the application. It utilizes clear toggle switches to allow the user to enable or disable notifications for various categories, including general alerts, sound options, Payments, and promotional content like Special Offers and Promo And Discount. The inclusion of separate controls for Sound Call and Vibrate allows for highly customized interruption settings. Furthermore, specific categories like Cashback demonstrate the app's focus on transactional and marketing communications that the user can opt-in or opt-out of.

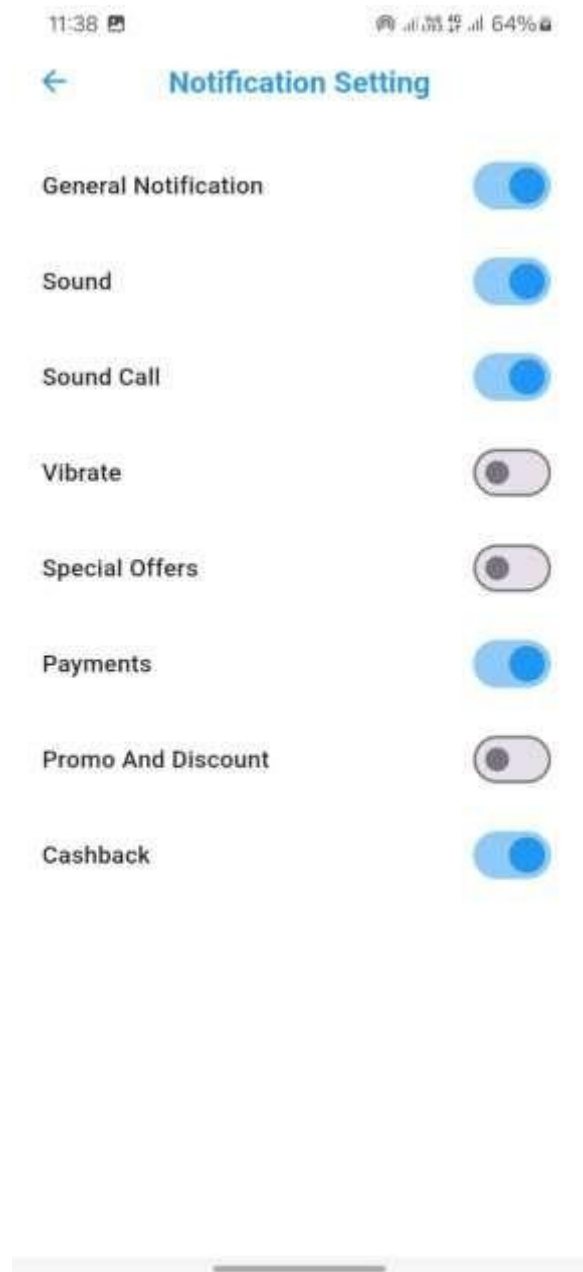


Figure 5.11: Notification Setting Screen

5.3.12 Change Password Screen

This screen, titled "Password Manager," is dedicated to enabling the user to update their security credentials, requiring the input of their Current Password followed by the New Password and Confirm Password fields. It prominently features a large, blue "Change Password" button to execute the update and includes a link for "Forgot Password?" for easy recovery access. The design is simple and focused, utilizing an eye icon next to each password field, which provides the user with the important ability to toggle password visibility for better input accuracy and security assurance.

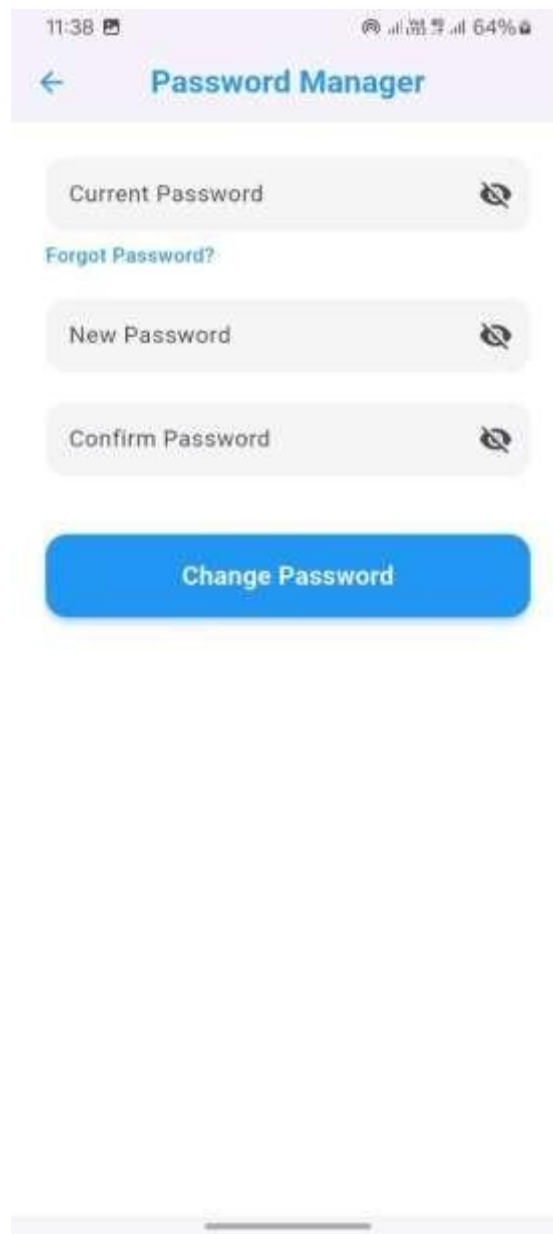


Figure 5.12: Change Password Screen

5.3.13 Password Reset Screen

This screen, titled "Forgot Password," guides the user through the account recovery process, requiring them to enter their registered email address into a dedicated field. The primary action is the "Send Reset Link" button, which initiates the process of emailing a link to the user to securely reset their password, and a "Login" link is also provided for users who remember their password.

A screenshot of a mobile application's "Forgot Password" screen. The screen has a light purple header bar with a back arrow on the left and the title "Forgot Password" in the center. Below the header is a large blue circular icon containing a white padlock and a circular arrow. The main heading "Reset Your Password" is centered below the icon. Underneath, a message reads: "Enter your registered email address below. We'll send you a link to reset your password." This is followed by a text input field with a grey envelope icon on the left and the placeholder text "Email Address". Below the input field is a prominent blue button with the text "Send Reset Link". At the bottom of the screen, there is a link that says "Remembered Password? Login". The status bar at the very top shows the time as 11:38, signal strength, 4G network, and 64% battery.

Figure 5.13: Password Reset Screen

5.3.14 Account Deleting Warning

This screen is a critical Account Deletion Warning that explicitly asks the user for final confirmation ("Are you sure you want to delete your account?"). It emphasizes the permanence of the action, warning that "Deleting your account will remove all your data permanently," and offers two clear buttons: a calming "Cancel" (blue) and a final, irreversible "Delete Account" (red) action.

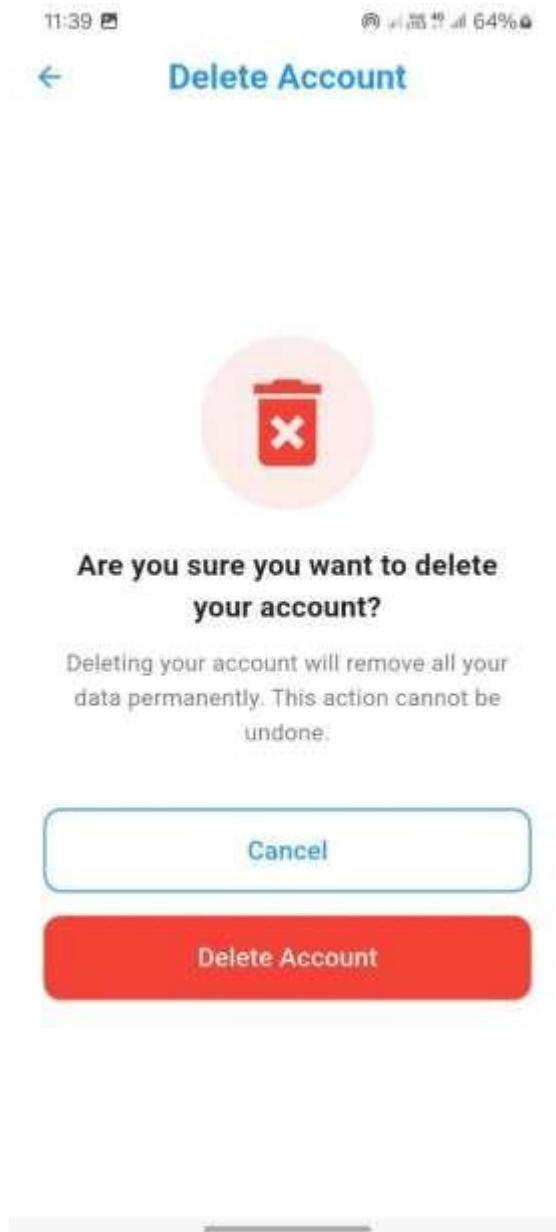


Figure 5.14: Account Deleting Screen

5.3.15 AI Specialist Screen

This screen is dedicated to connecting the user with advanced digital health services and medical professionals, prominently featuring a large, purple button labeled "Smart Diagnosis" to initiate an AI-driven health assessment. Beneath this primary feature, the screen provides a section to "Explore Specialties," listing various medical fields such as Gynecologist, Dermatologist, Cardiologist, and Neurologist. This structure allows users to either utilize the automated AI tool or manually select a specific type of specialist for consultation.

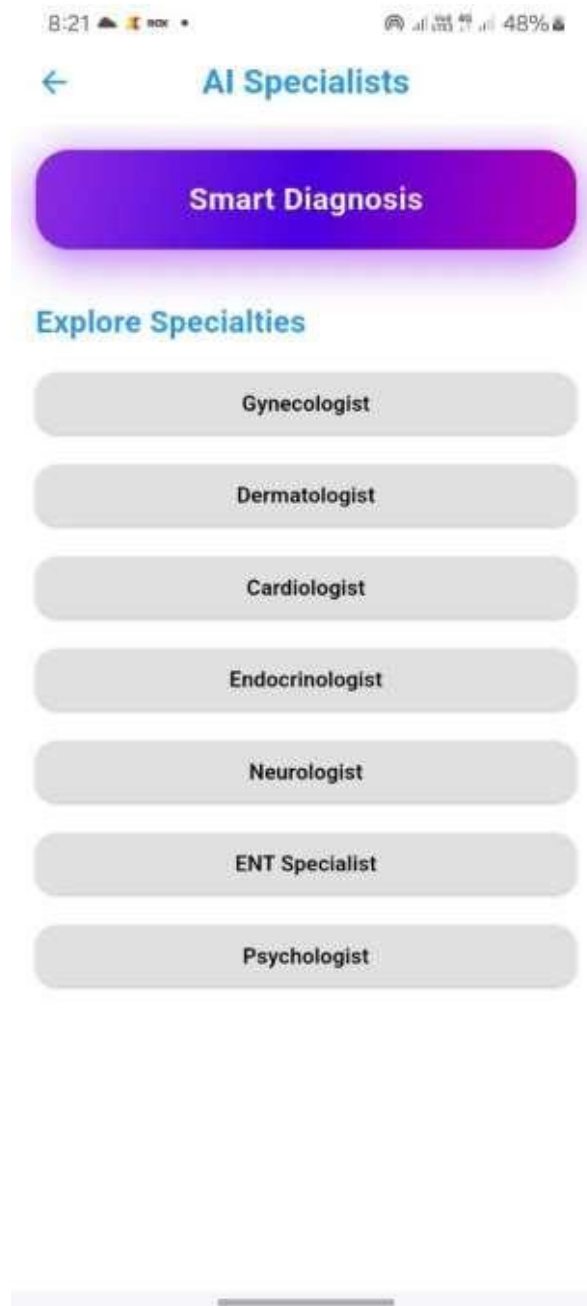


Figure 5.15: AI Specialist screen

5.3.16 AI Medical Assistant Screen

This screen, titled "All Medical Fields," serves as the primary chat interface for the app's integrated medical AI assistant, asking the user "How can I help you today?". The large central area is reserved for the chat history, while the bottom features a clear text input field labeled "Type your message..." and a send icon for user interaction. A small but important disclaimer at the bottom notes that this is an "Educational demo – not a substitute for professional medical advice," clearly setting expectations for the AI's role.

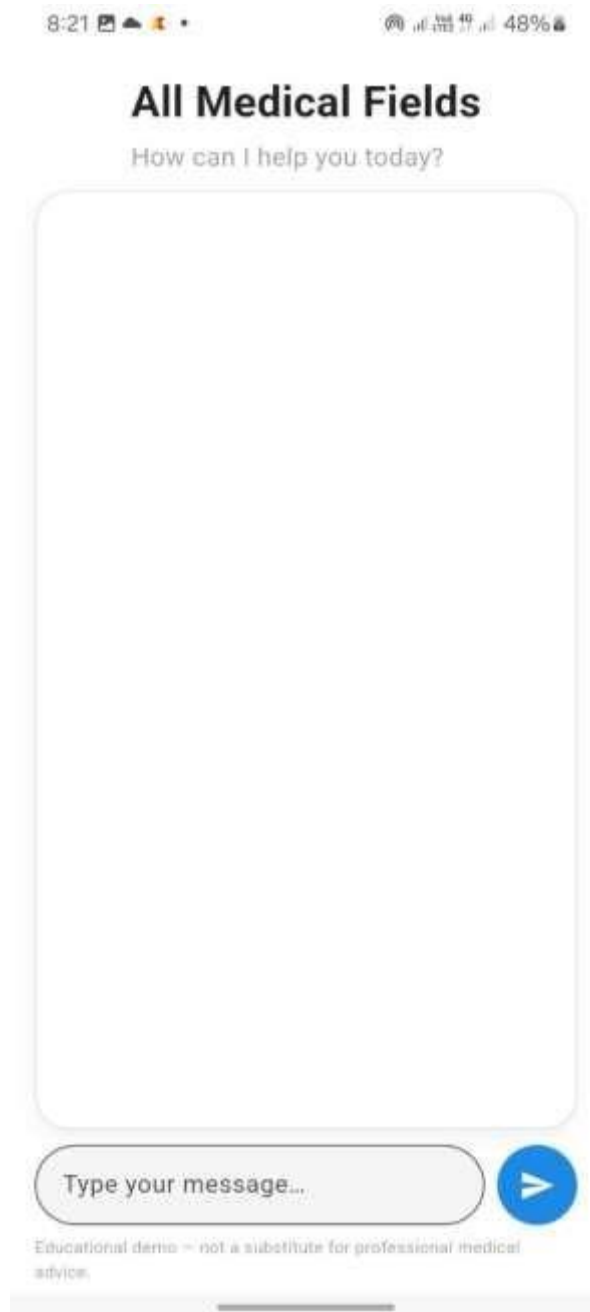


Figure 5.16: AI Medical Assistant Screen

5.3.17 Specialized AI Doctor

This screen represents a specialized chat interface dedicated to a particular medical field, as evidenced by the title "Cardiologist," where a user is consulting the application's integrated medical AI. The chat history displays a user's symptom query and a detailed, context-aware response from the AI that successfully connects the patient's non-cardiac symptom (bone pain) to potential related heart issues like anemia or medication side.

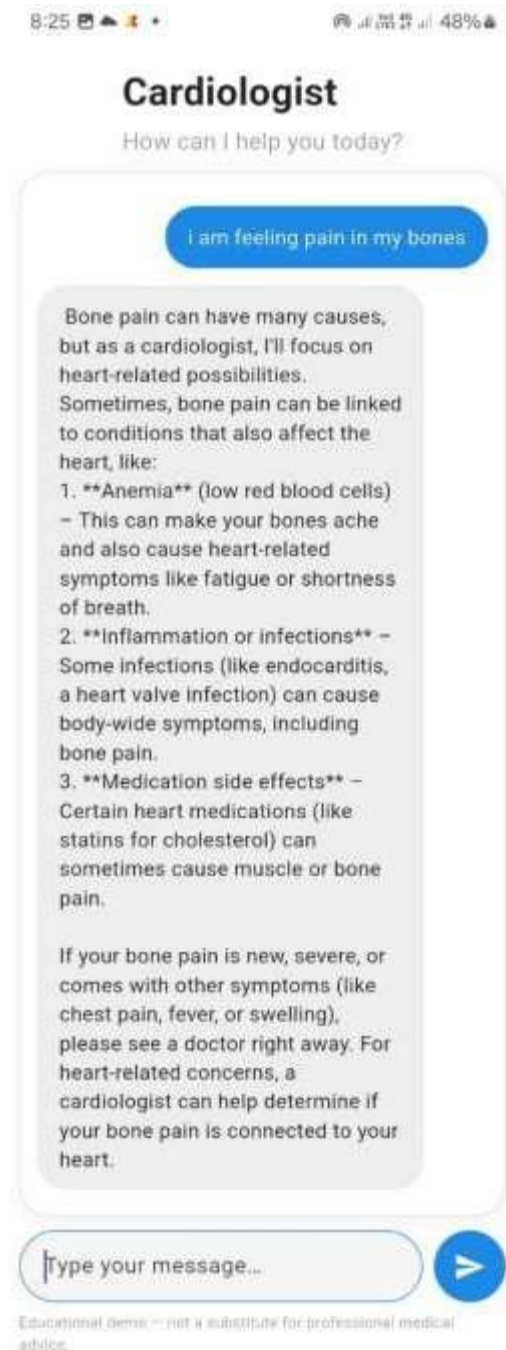


Figure 5.17: Specialized AI Doctor

5.3.18 AI Specialist Screen

This screen is dedicated to connecting the user with advanced digital health services and medical professionals, prominently featuring a large, purple button labeled "Smart Diagnosis" to initiate an AI-driven health assessment. Beneath this primary feature, the screen provides a section to "Explore Specialties," listing various medical fields such as Gynecologist, Dermatologist, Cardiologist, and Neurologist. This structure allows users to either utilize the automated AI tool or manually select a specific type of specialist for consultation.

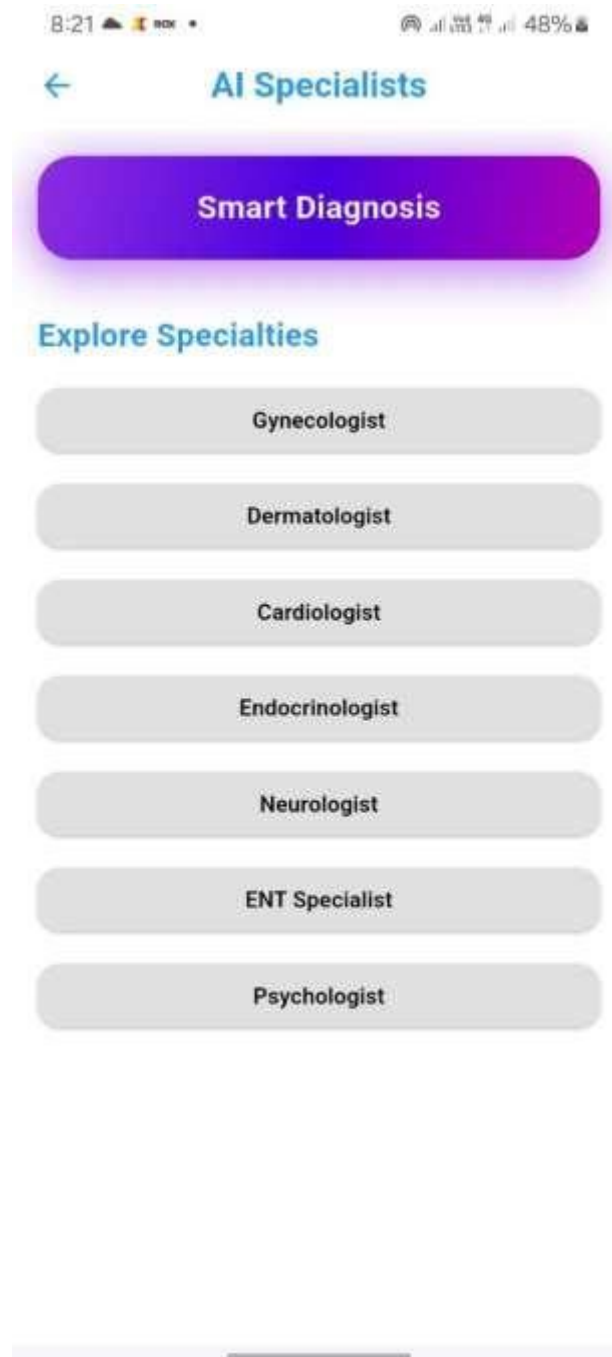


Figure 5.18: AI Specialist Screen

5.3.19 Account Management Screen

This screen is the My Profile Screen, which is the central Account Management Hub for the user, displaying their name and profile picture at the top. The main area presents a menu of essential options, including links to manage personal Profile details, Favorite doctors, Payment Method, and crucial app actions like Settings, Privacy Policy, and Logout.

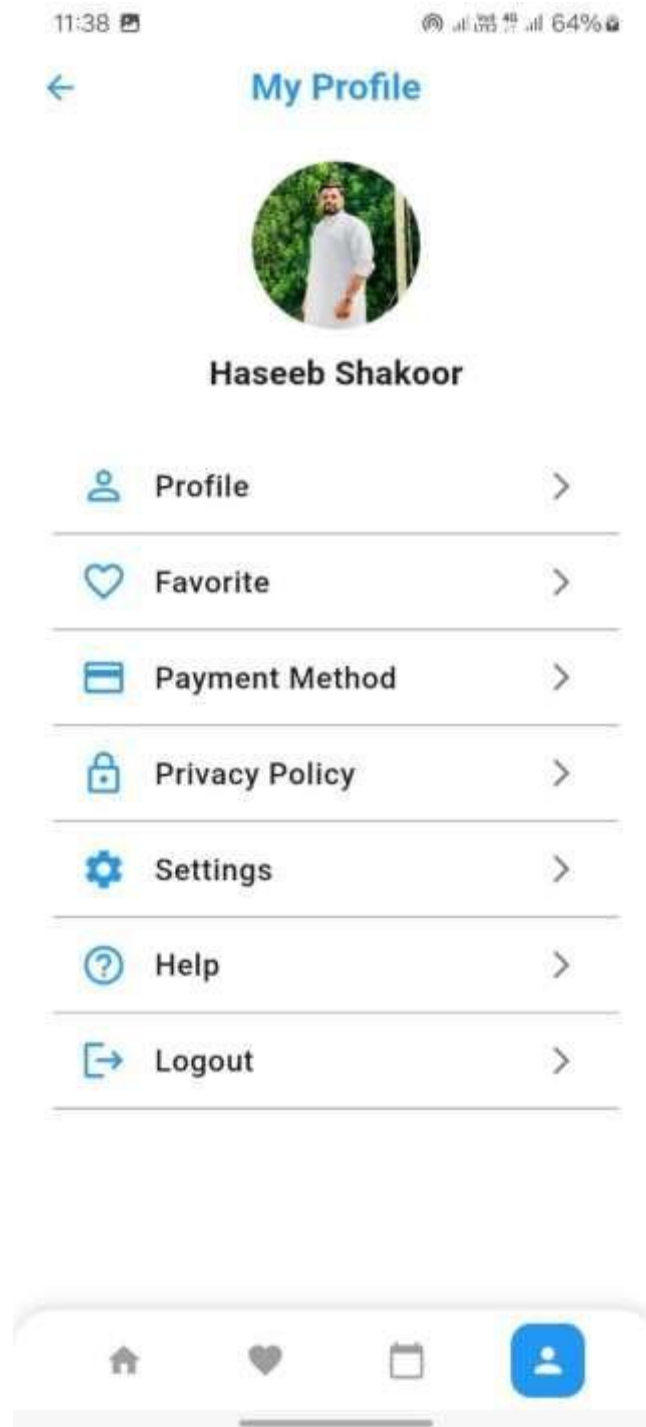
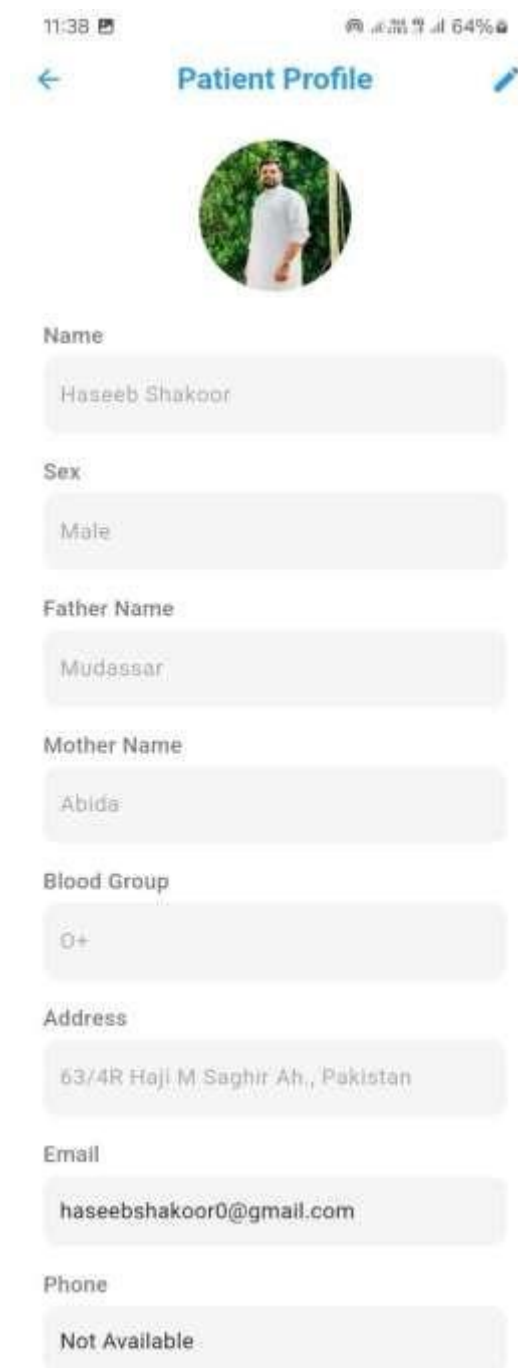


Figure 5.19: Account Management screen

5.3.20 Patient Profile Screen

This screen, titled "Patient Profile," displays the user's complete personal and demographic information, including their Name ("Haseeb Shakoor"), Sex, Father's Name, Mother's Name, Blood Group ("O+"), Address, Email, and Phone number. A prominent edit icon (pencil) is visible in the header, indicating that the user has the ability to modify or update the information displayed in these various non-editable text fields.



The screenshot displays the "Patient Profile" screen. At the top, there is a status bar with the time 11:38 and a battery level of 64%. Below the status bar is a header with a back arrow on the left, the title "Patient Profile" in the center, and an edit icon (pencil) on the right. Below the header is a circular profile picture of a man. Underneath the profile picture are several text input fields, each with a label above it: "Name" (Haseeb Shakoor), "Sex" (Male), "Father Name" (Mudassar), "Mother Name" (Abida), "Blood Group" (O+), "Address" (63/4R Haji M Saghir Ah., Pakistan), "Email" (haseebshakoor0@gmail.com), and "Phone" (Not Available).

Field	Value
Name	Haseeb Shakoor
Sex	Male
Father Name	Mudassar
Mother Name	Abida
Blood Group	O+
Address	63/4R Haji M Saghir Ah., Pakistan
Email	haseebshakoor0@gmail.com
Phone	Not Available

Figure 5.20: Patient Profile Screen

5.3.21 Privacy Terms and Conditions Screen

This screen is dedicated to displaying the app's Privacy Policy and Terms & Conditions, detailing the company's commitment to protecting user data and explaining how information is collected. The screen also clearly outlines the user's responsibilities, such as ensuring the accuracy of information, and states that continued use of the app signifies acceptance of these policies.



Figure 5.21: Privacy Terms and Conditions Screen

5.3.22 Help Center Screen

This screen is the dedicated Help Center, which serves as the primary resource for user support, featuring a central search bar to quickly find answers. The content is organized under the FAQ (Frequently Asked Questions) section, presenting common inquiries like "How do I book an appointment?" and "Can I cancel or reschedule?" with an option to switch to a Contact Us.

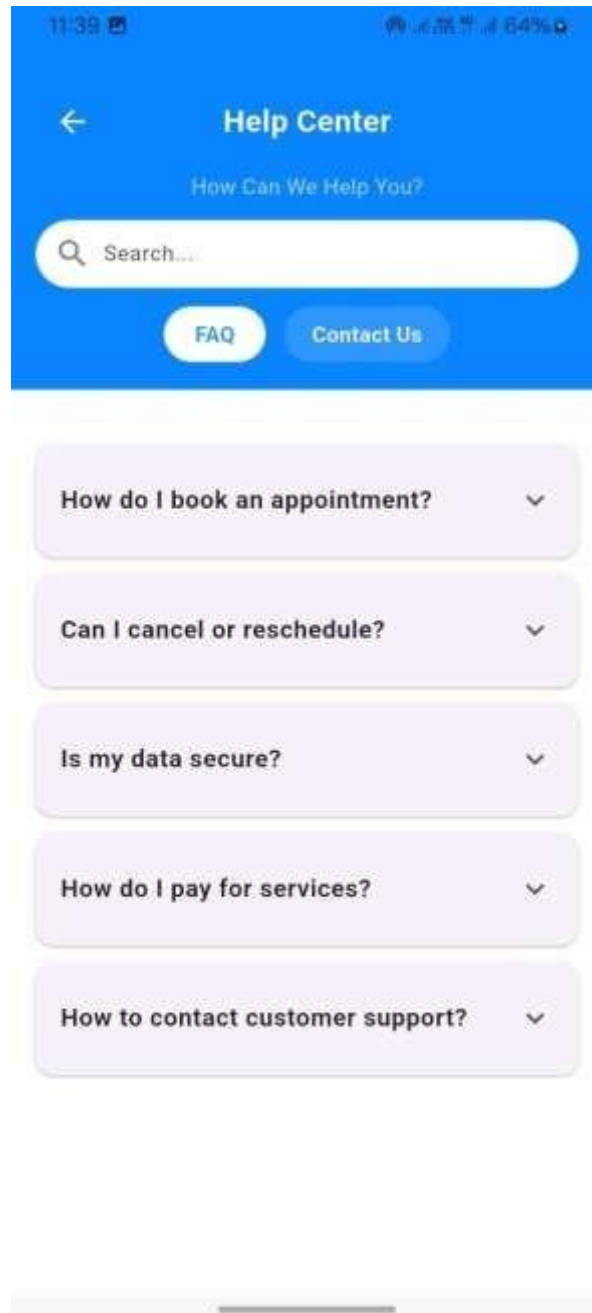


Figure 5.22: Help Center Screen

5.3.23 Contact Us Screen

This screen is the "Contact Us" view within the Help Center, which lists multiple channels for the user to seek support, including Customer Service phone number, the Website URL, and direct contact options via popular social media platforms like WhatsApp, Facebook (HealthLink Support), and Instagram. This centralized list ensures users can choose their preferred method of communication to resolve any issues.

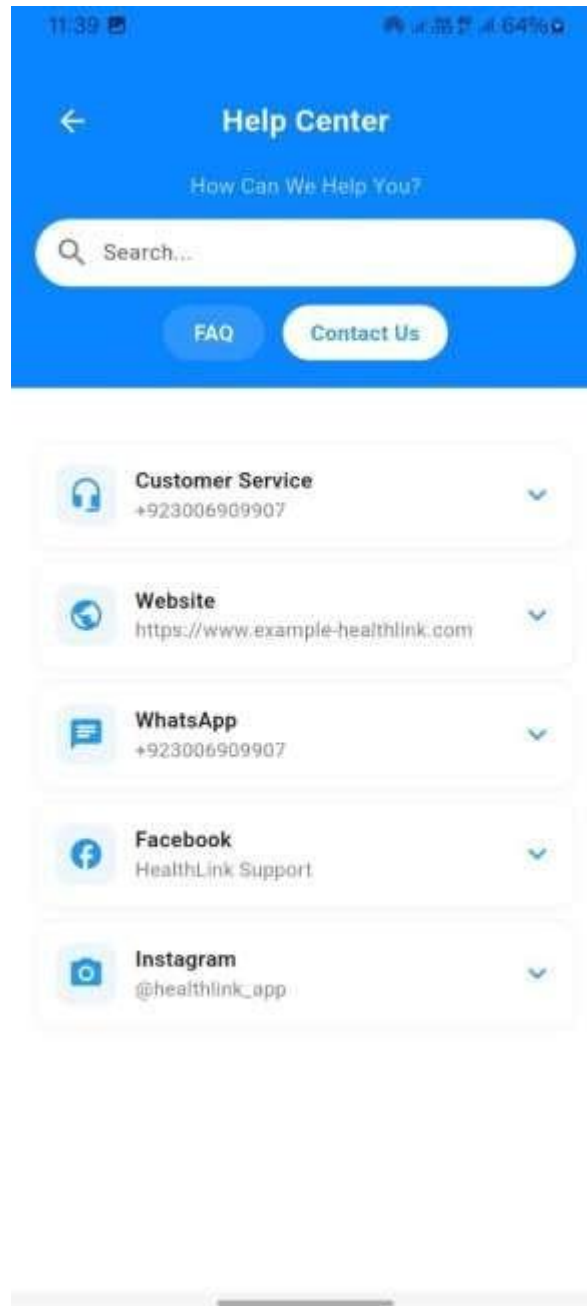


Figure 5.23: Contact Us Screen

5.3.24 Logout Confirmation Screen

This screen is a crucial Logout Confirmation Modal that overlays the user's "My Profile" page, providing a final safeguard before the user exits the session. It asks a direct confirmation question, "Are you sure you want to logout?" to prevent accidental sign-out. The modal offers two distinct, high-contrast buttons: a white "Cancel" button to dismiss the action and a blue "Logout" button to proceed with ending the current user session.

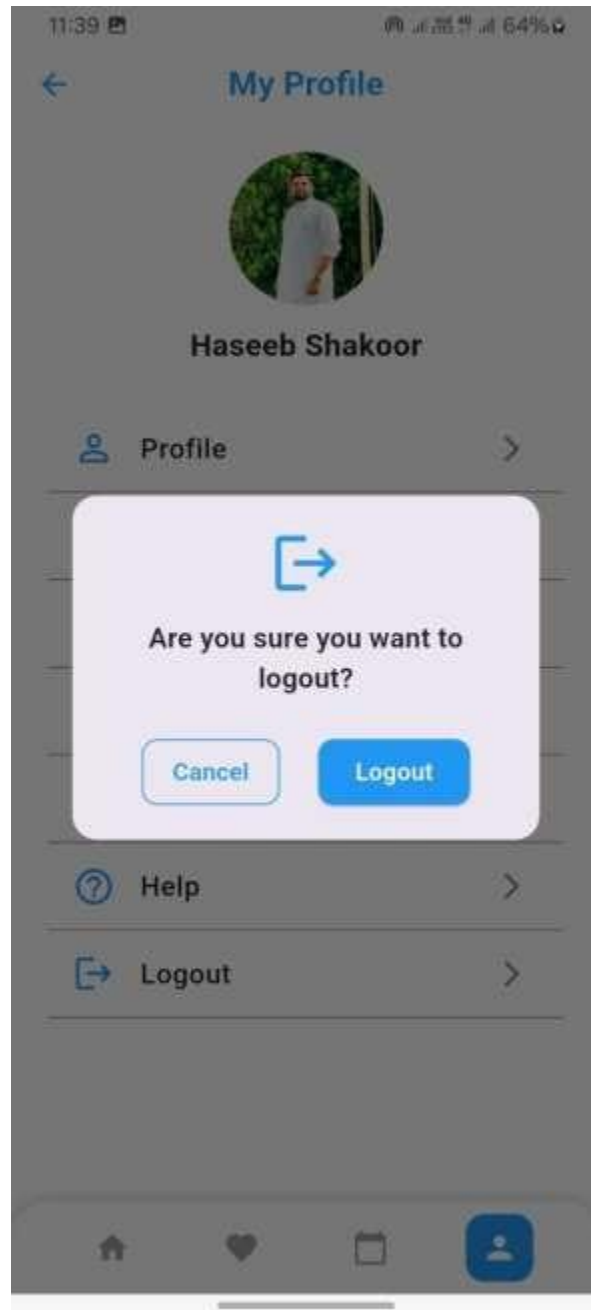


Figure 5.24: Logout Confirmation Screen

5.3.25 Suggested Specialist Screen

This screen appears as a crucial intermediate step following an AI assessment, titled "Suggested Specialists", indicating that the AI has analyzed the user's symptoms and is recommending the next course of action. It prominently features a "Continue Consultation with AI" option for further self-assessment and lists two specific, recommended specialists: Dr. Zamara Tauseef (General Doctor) and kashan (General), complete with their profile pictures.

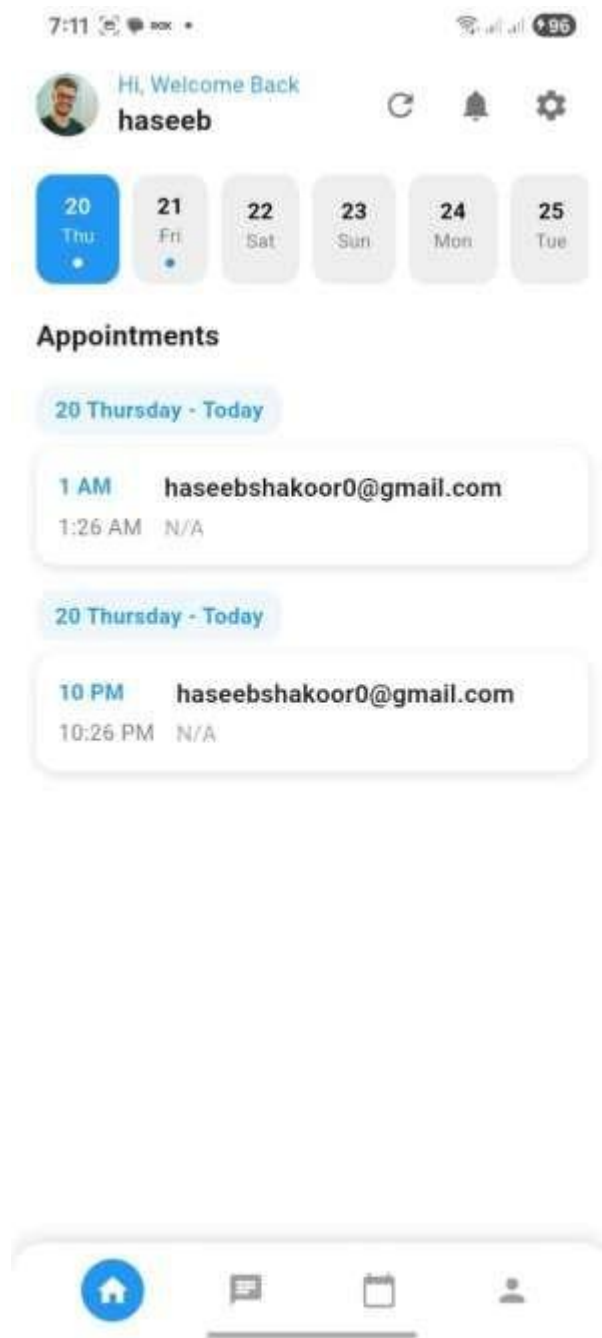


Figure 5.25: Suggested Specialist Screen

5.3.26 Cancel Confirmation Appointment Screen

This screen is a critical Confirmation Modal that overlays the main dashboard, appearing when a user attempts to modify or view an upcoming appointment. It clearly shows the appointment details, including the doctor's name (haseeb), their email, and an "Issue: N/A" field, allowing the user to provide context before cancellation. The primary action presented to the user is a large, high-contrast red "Cancel Appointment" button, which confirms the user's intention to remove the booking from their schedule.

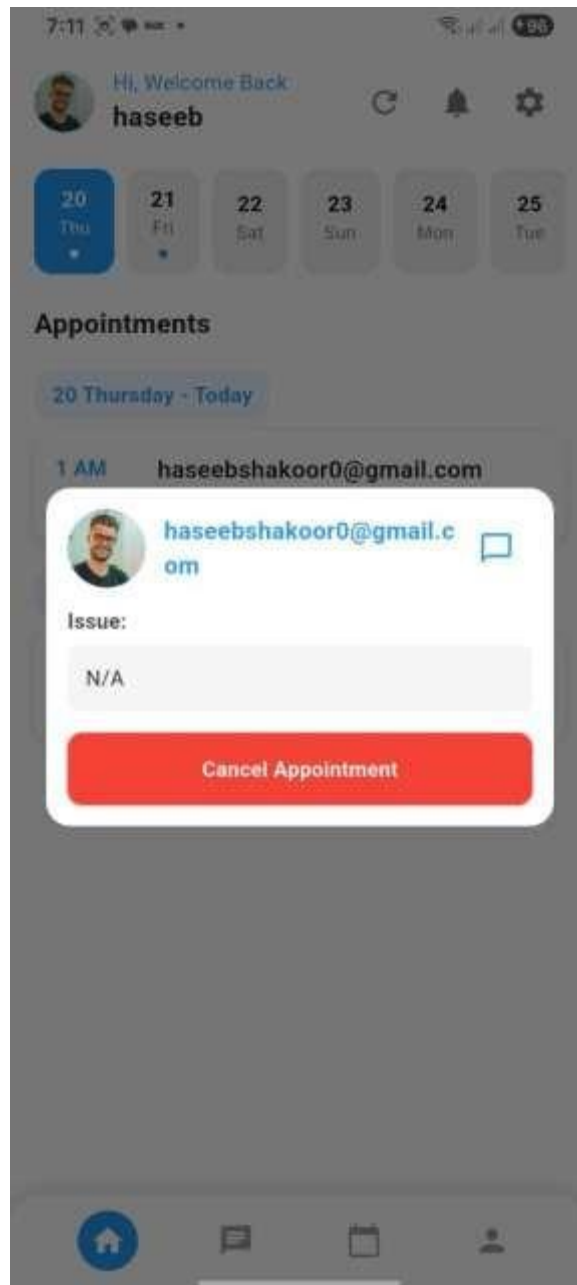


Figure 5.26: Cancel Confirmation Appointment

5.3.27 Doctor Patient Chat Screen

This screen is a live, one-on-one communication interface used for doctor-patient consultations, which displays the patient's email at the top and the clarifying text "You are chatting with your patient." The main area shows the conversation history, indicating multiple messages sent by the doctor ("hello") and a response from the patient ("hi haseeb"). The bottom is dedicated to active interaction, featuring a clear text input field and a send icon, enabling real-time dialogue between the professional and the patient.



Figure 5.27: Doctor Patient Chat Screen

5.3.28 Patient Conversation List Screen

This screen, titled "Your Chats," serves as the central hub for a doctor to manage and access their ongoing conversations with patients, clearly stating, "Talk with your patients." It displays a list of active chat threads, with each entry showing the patient's identifier (e.g., "Patient 1"), the last message sent ("hi haseeb"), and the timestamp of that message (e.g., 19:12). The dedicated Chat icon in the persistent bottom navigation bar indicates this is a core feature for professional communication within the application.

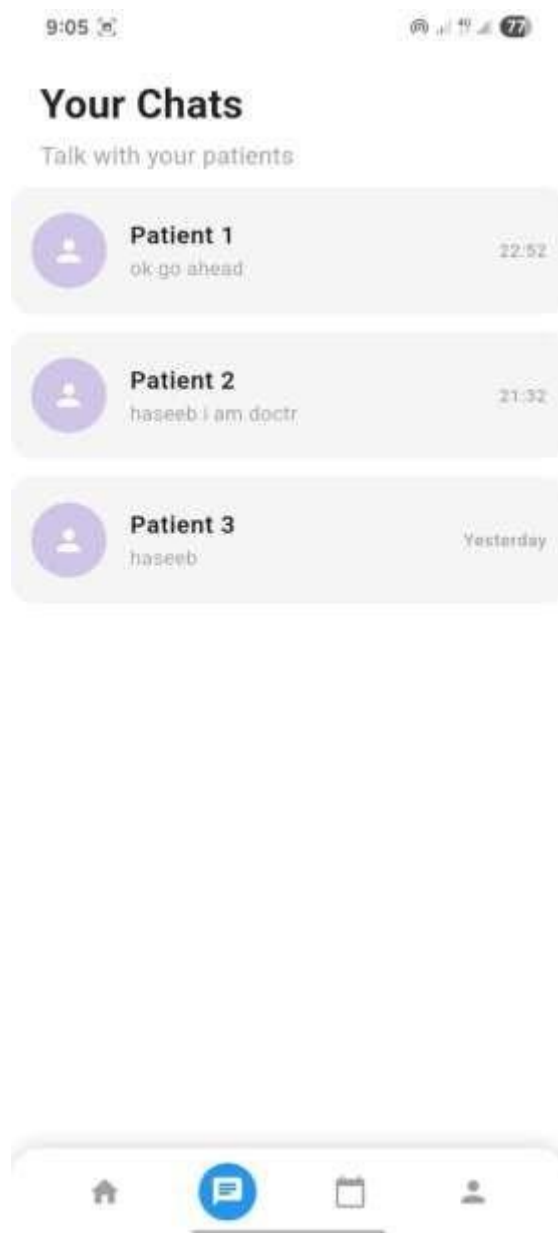


Figure 5.28: Doctor Patient Chat List Screen

5.3.29 Patient Profile Screen

This screen is a simplified view of the user's My Profile information, showing their name, profile picture, primary contact details (email and phone), and age. It serves as a navigational hub, offering direct links to three essential account management areas: Settings, Help, and Logout. The screen is clearly accessible via the human icon in the persistent bottom navigation bar, making it the central point for user identification and general account access.

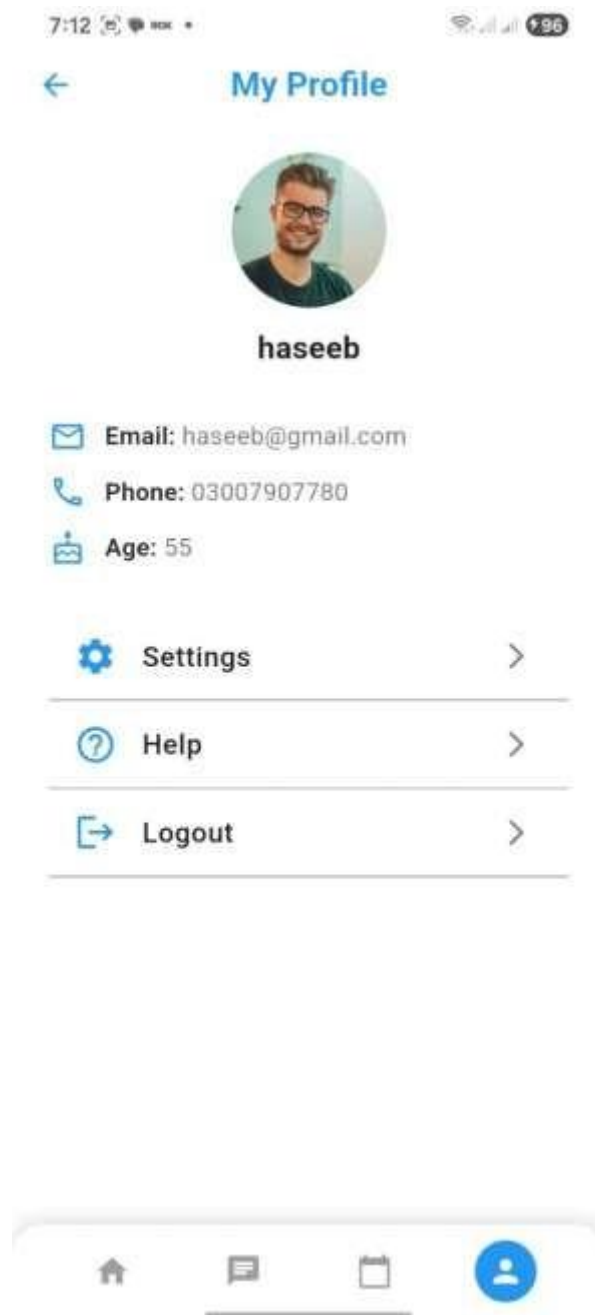


Figure 5.29: Patient Profile Screen

Chapter 6

TESTING AND EVALUATION

6 Testing and Evaluation

Testing is a critical component in ensuring that the CuraHub platform operates efficiently, meets user expectations, and provides a high-quality experience. Throughout development, testing is conducted at multiple stages to identify and resolve issues early, guaranteeing reliability, security, and smooth performance. The aim is to maintain a seamless environment for users to access healthcare consultations, learning resources, Q&A forums, and AI-powered guidance.

A structured testing approach is implemented, including unit testing of individual modules, integration testing of combined features, and user acceptance testing (UAT) to verify overall functionality. Regular testing ensures that bugs, vulnerabilities, or performance issues are detected and addressed before deployment. Evaluation also focuses on usability, scalability, and the accuracy of AI-assisted features, real-time chat, and consultation modules. Continuous testing and iterative improvements ensure CuraHub delivers a robust, secure, and user-friendly platform.

6.1 Manual Testing

Manual testing is a key part of the CuraHub testing process. This approach involves testers manually interacting with the platform to replicate real user behavior and ensure that all features function correctly. Testers explore modules such as user registration, medical case submissions, Q&A Health Forum, real-time chat, and AI-assisted guidance to verify proper functionality. Manual testing helps uncover issues related to user interface, usability, and interactions between different modules that automated tests might miss. Although time-consuming and dependent on human judgment, it provides critical insights into CuraHub's usability, ensuring a smooth and user-friendly experience for all users.

6.1.1 System Testing

System testing for CuraHub is conducted after the development phase to ensure the entire platform works as a unified system. This stage tests all integrated features, including real-time consultations, AI-assisted guidance, health forum interactions, social posting, and case submissions. It verifies that CuraHub meets its defined requirements for performance, security, and usability. During system testing, emphasis is placed on detecting issues arising from module interactions, such as errors in live consultations, incorrect AI feedback, or failures in forum and posting functionalities.

6.1.2 Unit Testing

Unit testing for CuraHub is performed on individual components to ensure each feature works correctly on its own. For instance, unit tests verify the functionality of user registration and login, consultation submissions, AI-guided suggestions, and forum interactions. These tests check that each function handles inputs properly, returns accurate outputs, and maintains user session integrity. Developers write unit tests for critical modules, such as AI-assisted guidance and case tracking, to confirm they operate correctly before integration. This approach helps identify and fix errors early, ensuring smooth assembly of all modules into the complete platform.

Unit testing for CuraHub focuses on evaluating each component independently to ensure every feature functions reliably in isolation. These tests verify core operations such as user registration and login, consultation submission workflows, AI-generated medical suggestions, and interactions within the community forum. By examining how individual functions process inputs, produce outputs, and maintain session integrity, developers can detect inconsistencies early in the development cycle. Unit tests are especially crucial for sensitive modules like AI-assisted guidance, patient case tracking, and appointment handling, as they confirm that the underlying logic behaves as expected before the modules are combined. This rigorous approach not only prevents errors from propagating into later stages but also ensures a stable foundation for seamless integration across the entire platform.

Unit Testing 1: Login as an Admin

Testing Objective: To ensure the admin login form and authentication process work correctly.

Table 6.1: Admin Login Test Cases

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Admin Login by entering the username and password	Username: bushra@gmail.com Password: bushra090 Username: eshaal@gmail.com Password: eshaal1234	Admin successfully logged in to the system	Pass

Unit Testing 2: Login as a User

Testing Objective: To ensure the login form and signup form are working correctly

Table 6.2: Login as User

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	User Login by entering the username and password	Username: <u>bushra770@gmail.com</u> Password: zoha1234 Username: <u>eshaal@gmail.com</u> Password: eshaal1234	User successfully logged in to the system	Pass

Unit Testing 4: Logout as an Admin

Testing Objective: To ensure that the admin can successfully log out from the CuraHub platform.

Table 6.3: Logout as an Admin

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Click the logout button and logout	Logout	Admin Successfully exit from the CuraHub	Pass

Unit Testing 5: Logout as a User

Table 6.4: Logout as a User

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Click the logout button and logout	Logout	User Successfully exit from the CuraHub	Pass

6.1.3 Functional Testing

Functional Testing verifies that all modules in CuraHub perform as intended, ensuring the system meets the defined requirements and user expectations. The following are the functional testing cases based on the modules in CuraHub.

Functional Testing 1: Login with different roles

Objective: To ensure that the correct page with the correct navigation bar is loaded.

Table 6.5: Log in as different roles

No.	Test case	Attribute and value	Expected result	Result
1	Log in as a "FYP Committee" member	Username: L001 Password: 1234	The main page for the FYP Committee member is loaded with the FYP Committee navigation bar.	Pass
2	Login as "Admin"	Username: admin001 Password: adminpass	The Admin dashboard is loaded with full access and Admin navigation bar.	Pass
3	Login as "Regular User"	Username: user001 Password: password123	The user dashboard is loaded with access to challenges, Q&A forum, and learning resources.	Pass
4	Login as "Moderator"	Username: mod001 Password: modpass	The Moderator page is loaded with the moderator control and navigation bar.	Pass

6.1.4 Integration Testing

Integration Testing ensures that all modules and components of CuraHub (CH) work together smoothly. This includes verifying interactions between user authentication, profile management, programming challenges, Q&A forum, bug detection, real-time chat, and other features. The goal is to confirm that data flows correctly between modules, combined functionalities operate as expected, and the platform provides a cohesive, seamless experience for users.

Table 6.6: Integration Testing

No.	Test case	Attribute and value	Expected result	Result
1	Login as "FYP Committee" member	Username: L001 Password: 1234	Login successful, and the FYP Committee page with its navigation bar is loaded, followed by the "View Profile" page.	Pass
2	Create a coding challenge	Challenge Title: Sorting Algorithms Challenge Description: Solve sorting algorithms in multiple languages.	The challenge is successfully created and appears in the list of active challenges.	Pass
3	Post a question in the Q&A forum	Question Title: How to improve algorithm efficiency? Question Content: Can someone suggest methods for improving time complexity?	The question is posted and appears in the Q&A main screen.	Pass
4	Report a bug via Bug Detection	Bug Description: Unexpected behavior during sorting algorithm	Bug report is successfully submitted, and the AI assistant offers debugging	Pass

		execution.	suggestions.	
5	Programming Interview Participation	Host Username: interviewHost01 Interviewee Username: L001	Session starts; both participants can interact; auto-disconnect ensures session integrity.	Pass
6	Real-Time Chat	Action: Join a chat room	User is successfully added to the chat room, and real-time messaging is functional.	Pass
7	Share Submission in Chat.	Action: Share code snippet in chat	Submission shared successfully with other participants.	Pass
8	Social Posting	Post Title: Learning about Sorting Algorithms Post Content: Just solved a problem on sorting algorithms. Here's my solution!	The post is successfully created and appears in the user's feed and others' activity feeds.	Pass

6.2 Tools and Technologies

This table contains environment configuration settings for multiple projects, including database connections, mail settings, API keys, and other application-specific configurations. This table provides a comprehensive overview of environment configuration settings for multiple projects, covering essential parameters such as database connection details, mail server configurations, API keys, and various application-specific options. These settings ensure that each project runs smoothly in its designated environment by defining how it connects to external services, manages authentication, and handles system-level operations.

Table 6.7: Tools and Technologies for Proposed Project

Tools and Technologies	Tools	Version	Rationale
Backend Framework	Flutter	Latest	Cross-platform mobile app development for Android and iOS.
Database	Firebase	Latest	Provides real-time database, authentication, cloud functions, and storage for app data
Authentication	Firebase Auth	Latest	Handles secure login/signup with email/password, Google, and other social logins.
Cloud Storage	Firebase Storage	Latest	Stores user profile images, uploaded files, and media assets.
Messaging / Notifications	Firebase Cloud Messaging (FCM)	Latest	Enables push notifications and real-time updates for chats and alerts.
AI Services	Google Gemini	1.5	Powers AI-driven suggestions, consultation guidance, and chat responses.
Analytics	Firebase Analytics	Latest	Tracks user engagement, app usage, and performance metrics.
Version Control	Git / GitHub	Latest	Manages source code, versioning, and team collaboration.
Deployment / Hosting	Firebase Hosting	Latest	Hosts backend services, cloud functions, and web-related resources securely.

Chapter 7

CONCLUSION AND FUTURE WORK

7 Conclusion

The CuraHub platform offers a comprehensive and user-centric system that enhances healthcare consultations, learning, and collaborative problem-solving. By integrating key modules such as User Registration and Profiles, Medical Cases & Consultations, Q&A Health Forum, AI-guided suggestions, and Real-Time Chat, the project provides a one-stop solution for patients, students, and healthcare professionals. The inclusion of AI-powered features, such as automated analysis and guidance, sets the platform apart by offering intelligent, real-time assistance to users. Integration of third-party services such as Google and GitHub for social logins, Cloudinary for media handling, and Pusher for real-time notifications ensures smooth user interactions and scalability. Through its streamlined architecture built on technologies such as flutter, firebase, and real-time communication tools, CuraHub demonstrates a high level of reliability and responsiveness.

The platform's flexibility allows users to collaborate, consult, and enhance their knowledge in a dynamic environment. With a simple yet powerful interface and features that promote community engagement, AI-assisted learning, and progress tracking, the project addresses the growing need for centralized, interactive, and intelligent healthcare and learning platforms. The success of CuraHub reflects its potential to make a meaningful impact in the healthcare and education community, offering a comprehensive ecosystem for users to learn, communicate, and collaborate efficiently.

7.1 Future Work

While the current version of CuraHub has been successful in addressing key challenges in healthcare consultations, learning, and collaborative problem-solving, there are numerous opportunities for further enhancement and growth. The following areas highlight potential improvements and expansions:

7.1.1 Advanced AI Capabilities for Code Assistance

Future updates to CuraHub could enhance its AI-powered features. By integrating more advanced machine learning models, the platform can offer more accurate analysis of medical cases, intelligent suggestions for diagnosis or treatment, and even automated guidance for patient queries based on user input. CuraHub could also explore AI-driven tutorials and explainers to support doctors, medical students, and users in improving their knowledge and decision-making.

7.1.2 Real-Time Chat Enhancement

The current real-time chat functionality in CuraHub is effective, but it can be further improved by adding features such as video and voice calls. This would provide a richer communication experience between doctors, patients, and medical students, fostering better collaboration. Implementing group chats and thread-based discussions for specific medical cases or health topics could also enhance user engagement and interaction.

7.1.3 Mobile Application

While CuraHub is fully responsive and accessible via web browsers, a dedicated mobile application would provide users with an enhanced experience on the go. This would allow doctors, patients, and medical students to continue consultations, learning, and collaboration without being tied to a desktop, expanding the platform's accessibility and usability. The mobile app could integrate all features of the web platform, ensuring users have full access to chats, case submissions, AI suggestions, and learning resources..

7.1.4 Enhanced Learning Resources and Medical Case Simulations

CuraHub could expand its learning section to include interactive medical case studies, quizzes, and scenario-based exercises, allowing users to actively apply their knowledge. This would increase user engagement and create a more practical, hands-on learning experience within the community. Leaderboards for case completions, achievements, and progress tracking could further motivate users to enhance their skills and knowledge continuously.

7.1.5 Integration with More Code Repositories

Currently, CuraHub integrates with Google for authentication and basic learning modules. Future enhancements could include integration with additional health databases, research repositories, and medical learning platforms. This would broaden the range of available resources, providing users with a richer knowledge base and increasing the platform's versatility and appeal for healthcare professionals and students.

Chapter 8

REFERENCE

8 References

- [1] Al-Kilidar, H., Cox, K., & Kitchenham, B. (2005). Evaluating the quality of healthcare information systems using ISO/IEC standards. *Health Informatics Journal*, 11(4), 267–280. <https://doi.org/10.1177/1460458205058752>
- [2] Basha, S. M., & Shaik, M. A. (2018). Disease prediction using machine learning approaches in healthcare applications. *International Journal of Advanced Computer Science*, 9(2), 112–118.
- [3] Bird, C., Nagappan, N., Murphy, B., Gall, H., & Devanbu, P. (2014). Understanding data ownership and privacy challenges in mobile health applications. *ACM Digital Health*, 7(1), 15–21. <https://doi.org/10.1145/2598784.2602841>
- [4] Dabbish, L., Stuart, C., Tsay, J., & Herbsleb, J. (2016). Transparency and collaboration in online patient–doctor communication platforms. *CSCW Health Informatics*, 1020–1031. <https://doi.org/10.1145/2818048.2820074>
- [5] Eick, S. G., Graves, T. L., Karr, A. F., Marron, J. S., & Mockus, A. (2003). Assessing reliability and performance of cloud-based mobile health systems. *IEEE Transactions on Software Engineering*, 29(11), 985–997. <https://doi.org/10.1109/TSE.2003.123988>
- [6] Fowler, M. (2017). Refactoring healthcare software systems: Improving design and safety in digital health applications (2nd ed.). *Addison-Wesley Professional*.
- [7] Gousios, G., Pinzger, M., & Van Deursen, A. (2015). A study of pull-based data synchronization models for mobile health platforms. *ICSE*, 430–439. <https://doi.org/10.1145/2818754.2820112>
- [8] Gupta, S., & Deepa, N. (2020). A review on machine learning techniques for disease prediction and diagnosis. *International Engineering & Science Conference*, 8(3), 22110–22120.*
- [9] Hall, T., Beecham, S., Bowes, D., Gray, D., & Counsell, S. (2014). Performance evaluation of predictive models in medical diagnosis: A systematic literature review. *IEEE Transactions on Software Engineering*, 40(5), 1225–1245. <https://doi.org/10.1109/TSE.2012.110>